

بِسْمِ اللَّهِ الرَّحْمَنِ
الرَّحِيمِ



دانشگاه شهید باهنر کرمان

دانشکده فنی و مهندسی
بخش مهندسی کامپیوتر

پایان نامه تحصیلی برای دریافت درجه کارشناسی ارشد
رشته مهندسی کامپیوتر گرایش هوش مصنوعی

تشخیص قدرتمند بدافزارهای اندروید با استفاده از
شبکه‌های عصبی ترنسفورمر

مؤلف:
علیرضا ایرانمنش

استاد راهنما:
دکتر حمید میروزی

خرداد ۱۴۰۴



دانشگاه شهید باهنر کرمان

دانشگاه شهید باهنر کرمان

دانشکده فنی و مهندسی

بخش مهندسی کامپیوتر

این پایان نامه با عنوان تشخیص قدرتمند بدافزارهای اندروید با استفاده از شبکه‌های عصبی ترنسفورمر توسط آقای علیرضا ایرانمنش دانشجوی رشته مهندسی کامپیوتر گرایش هوش مصنوعی و رباتیکز با شماره دانشجویی ۴۰۱۱۵۵۰۱۵ تدوین شده است و در تاریخ ۱۴۰۴/۰۳/۱۹ با درجه خوب و نمره ۱۷.۵ مورد پذیرش هیئت محترم داوران قرار گرفت.

این پایان نامه هیچگونه مدرکی به عنوان فراغت از تحصیل دوره کارشناسی ارشد شناخته نمی‌شود.

سمت	نام و نام خانوادگی	امضاء
استاد راهنما	دکتر حمید میروزی	
داور اول	دکتر محمد جواد رستمی	
داور دوم	دکتر وحید ستاری	

معاون آموزشی و پژوهشی دانشکده:

دکتر سعید شجاعی

امضاء

نماینده تحصیلات تکمیلی:

دکتر وحید جمشیدی

امضاء

حق چاپ محفوظ و مخصوص به دانشگاه شهید باهنر کرمان است.

به نام خدا

منور اخلاق پژوهش

با استعانت از خدای سبحان و اعتماد راسخ بر این که عالم مصغر خداست و او بواره ناغیر بر افعال ماست و به منظور انجام شایسته پژوهش های اصل، تولید دانش جدید و بسازی زندگیانی مشرود انجمنیان و اعضای ریاست علمی دانشگاه پژوهشگاه های کشور:

- ☐ تمام تلاش خود را برای کشف حقیقت و خطا حقیقت به کار خواهیم بست و از هر گونه جعل و تحریف و خفایت های علمی پرهیزی کنیم.
- ☐ حقوق پژوهشگران، پژوهیدگان (انسان، حیوان، نبات و اشیا)، سازمان دایر صاحبان حق را بر رعایت می نمایم و در حفظ آن می کوشیم.
- ☐ به کلیت بلای و ممنوی آثار پژوهشی ارجح می نسیم، برای انجام پژوهشی اصل اتمام در زنده و از سرعت علمی و ارجاع نامناسب اجتناب می کنیم.
- ☐ ضمن پابندی به انصاف و اجتناب از هر گونه تبعیض و تعصب و کینه خفایت های پژوهشی، ریهائی تعاون اتخاذ خواهیم کرد.
- ☐ ضمن امانت داری، از منابع و امکانات اقتصادی، انسانی و فنی موجود، استفاده بهر دور را خواهیم کرد.
- ☐ از انتشار غیر اخلاقی نتایج پژوهش، نظیر انتشار موازی، بهوشان و چندگانه دکلهای پرهیزی کنیم.
- ☐ اصل محرمانه بودن و رازداری را محور تمام خفایت های پژوهشی خود قرار می دهیم.
- ☐ در بهر خفایت های پژوهشی به منافع ملی توجه کرده و برای تحقق آن می کوشیم.
- ☐ خویش را ملزم به رعایت کلیه بهارهای علمی رسته خود، قوانین و مقررات، سیاست های حره ای، سازمانی، دولتی و راهبردی ملی و بهر مراحل پژوهشی می دانیم.
- ☐ رعایت اصول اخلاق در پژوهش را اقدامی فرهنگی می دانیم و به منظور بالندگی این فرهنگ، به ترویج و اسامه آن در جامعه اهتمام می ورزیم.



دانشگاه شهید باهنر کرمان

تعهدنامه

اینجانب علیرضا ایرانمنش به شماره دانشجویی ۴۰۱۱۵۵۰۱۵ دانشجوی مقطع کارشناسی ارشد رشته مهندسی کامپیوتر-هوش مصنوعی دانشکده فنی مهندسی دانشگاه شهید باهنر کرمان نویسنده پایان نامه با عنوان «تشخیص قدرتمند بدافزارهای اندروید با استفاده از شبکه های عصبی ترنسفورمر» تحت راهنمایی دکتر حمید میروزیری تأیید می کنم که این پایان نامه نتیجه پژوهش اینجانب می باشد و در عین حال که موضوع آن تکراری نیست، در صورت استفاده از منابع دیگران، نشانی دقیق و مشخصات کامل آن درج شده است. همچنین موارد زیر را نیز تعهد می کنم: ۱- برای انتشار تمام یا قسمتی از داده ها یا دستاوردهای خود در مجامع و رسانه های علمی اعم از همایش ها و مجلات داخلی و خارجی به صورت مقاله، کتاب، ثبت اختراع و به صورت مکتوب یا غیرمکتوب، با کسب مجوز از دانشگاه شهید باهنر کرمان و استاد(ان) راهنما اقدام نمایم.

۲- از درج اسامی افراد خارج از کمیته پایان نامه در جمع نویسندگان مقاله های مستخرج از پایان نامه، بدون مجوز استاد(ان) راهنما اجتناب نمایم و اسامی افراد کمیته پایان نامه را در جمع نویسندگان مقاله درج نمایم.

۳- از درج نشانی یا وابستگی کاری (affiliation) نویسندگان سازمان های دیگر (غیر از دانشگاه شهید باهنر کرمان) در مقاله های مستخرج از پایان نامه بدون تأیید استاد(ان) راهنما اجتناب نمایم^۱.

۴- کلیه ضوابط و اصول اخلاقی مربوط به استفاده از موجودات زنده یا بافتهای آنها را برای انجام پایان نامه رعایت نمایم.

۵- در صورت اثبات تخلف (در هر زمان) مدرک تحصیلی صادر شده توسط دانشگاه شهید باهنر کرمان از درجه اعتبار ساقط و اینجانب هیچ گونه ادعایی نخواهم داشت.

کلیه حقوق مادی و معنوی این اثر (مقالات مستخرج، برنامه های رایانه ای، نرم افزارها و تجهیزات ساخته شده) مطابق با آیین نامه مالکیت فکری، متعلق به دانشگاه شهید باهنر کرمان است و بدون اخذ اجازه کتبی از دانشگاه قابل واگذاری به شخص ثالث نیست. همچنین استفاده از اطلاعات و نتایج این پایان نامه بدون ذکر مرجع مجاز نمی باشد. چنانچه مبادرت به عملی خلاف این تعهدنامه محرز گردد، دانشگاه شهید باهنر کرمان در هر زمان و به هر نحو مقتضی حق هر گونه اقدام قانونی را در استیفای حقوق خود دارد.

تاریخ و امضا:
علیرضا ایرانمنش
۱۴۰۴ خرداد

^۱ تنها آدرس مورد قبول برای دانشگاه به این صورت می باشد:

Shahid Bahonar University of Kerman, Kerman, Iran.

نام و آدرس واحدهای دانشگاه در تولیدات علمی محققان دانشگاه به تشخیص بخش و دانشکده به شرح زیر می باشد:
Department of Computer, Faculty of Engineering Shahid Bahonar University of Kerman, Kerman, Iran.

آدرس صحیح جهت درج در مقالات و سایر تولیدات علمی فارسی:

گروه (بخش) کامپیوتر، دانشکده فنی مهندسی، دانشگاه شهید باهنر کرمان، کرمان، ایران.

تقدیم به:

«با نهایت احترام و سپاس، این پایان نامه را تقدیم می کنم به:

مهندس علیرضا افضلی پور و بانو فاخره صبا؛

دو انسان فرهیخته و نیک اندیش که با ایثار و آینده نگری، بنیان گذار دانشگاه شهید باهنر کرمان شدند و مسیر علم و دانش را در این دیار هموار ساختند.

آنها با وقف دارایی و زندگی خود، نهالی از دانش کاشتند که امروز به درختی تناور بدل شده و ثمرات آن در سراسر کشور نمایان است.

یاد و خاطره شان همواره الهام بخش نسل های آینده خواهد بود.»

تشکر و قدردانی:

با سپاس از خداوند بزرگ که به من توانایی و انگیزه برای پیمودن این مسیر علمی را عطا نمود. این پایان‌نامه حاصل تلاش و کوشش‌های فراوان است و بدون حمایت و راهنمایی‌های ارزشمند افراد بسیاری به ثمر نمی‌نشست.

به مصداق شعر «به یاد کسی که در این راه بود، به یاد کسی که در این راه رفت»، شایسته می‌دانم مراتب سپاس و قدردانی صمیمانه خود را تقدیم نمایم به استاد فرهیخته و فرزانه، جناب آقای دکتر حمید میروزی، که با دانش و تجربه‌ی خود همواره راهنمای من بودند و با صبر و شکیبایی به سوالات و ابهاماتم پاسخ دادند، صمیمانه تشکر می‌کنم.

همچنین از دوست عزیزم، محمدحسین شبانی، که با حمایت‌های بی‌دریغ و تشویق‌های همیشگی‌اش، انگیزه و انرژی مضاعفی به من بخشید، قدردانی می‌نمایم.

در پایان، از خانواده‌ی عزیزم که با عشق و محبت بی‌پایان خود همواره پشتیبان من بودند و در تمامی مراحل این مسیر پرچالش، همراه و همدل من بودند، بی‌نهایت سپاسگزارم.

چکیده:

با افزایش روزافزون تهدیدات سایبری، تشخیص بدافزارهای اندرویدی به یکی از چالش‌های اساسی در حوزه امنیت اطلاعات تبدیل می‌شود. روش‌های سنتی، به‌ویژه آن‌هایی که صرفاً بر تحلیل ویژگی‌های تک‌وجهی تکیه دارند، اغلب در پردازش داده‌های پیچیده چندوجهی ناتوان هستند و در مواجهه با تهدیدات جدید، از تعمیم‌پذیری مناسبی برخوردار نیستند. این محدودیت‌ها، ضرورت توسعه رویکردهای نوین و کارآمد را آشکار می‌سازند. در این پژوهش، مدلی چندوجهی با عنوان «تبدیل گر چندوجهی مبتنی بر جاسازی گراف دینامیک با توجه پویا (MAGNET)» توسعه می‌یابد که با ترکیب داده‌های جدولی، گراف و ترتیبی، از جمله توالی فراخوانی‌های API، به شناسایی بدافزارهای اندرویدی می‌پردازد. هدف اصلی، ارتقای دقت و پایداری تشخیص با بهره‌گیری از معماری پیشرفته مبتنی بر یادگیری عمیق و ترنسفورمر است. در این راستا، بهینه‌سازی هایپرپارامترها با استفاده از الگوریتم‌های پیشرفته‌ای مانند PIRATES و Optuna انجام می‌گیرد و مدل با مجموعه داده‌ای شامل ۴۶۴۱ نمونه آموزشی و ۱۴۵۱ نمونه آزمایشی، همراه با اعتبارسنجی متقاطع پنج‌تایی، آموزش می‌بیند. ویژگی‌های مورد استفاده شامل ویژگی‌های ایستا نظیر مجوزها، فراخوانی‌های API، مقاصد و نام مؤلفه‌ها و همچنین ویژگی‌های پویا مانند فعالیت شبکه و دسترسی به فایل‌ها هستند. داده‌ها به صورت بردارهای عددی باینری یا نرمال‌سازی شده آماده‌سازی می‌شوند و پس از پیش‌پردازش، ابعاد ویژگی‌ها به ۴۳۰ ویژگی تنظیم می‌گردند. ابزارهای مورد استفاده شامل کتابخانه‌های یادگیری عمیق مانند PyTorch، تکنیک‌های استانداردسازی و نرمال‌سازی داده‌ها و ساختارهای داده‌ای گرافی هستند. نتایج نشان می‌دهند که مدل پیشنهادی عملکردی برجسته با دقت بالا، پایداری قابل توجه و قابلیت تعمیم‌پذیری مطلوب ارائه می‌دهد و نسبت به روش‌های پیشین بهبود قابل ملاحظه‌ای دارد. این یافته‌ها، پتانسیل کاربرد مدل در سیستم‌های امنیتی واقعی را برجسته می‌سازند.

واژگان کلیدی: تشخیص بدافزار، ترنسفورمر، یادگیری عمیق، داده‌های چندوجهی، امنیت اندروید.

فهرست مطالب

صفحه

عنوان

۱	فصل اول: کلیات پژوهش
۱-۱	مقدمه و بیان مسئله
۱-۱-۱	روش های تشخیص بدافزار
۱-۱-۲	مجموعه داده های مربوطه
۲-۱	ضرورت تحقیق و اهداف
۳-۱	سازماندهی پایان نامه
۶	فصل دوم: پیشینه تحقیق و مفاهیم پایه
۱-۲	مقدمه
۲-۲	مفاهیم پایه
۱-۲-۲	تکامل روش های یادگیری ماشین
۲-۲-۲	انواع بدافزار
۱-۲-۲-۲	باج افزار
۲-۲-۲-۲	تروجان
۳-۲-۲-۲	جاسوس افزار
۴-۲-۲-۲	تبلیغ افزار
۳-۲-۲	مفاهیم مرتبط با اپلیکیشن های اندرویدی
۱-۳-۲-۲	مجوزهای دسترسی
۲-۳-۲-۲	فایل APK
۳-۳-۲-۲	سورس کد
۴-۲-۲	داده های چندوجهی در تشخیص بدافزار
۱-۴-۲-۲	داده های جدولی
۲-۴-۲-۲	داده های گرافی
۳-۴-۲-۲	داده های ترتیبی
۴-۴-۲-۲	اهمیت و یکپارچگی داده های چندوجهی
۵-۲-۲	مدل های یادگیری عمیق
۱-۵-۲-۲	شبکه های عصبی کانولوشنی (CNN)
۲-۵-۲-۲	شبکه های عصبی بازگشتی و واحدهای حافظه بلند-کوتاه
۱۶	(RNN و LSTM)

۱۷	شبکه‌های عصبی گرافی (GNN)	۳-۵-۲-۲
۱۸	ترنسفورمرها	۶-۲-۲
	معرفی ترنسفورمر و تحول در مدل‌های یادگیری عمیق	۱-۰-۶-۲-۲
۱۹	ترنسفورمرها در مدل MAGNET	۷-۲-۲
۲۱	ماشین بردار پشتیبان (SVM)	۸-۲-۲
۲۱	انواع SVM	۱-۰-۸-۲-۲
۲۲	کاربرد در تشخیص بدافزار	۲-۰-۸-۲-۲
۲۲	مزایا و معایب	۳-۰-۸-۲-۲
۲۲	تکنیک‌های آموزشی و ارزیابی	۳-۲
۲۲	اعتبارسنجی متقاطع (Cross-Validation)	۱-۳-۲
۲۳	مدیریت بیش‌برازش و کم‌برازش	۲-۳-۲
۲۵	روش‌های بهینه‌سازی	۳-۳-۲
۲۵	الگوریتم Adam	۱-۳-۳-۲
۲۵	الگوریتم PIRATES	۲-۳-۳-۲
۲۶	الگوریتم Optuna	۳-۳-۳-۲
۲۷	معیارهای ارزیابی عملکرد	۴-۳-۲
۲۷	دقت (Accuracy)	۱-۴-۳-۲
۲۸	F1 Score	۲-۴-۳-۲
۲۹	AUC Area Under the ROC Curve	۳-۴-۳-۲
۲۹	جمع‌بندی ارزیابی	۱-۳-۴-۳-۲
۳۰	مروری بر مطالعات پیشین	۴-۲
۳۰	روش‌های تحلیل ایستا	۱-۴-۲
۳۰	تحلیل ویژگی‌های مبتنی بر مانیفست و متاداده	۱-۱-۴-۲
۳۰	تحلیل کد (Code Analysis)	۲-۱-۴-۲
۳۱	تحلیل ایستای ساختاری	۳-۱-۴-۲
۳۱	محدودیت‌های تحلیل ایستا	۱-۳-۱-۴-۲
۳۲	روش‌های تحلیل پویا	۲-۴-۲
۳۲	مانیتورینگ رفتار سیستم	۱-۲-۴-۲
۳۲	تحلیل شبکه	۲-۲-۴-۲
۳۲	بررسی مصرف منابع	۳-۲-۴-۲
۳۳	ابزارها و محیط‌های تحلیل پویا	۱-۳-۲-۴-۲

۳۳	 محدودیت‌های تحلیل پویا	۲-۳-۲-۴-۲
۳۳	 Hybrid Approaches روش‌های ترکیبی	۴-۲-۴-۲
۳۴	 کارهای پیشین و تاریخچه مختصر	۳-۴-۲
۳۴	 جمع‌بندی فصل	۵-۲

فصل سوم: مطالعه موردی: شرکت‌های پیشرو در امنیت اندروید

۳۶	 هدف فصل	۱-۳
۳۷	 معیارهای انتخاب	۲-۳
۳۷	 مطالعه موردی شرکت‌ها	۳-۳
۳۷	 NowSecure	۱-۳-۳
۳۷	 Qualys (VM DR Mobile)	۲-۳-۳
۳۷	 Checkmarx / Synopsys	۳-۳-۳
۳۸	 روش‌ها و ابزارهای مشترک	۴-۳
۳۸	 مقایسه و تحلیل	۵-۳
۳۸	 جمع‌بندی	۶-۳

فصل چهارم: روش پیشنهادی

۴۱	 روش پیشنهادی	۱-۴
۴۱	 مقدمه روش پیشنهادی	۱-۱-۴
۴۲	 فرضیات و ابزارهای محاسباتی	۲-۱-۴
۴۲	 فرضیات	۱-۲-۱-۴
۴۳	 ابزارهای محاسباتی	۲-۲-۱-۴
۴۴	 روش‌شناسی	۳-۱-۴
۴۴	 پیش‌پردازش داده‌ها	۱-۳-۱-۴
۴۵	 طراحی مدل MAGNET	۲-۳-۱-۴
۴۵	 EnhancedTabTransformer (ماژول جدولی)	۱-۲-۳-۱-۴
۴۶	 GraphTransformer (ماژول گرافی)	۲-۲-۳-۱-۴
۴۷	 SequenceTransformer (ماژول ترتیبی)	۳-۲-۳-۱-۴
۴۷	 مکانیزم توجه پویا	۴-۲-۳-۱-۴
۴۷	 ادغام چندوجهی	۵-۲-۳-۱-۴
۴۸	 مدل نهایی MAGNET	۶-۲-۳-۱-۴
۴۸	 آموزش مدل	۳-۳-۱-۴

۴۹	بهینه‌سازی ابرپارامترها	۴-۳-۱-۴
۴۹	ارزیابی مدل	۵-۳-۱-۴
۴۹	جمع‌بندی روش پیشنهادی	۴-۱-۴

فصل پنجم: نتایج و بحث

۵۲		
۵۳	مقدمه	۱-۵
۵۳	تنظیمات آزمایشی	۲-۵
۵۳	ویژگی‌های داده	۱-۲-۵
۵۴	پیکربندی آزمایش‌ها	۲-۲-۵
۵۵	تحلیل فرآیند بهینه‌سازی با الگوریتم Pirates	۳-۵
۵۵	نمایش موقعیت کشتی‌ها و رهبر	۱-۳-۵
۵۵	روند تغییر هزینه و همگرایی	۲-۳-۵
۵۶	تحلیل پویایی جمعیت: باد، سرعت و شتاب	۳-۳-۵
۵۷	جمع‌بندی تحلیل فرآیند بهینه‌سازی	۴-۳-۵
۵۸	تحلیل حساسیت پارامترهای الگوریتم	۴-۵
۵۸	تحلیل حساسیت پارامترهای معماری مدل	۱-۴-۵
۵۸	تحلیل حساسیت پارامترهای آموزش	۲-۴-۵
۵۹	مقایسه حساسیت بین الگوریتم‌های بهینه‌سازی	۳-۴-۵
۵۹	توصیه‌های عملی	۴-۴-۵
۶۰	مدل‌های پایه	۵-۴-۵
۶۱	محیط اجرا	۶-۴-۵
۶۱	معیارهای ارزیابی	۵-۵
۶۲	نتایج کلی مدل MAGNET	۶-۵
۶۲	ماتریس درهم‌ریختگی و عملکرد به تفکیک کلاس	۱-۶-۵
۶۳	نتایج اعتبارسنجی متقاطع	۲-۶-۵
۶۳	تحلیل عملکرد کلی مدل در مراحل مختلف	۳-۶-۵
۶۵	مقایسه با روش‌های پایه و پیشرفته	۷-۵
۶۶	مقایسه با روش‌های پایه	۱-۷-۵
۶۶	مقایسه با روش‌های پیشرفته	۲-۷-۵
۶۷	تحلیل مقایسه با روش‌های پیشرفته	۳-۷-۵
۶۸	نتایج مقایسه با روش‌های پایه	۴-۷-۵

۶۹	تحلیل عملکرد ماژول‌های مدل	۵-۷-۵
۶۹	مطالعه حذف اجزا	۶-۷-۵
۶۹	روند آموزش	۷-۷-۵
۷۰	جمع‌بندی	۸-۵

فصل ششم: نتیجه‌گیری و پیشنهادات آتی

۷۳		
۷۴	نتیجه‌گیری	۱-۶
۷۵	پیشنهادات آتی	۲-۶
۷۵	پژوهش‌های تکمیلی	۱-۲-۶
۷۶	پیشنهادات اجرایی	۲-۲-۶
۷۶	تولید داده‌های جدید	۳-۲-۶
۷۶	تحلیل‌های آینده	۴-۲-۶

مراجع

۷۹

پیوست

۸۳		
۸۴	پیوست A: کدهای پیاده‌سازی مدل MAGNET	۱-
۸۴	کد معماری مدل MAGNET	۱-۱-
۸۵	کد بهینه‌سازی با PIRATES	۲-۱-
۸۶	پیوست B: داده‌های خام و پیش‌پردازش	۲-
۸۶	نمونه داده‌های خام DREBIN	۱-۲-
۸۶	توضیحات پیش‌پردازش	۲-۲-
۸۷	پیوست C: جزئیات سخت‌افزاری و نرم‌افزاری	۳-
۸۷	مشخصات سخت‌افزاری	۱-۳-
۸۷	مشخصات نرم‌افزاری	۲-۳-
۸۷	پیوست D: نتایج اضافی و ماتریس‌های کامل	۴-
۸۷	ماتریس درهم‌ریختگی کامل	۱-۴-
۸۸	گزارش طبقه‌بندی برای هر دسته	۲-۴-

فهرست جداول

عنوان	صفحه
جدول ۱-۵ نتایج اعتبارسنجی مقاطع 5-تایی مدل MAGNET	۶۳
جدول ۲-۵ مقایسه کلی عملکرد مدل MAGNET در مراحل مختلف	۶۳
جدول ۳-۵ مقایسه عملکرد مدل MAGNET با روش های پایه و پیشرفته	۶۷
جدول ۴-۵ مقایسه عملکرد مدل MAGNET با مدل های پایه	۶۹
جدول ۱ نمونه ای از داده های خام دیتاست DREBIN	۸۶
جدول ۲ ماتریس درهم ریختگی برای مجموعه تست	۸۸
جدول ۳ گزارش طبقه بندی برای هر دسته در اعتبارسنجی مقاطع	۸۸

فهرست اشکال

صفحه

عنوان

شکل ۱-۲	ساختار کلی شبکه عصبی کانولوشنی (CNN) شامل لایه‌های کانولوشنی، تجمعی و کاملاً متصل برای طبقه‌بندی. برگرفته از [۹].	۱۶
شکل ۲-۲	ساختار شبکه عصبی گرافی (GNN) برای پردازش داده‌های گرافی و طبقه‌بندی. برگرفته از [۱۲].	۱۸
شکل ۳-۲	معماری ترنسفورمر شامل کدگذار و گشاینده با مکانیزم توجه چندسری. برگرفته از [۱۵].	۲۰
شکل ۱-۴	معماری کلی مدل MAGNET.	۴۵
شکل ۱-۵	نمایش اجزای الگوریتم PIRATES در فضای جستجو.	۵۵
شکل ۲-۵	روند همگرایی الگوریتم PIRATES.	۵۶
شکل ۳-۵	تغییرات پارامترهای الگوریتم PIRATES در طول تکرارها.	۵۷
شکل ۴-۵	نتایج اعتبارسنجی متقاطع ۵-تایی مدل MAGNET.	۶۴
شکل ۵-۵	تغییرات زیان در اعتبارسنجی متقاطع ۵-تایی.	۶۴
شکل ۶-۵	مقایسه عملکرد مدل MAGNET در مراحل مختلف.	۶۵
شکل ۷-۵	مقایسه دقت روش‌های مختلف.	۶۸
شکل ۸-۵	مقایسه F1 Score و AUC روش‌های مختلف.	۶۸
شکل ۹-۵	مقایسه F1 Score و AUC مدل MAGNET با سایر مدل‌ها.	۶۹
شکل ۱۰-۵	عملکرد ماژول‌های مختلف مدل MAGNET.	۷۱
شکل ۱۱-۵	تأثیر مکانیزم توجه پویا و لایه ادغام چندوجهی بر عملکرد.	۷۱
شکل ۱۲-۵	روند آموزش مدل در طول ۳ دوره.	۷۲

فهرست الگوریتم‌ها

صفحه	عنوان
۴۶	EnhancedTabTransformer Module Structure
۴۶	GraphTransformer Module Structure
۴۷	SequenceTransformer Module Structure
۴۸	Dynamic Attention Mechanism
۴۸	Modality Fusion
۴۸	Final MAGNET Model

فصل اول:
کلیات پژوهش

۱-۱ مقدمه و بیان مسئله

در سال‌های اخیر، گسترش تلفن‌های همراه و به‌ویژه سیستم عامل اندروید^۱، موجب افزایش وابستگی کاربران به این ابزارها شده است. این دستگاه‌ها نه تنها در زندگی روزمره، بلکه در حوزه‌های تجاری و نظامی نیز نقش مهمی ایفا می‌کنند. با این حال، محبوبیت و فراگیری اندروید، آن را به هدفی جذاب برای حملات بدافزاری^۲ تبدیل کرده است. عرضه نرم‌افزارهای غیرمعتبر و تهدیداتی مانند ویروس‌ها و بدافزارها، امنیت کاربران را به خطر انداخته است. مطالعات اخیر نشان می‌دهد که بیش از 70 درصد دستگاه‌های هوشمند از سیستم عامل اندروید استفاده می‌کنند و این امر باعث شده است که این پلتفرم به هدف اصلی حملات امنیتی تبدیل شود [۱]. با وجود پیشرفت‌های قابل توجه در روش‌های تشخیص بدافزار، همچنان چالش‌های جدی در شناسایی بدافزارهای جدید و پیچیده وجود دارد.

در ابتدا، روش‌های سنتی مبتنی بر تحلیل مجوزها^۳ و باز کردن فایل‌ها مورد استفاده قرار می‌گرفتند که به دلیل دقت پایین و ضعف در شناسایی بدافزارهای پیچیده، محدودیت‌هایی داشتند. پژوهش‌های اخیر نشان داده‌اند که روش‌های مبتنی بر یادگیری ماشین^۴ و یادگیری عمیق^۵ می‌توانند عملکرد بهتری در تشخیص بدافزارها داشته باشند [۲]. با این حال، همچنان چالش‌های مهمی در زمینه تفسیرپذیری مدل‌ها^۶ و قابلیت تعمیم‌پذیری^۷ وجود دارد. این چالش‌ها به ویژه در مواجهه با بدافزارهای جدید و ناشناخته^۸ بیشتر خود را نشان می‌دهند.

مدل MAGNET^۹ که در این پژوهش معرفی شده است، با بهره‌گیری از معماری ترنسفورمر^{۱۰} چندوجهی و ترکیب داده‌های جدولی، گراف و توالی، تلاش می‌کند تا این چالش‌ها را برطرف کند. این مدل با استفاده از مکانیزم‌های توجه پویا^{۱۱} و تحلیل همزمان داده‌های مختلف، قادر به تشخیص دقیق‌تر بدافزارها خواهد بود. نتایج نشان می‌دهد که این رویکرد با دقت $97.24\% \pm 0.5\%$ ، معیار $F1 = 0.9823 \pm 0.002$ ، و معیار $AUC = 0.9932 \pm 0.003$ عملکرد بهتری نسبت به مدل‌های پایه مانند SVM (دقت 90.6%)، CNN (دقت 92.8%) و LSTM (دقت 91.5%) دارد.

¹ Android

² Malware

³ Permissions

⁴ Machine Learning

⁵ Deep Learning

⁶ Model Interpretability

⁷ Generalization

⁸ Zero-Day

⁹ MAGNET (Multi-Modal Analysis for Graph and Network Threat Detection)

¹⁰ Transformer

¹¹ Attention Mechanism

۱-۱-۱ روش‌های تشخیص بدافزار

تشخیص بدافزارهای اندرویدی به دو روش کلی پویا^۱ و ایستا^۲ انجام می‌شود. در روش پویا، رفتار اپلیکیشن در زمان اجرا مانند مصرف باتری، پردازنده یا ترافیک شبکه بررسی می‌شود تا الگوهای غیرعادی شناسایی گردد. این روش به تنهایی کافی نیست و ممکن است برخی تهدیدات پنهان را نادیده بگیرد. روش ایستا با تحلیل ساختار و کد اپلیکیشن، مانند بررسی فراخوانی‌های API^۳ و مجوزها، اطلاعات ارزشمندی ارائه می‌دهد که می‌تواند در تشخیص دقیق‌تر کمک کند. پژوهش‌های اخیر نشان داده‌اند که ترکیب این دو روش می‌تواند نتایج بهتری در تشخیص بدافزارها ارائه دهد [۳].

۲-۱-۱ مجموعه داده‌های مربوطه

در حوزه تشخیص بدافزار اندروید، مجموعه داده‌های متنوعی برای ارزیابی عملکرد مدل‌ها مورد استفاده قرار گرفته‌اند. از جمله این مجموعه داده‌ها می‌توان به موارد زیر اشاره کرد:

- مجموعه داده‌های Drebin [۴] و AndroZoo [۵] که شامل نمونه‌های گسترده‌ای از بدافزارها و برنامه‌های سالم اندرویدی هستند.
- مجموعه داده‌های CICMalDroid [۶] و VirusShare که شامل نمونه‌های جدید و به‌روز از بدافزارها می‌باشند.
- مجموعه داده‌های خصوصی و صنعتی که توسط شرکت‌های امنیتی و مراکز تحقیقاتی گردآوری شده‌اند.

این مجموعه داده‌ها به عنوان شاخص‌های استاندارد، امکان ارزیابی دقیق و جامع عملکرد الگوریتم‌های تشخیص بدافزار را فراهم می‌کنند و نقش مهمی در اثبات قابلیت تعمیم و کارایی روش‌های پیشنهادی دارند.

۲-۱-۲ ضرورت تحقیق و اهداف

پلتفرم اندروید به دلیل محبوبیت گسترده و سهم عظیمش از بازار جهانی، به هدف اصلی بدافزارها و حملات امنیتی تبدیل شده است. این سیستم عامل، که بیش از 70 درصد دستگاه‌های هوشمند را پشتیبانی می‌کند، به دلیل ساختار باز و دسترسی پذیری بالا، با تهدیدات پیشرفته‌ای مواجه است. بدافزارهای

¹Dynamic Analysis

²Static Analysis

³API

اندرویدی، از جمله تروجان‌ها^۱، جاسوس‌افزارها^۲ و باج‌افزارها^۳، با روش‌های پیچیده‌ای طراحی شده‌اند و پیشرفت‌های چشمگیری داشته‌اند. این تهدیدات، از سرقت اطلاعات حساس گرفته تا ایجاد اختلال در عملکرد دستگاه‌ها، چالش‌های امنیتی جدی ایجاد کرده‌اند. از این رو، نیاز به سیستمی قدرتمند و کارآمد برای تشخیص بدافزارهای اندرویدی بیش از پیش احساس می‌شود. هدف اصلی این پژوهش، تمرکز بر شناسایی بدافزارهای ناشناخته و نادیده^۴ است که تا کنون شناسایی نشده‌اند و می‌توانند تهدیداتی پنهان برای کاربران ایجاد کنند.

با توجه به چالش‌ها و نیازهای مطرح شده، اهداف اصلی این تحقیق بدین شرح می‌باشد:

- توسعه یک مدل چندوجهی پیشرفته با نام MAGNET که قادر به تحلیل همزمان داده‌های جدولی، گراف‌ی و ترتیبی باشد.
- بهبود دقت تشخیص بدافزارهای اندرویدی با استفاده از معماری ترنسفورمر و مکانیزم‌های توجه پویا^۵.
- کاهش نرخ خطای تشخیص و افزایش قابلیت تعمیم‌پذیری مدل در مواجهه با بدافزارهای جدید.
- بهینه‌سازی مصرف منابع محاسباتی و افزایش سرعت تشخیص با استفاده از الگوریتم‌های پیشرفته.
- ایجاد یک چارچوب استاندارد برای ارزیابی و مقایسه روش‌های مختلف تشخیص بدافزار.

۳-۱ سازماندهی پایان‌نامه

در این پایان‌نامه، ساختار مطالب به گونه‌ای تدوین شده که مسیر پژوهش از مبانی نظری و معرفی مسئله تا ارائه نتایج تجربی به صورت پیوسته و منطقی دنبال شود. به عبارت دیگر، هدف از سازماندهی مطالب این است که خواننده بتواند به راحتی با مباحث پایه، چالش‌ها، روش‌های موجود و نوآوری‌های پیشنهادی آشنا شود و در نهایت به درک جامع از دستاوردهای تحقیق دست یابد. ساختار کلی پایان‌نامه به شرح زیر است:

• فصل ۲ - پیشینه تحقیق و مفاهیم پایه:

¹Trojan

²Spyware

³Ransomware

⁴Zero-Day

⁵Dynamic Attention Mechanism

در این فصل، ابتدا به بررسی کلی امنیت اندروید و اهمیت تشخیص بدافزار پرداخته می‌شود. سپس، چالش‌ها و محدودیت‌های روش‌های سنتی بیان شده و مسئله تحقیق به تفصیل معرفی می‌شود. هدف این فصل ایجاد زمینه نظری مناسب برای درک اهمیت تشخیص خودکار بدافزارهاست.

در ادامه به بررسی جامع مطالعات پیشین در حوزه تشخیص بدافزار اندروید پرداخته می‌شود. در این بخش، رویکردهای مختلف از جمله روش‌های مبتنی بر یادگیری ماشین و یادگیری عمیق مورد تحلیل قرار می‌گیرند. نقاط قوت و ضعف هر یک از این رویکردها همراه با چالش‌های موجود در هر کدام به تفصیل بررسی می‌شود.

• فصل ۳ - روش پیشنهادی (MAGNET):

در این فصل، مدل پیشنهادی MAGNET به صورت کامل تشریح می‌شود. ابتدا معماری کلی مدل و اجزای اصلی آن معرفی می‌شوند. سپس، جزئیات پیاده‌سازی و الگوریتم‌های بهینه‌سازی مورد استفاده توضیح داده می‌شود. در نهایت، نوآوری‌های اصلی این روش نسبت به سایر روش‌ها برجسته می‌شود.

• فصل ۴ - نتایج و بحث:

این فصل به ارائه نتایج آزمایش‌های انجام شده بر روی چندین مجموعه داده معتبر اختصاص دارد. عملکرد مدل MAGNET از نظر دقت، کارایی و صرفه‌جویی در منابع محاسباتی مورد مقایسه قرار گرفته و نتایج به دست آمده تحلیل می‌شوند.

• فصل ۵ - نتیجه‌گیری و پیشنهادات آتی:

در فصل نهایی، یافته‌های اصلی تحقیق به طور خلاصه ارائه شده و به نتیجه‌گیری کلی از دستاوردهای پژوهش پرداخته می‌شود. در این بخش، چالش‌های باقی‌مانده، محدودیت‌های تحقیق و نیز پیشنهاداتی جهت تحقیقات آتی و بهبود رویکرد ارائه می‌شود.

فصل دوم:

پیشینه تحقیق و مفاهیم پایه

در سال‌های اخیر، با گسترش روزافزون استفاده از دستگاه‌های هوشمند مبتنی بر سیستم عامل اندروید، امنیت سایبری به یکی از دغدغه‌های اصلی کاربران و سازمان‌ها تبدیل شده است. محبوبیت گسترده اندروید، که طبق آمار Statista در سال ۲۰۲۴ بیش از ۷۰ درصد بازار جهانی دستگاه‌های هوشمند را در اختیار دارد، آن را به هدف اصلی حملات بدافزاری، از جمله تروجان‌ها^۱، جاسوس افزارها^۲ و باج افزارها^۳، مبدل ساخته است [۱]. این تهدیدات، با بهره‌گیری از روش‌های پیچیده و نوظهور، چالش‌های جدی در شناسایی و خنثی‌سازی ایجاد کرده‌اند، به‌ویژه در مورد بدافزارهای ناشناخته (Zero-Day)^۴ که تشخیص آن‌ها با روش‌های سنتی مبتنی بر امضا دشوار است [۳]. از این رو، توسعه مدل‌های پیشرفته مبتنی بر یادگیری عمیق و داده‌های چندوجهی، به‌عنوان راهکاری نوین برای مقابله با این تهدیدات، مورد توجه فزاینده‌ای قرار گرفته است [۲].

فصل حاضر به‌عنوان پایه‌ای برای درک روش پیشنهادی این پژوهش، به بررسی مفاهیم پایه و مرور مطالعات پیشین در حوزه تشخیص بدافزارهای اندرویدی می‌پردازد. ابتدا، مفاهیم کلیدی و اصطلاحات فنی مورد نیاز معرفی خواهند شد. سپس، تکنیک‌های رایج یادگیری ماشین و یادگیری عمیق که در این حوزه کاربرد دارند، تشریح می‌شوند. در ادامه، ابزارها و روش‌های بهینه‌سازی مانند ترنسفورمرها^۵، الگوریتم‌های PIRATES و Optuna^۶، که در طراحی مدل MAGNET (روش پیشنهادی این پژوهش) به کار رفته‌اند، معرفی خواهند شد. همچنین، تکنیک‌های آموزشی و معیارهای ارزیابی عملکرد، نظیر اعتبارسنجی متقاطع^۷ و معیارهای دقت^۸، AUCF1 Score^۹، به‌منظور تبیین چارچوب ارزیابی مدل توضیح داده می‌شوند.

در بخش بعدی، پیشینه تحقیق با تمرکز بر روش‌های تحلیل ایستا^{۱۰} و پویا^{۱۱}، که پایه‌های اصلی تشخیص بدافزار را تشکیل می‌دهند، مورد بررسی قرار می‌گیرد. این مرور، با تحلیل نقاط قوت و ضعف روش‌های پیشین، زمینه‌ای برای درک نوآوری‌های مدل پیشنهادی فراهم می‌کند. هدف این فصل، ارائه دیدگاهی جامع و منسجم از مفاهیم و مطالعات مرتبط است تا خواننده را برای درک بهتر روش شناسی و نتایج پژوهش آماده سازد. این ساختار، نه تنها به تبیین مسائل نظری کمک می‌کند، بلکه راه را برای

¹Trojan

²Spyware

³Ransomware

⁴Zero-Day

⁵Transformer

⁶Optuna

⁷Cross-Validation

⁸Accuracy

⁹Area Under Curve (AUC)

¹⁰Static Analysis

¹¹Dynamic Analysis

مقایسه و ارزیابی عملکرد مدل MAGNET هموار می سازد.

۲-۲ مفاهیم پایه

در این بخش، مفاهیم بنیادی مرتبط با موضوع پژوهش، از جمله تاریخچه مختصری از یادگیری ماشین، انواع بدافزارها، ساختار اپلیکیشن های اندروید، داده های چندوجهی و مدل های یادگیری عمیق مرتبط، تشریح می شوند.

۱-۲-۲ تکامل روش های یادگیری ماشین

در دهه های ۱۹۵۰ و ۱۹۶۰، اولین تلاش ها برای پردازش زبان طبیعی به وسیله ی روش های نمادین انجام شد. پژوهشگران در آن زمان سعی می کردند ساختارهای دستوری و قوانین زبان را به صورت صریح و دستی تعریف کنند. این رویکردها با وجود تلاش های ارزشمند، به دلیل محدودیت های محاسباتی و عدم وجود داده های کافی، نتوانستند به دقت و کارایی مورد انتظار دست یابند.

با گذر زمان و ورود به دهه های ۱۹۶۰ و ۱۹۷۰، رویکردهای آماری جایگزین بخش هایی از روش های نمادین شدند. در این دوران، مدل های n-gram که بر مبنای احتمال وقوع یک کلمه با توجه به کلمات قبلی محاسبه می شدند، به عنوان اولین قدم های موفق در مدلسازی زبان مطرح شدند. اگرچه این مدل ها ساده بودند، اما توانستند برخی از پیچیدگی های اولیه ی پردازش زبان را کاهش دهند.

در دهه های ۱۹۸۰ و ۱۹۹۰، با پیشرفت های چشمگیر در فناوری های محاسباتی و افزایش دسترسی به داده های متنی، روش های یادگیری ماشین وارد عرصه شدند. الگوریتم های یادگیری نظارت شده و غیرنظارتی به منظور تشخیص الگوهای زبانی به کار گرفته شدند. با این حال، محدودیت های موجود همچنان مانع از دستیابی به درک عمیق تر و تولید متن های طبیعی به سطح امروزی می شدند.

ورود به قرن ۲۱ و به ویژه دهه ۲۰۱۰، با ظهور شبکه های عصبی عمیق مانند شبکه های عصبی کانولوشنی^۱، شبکه های عصبی بازگشتی^۲ و شبکه های حافظه بلندمدت-کوتاه مدت^۳ همراه بود. این مدل ها توانستند وابستگی های زمانی، الگوهای تصویری و روابط بلندمدت موجود در داده ها را بهتر مدل سازی کنند [۷]. با این حال، چالش هایی همچنان در زمینه بهبود کیفیت و کارایی تحلیل داده های پیچیده مانند کدهای بدافزار وجود داشت که منجر به توسعه معماری های پیشرفته تری مانند ترنسفورمرها شد.

^۱ Convolutional Neural Networks (CNNs)

^۲ Recursive Neural Networks (RNNs)

^۳ Long Short-Term Memory (LSTM)

۲-۲-۲ انواع بدافزار

بدافزار (Malware) اصطلاحی کلی برای هر نوع نرم افزار مخرب است که با هدف آسیب رساندن به سیستم‌ها، سرقت اطلاعات یا اختلال در عملکرد طراحی می‌شود. در ادامه، برخی از رایج ترین انواع بدافزارها معرفی می‌شوند:

۱-۲-۲-۲ باج افزار

باج افزار^۱ گونه‌ای از بدافزارها است که با سرعت زیادی گسترش یافته و امروزه به شکل فراگیری در حال شیوع است. باج افزارها عمدتاً به دو نوع قفل کننده (کریپتور)^۲ و مسدود کننده^۳ تقسیم می‌شوند. پس از آلوده سازی رایانه، نوع قفل کننده داده‌های ارزشمند کاربر از جمله اسناد، تصاویر و پایگاه‌های داده را رمزنگاری می‌کند و تا زمانی که رمزگشایی نشوند، هیچ فایلی قابل دسترسی نخواهد بود. مهاجمان در ازای ارائه کلید رمزگشا برای بازگرداندن فایل‌های قفل شده، درخواست باج می‌کنند. نوع مسدود کننده نیز دسترسی به کل سیستم آلوده را مسدود می‌کند و معمولاً میزان باج خواهی آن کمتر از نوع قفل کننده است.

باج افزارها می‌توانند سیستم عامل‌های مختلفی مانند ویندوز، مک‌اواس^۴، لینوکس و اندروید را هدف قرار دهند و هم رایانه‌های دسکتاپ و هم دستگاه‌های موبایل را آلوده سازند [۱]. آلودگی معمولاً از طریق باز کردن فایل‌های ضمیمه مخرب، کلیک روی لینک‌های مشکوک یا نصب برنامه‌ها از منابع غیررسمی رخ می‌دهد. حتی وبسایت‌های رسمی نیز گاهی به واسطه شبکه‌های تبلیغاتی آلوده می‌شوند. حذف باج افزارها معمولاً دشوار است و در صورت رمزنگاری فایل‌ها، کاربر باید برای بازیابی دسترسی، رمزگشایی انجام دهد که پرداخت باج نیز تضمینی برای بازگشت اطلاعات نیست و توصیه نمی‌شود.

۲-۲-۲-۲ تروجان

تروجان^۵ یا اسب تروآ، نوعی برنامه مخرب است که خود را به عنوان نرم افزاری مشروع و بی خطر جلوه می‌دهد تا کاربر را فریب داده و به سیستم نفوذ کند. نام گذاری این بدافزار برگرفته از داستان اسب چوبی تروای یونانیان است که با حيله وارد شهر شد. انواع مختلفی از تروجان‌ها وجود دارد، از جمله:

• تروجان دسترسی از راه دور (Backdoor): به مهاجم امکان کنترل سیستم قربانی از راه دور

¹ Ransomware

² Cryptor

³ Locker

⁴ Mac OS X

⁵ Trojan Horse

را می دهد.

- **تروجان ارسال داده:** اطلاعات حساس مانند رمزهای عبور و داده های وارد شده توسط کاربر را به مهاجم ارسال می کند.
- **تروجان مخرب:** فایل های سیستمی را حذف یا تخریب می کند.
- **تروجان DoS:** باعث کاهش سرعت سیستم و اینترنت کاربر می شود (حملات منع سرویس).
- **تروجان پراکسی:** به مهاجم اجازه می دهد از طریق سیستم قربانی به سرورهای پراکسی حمله کند.
- **تروجان FTP:** پورت FTP را باز کرده و کنترل سیستم را از این طریق به مهاجم می دهد.
- **تروجان غیرفعال سازی امنیت:** نرم افزارهای امنیتی را غیرفعال می کند تا حمله آسان تر انجام شود.

۲-۲-۳ جاسوس افزار

جاسوس افزار^۱ نوعی نرم افزار مخرب است که بدون اطلاع کاربر، اطلاعاتی درباره او یا فعالیت هایش را جمع آوری و برای مهاجم ارسال می کند. کاربردهای رایج جاسوس افزارها شامل تبلیغات هدفمند، جمع آوری اطلاعات شخصی^۲ و ایجاد تغییرات ناخواسته در تنظیمات سیستم قربانی است. این بدافزارها می توانند باعث کاهش کارایی سیستم، نصب نواربازارهای ناخواسته، تغییر صفحه خانگی مرورگر و باز شدن صفحات پاپ آپ تبلیغاتی شوند.

۲-۲-۴ تبلیغ افزار

تبلیغ افزار^۳ نرم افزاری است که با هدف نمایش تبلیغات ناخواسته روی سیستم قربانی طراحی شده است. این تبلیغات معمولاً به صورت بنرها یا صفحات پاپ آپ ظاهر می شوند و گاهی صفحات اینترنتی متعددی را به طور متوالی باز می کنند. اگرچه همه تبلیغ افزارها ذاتاً مخرب نیستند^۴، اما انواع تهاجمی آنها می توانند فعالیت های آنلاین کاربر را دنبال کرده و تجربه کاربری را مختل کنند. همچنین، برخی تبلیغ افزارها اطلاعات مربوط به رفتار کاربر را جمع آوری و برای اهداف تبلیغاتی یا حتی فروش به اشخاص ثالث ارسال می کنند که می تواند حریم خصوصی را نقض کند.

^۱ Spyware

^۲ عبور رمزهای یا بانکی اطلاعات مانند

^۳ Adware

^۴ می کنند درآمزی تبلیغ نمایش طریق از رایگان نرم افزارهای برخی

۲-۲-۳ مفاهیم مرتبط با اپلیکیشن‌های اندرویدی

۲-۳-۱ مجوزهای دسترسی

اندروید از یک سیستم تخصیص مجوز^۱ برخوردار است که برای هر عملیات یا وظیفه^۲ حساس، مجوزها و سطوح دسترسی خاصی را تعیین می‌کند. هر اپلیکیشن می‌تواند برای انجام عملیات خاص، مجوزهای لازم را از این سامانه درخواست نماید. به عنوان مثال، یک برنامه کاربردی ممکن است برای دسترسی به اینترنت، دوربین، یا لیست مخاطبین، مجوزهای مربوطه را درخواست کند. این سیستم دارای سطوح مختلفی برای مجوزها است که به آن‌ها سطح حفاظت^۳ گفته می‌شود.

در نسخه‌های قدیمی‌تر اندروید (قبل از نسخه ۰.۶)، اپلیکیشن‌ها هنگام نصب، لیست تمامی مجوزهای مورد نیاز خود را به کاربر اعلام می‌کردند و کاربر یا همه را می‌پذیرفت یا نصب را لغو می‌کرد. اما از نسخه ۰.۶ اندروید^۴، API سطح ۲۳ به بعد، مدل مجوزدهی پویا^۵ معرفی شد که طی آن، مجوزهای حساس تنها در زمان اجرای برنامه و هنگامی که اپلیکیشن برای اولین بار به آن قابلیت نیاز دارد، از کاربر درخواست می‌شوند. اپلیکیشن‌های اندرویدی مجوزهای مورد نیاز خود را در فایل مانیفست اندروید^۶ اعلام می‌کنند. همچنین، اپلیکیشن می‌تواند مجوزهای اضافی را در این فایل تعریف کند که دسترسی به برخی اجزای داخلی نرم‌افزار توسط سایر برنامه‌ها را محدود می‌سازد.

در اندروید، مجوزها عمدتاً به دو سطح دسترسی و امنیت تقسیم می‌شوند:

- **معمولی (Normal):** مجوزهایی که ریسک پایینی برای حریم خصوصی کاربر یا عملکرد سایر اپلیکیشن‌ها دارند (مانند مجوز دسترسی به اینترنت یا تنظیم منطقه زمانی). این مجوزها معمولاً به صورت خودکار هنگام نصب یا به‌روزرسانی به اپلیکیشن اعطا می‌شوند و نیازی به تأیید صریح کاربر ندارند.

- **خطرناک (Dangerous):** مجوزهایی که به اطلاعات خصوصی کاربر^۷ دسترسی دارند یا می‌توانند بر عملکرد دستگاه یا سایر اپلیکیشن‌ها تأثیر بگذارند^۸. این مجوزها باید حتماً در زمان اجرا و به صورت صریح از کاربر درخواست شوند.

تحلیل الگوهای درخواست مجوز یکی از روش‌های مهم در تشخیص بدافزارهای اندرویدی است، زیرا

^۱Permission System

^۲Task

^۳Protection Level

^۴Marshmallow

^۵Runtime Permissions

^۶AndroidManifest.xml

^۷پیامک‌ها مکانی، موقعیت مخاطبین، مانند

^۸میکروفون یا دوربین به دسترسی مانند

بدافزارها اغلب مجوزهای خطرناک بیشتری نسبت به کاربرد ظاهری خود درخواست می کنند [۴].

۲-۳-۲ فایل APK

فرمت APK^۱ مخفف عبارت^۲ است و فرمت فایل بسته بندی شده ای است که برای توزیع و نصب برنامه ها و میان افزارها در سیستم عامل اندروید استفاده می شود. فایل APK شامل تمام کدهای برنامه (مانند فایل های dex)، منابع (مانند تصاویر و لایه ها)، دارایی ها، گواهی نامه ها و فایل مانیفست^۳ است. این فایل بسیار شبیه به فرمت های بسته بندی دیگر مانند APPX در مایکروسافت ویندوز یا deb در سیستم های مبتنی بر دیبیا^۴ عمل می کند.

برای ساخت یک فایل APK، ابتدا برنامه اندروید با استفاده از ابزارهایی مانند اندروید استودیو کامپایل شده و سپس تمام اجزای آن در یک فایل فشرده با پسوند apk بسته بندی می شود. فایل های APK پایه و اساس برنامه های اندرویدی هستند و بخش اصلی اکوسیستم اندروید محسوب می شوند. کاربران می توانند فایل های APK را از منابع مختلف، از جمله فروشگاه رسمی گوگل پلی^۵ یا به صورت دستی^۶ از وبسایت ها یا منابع دیگر، دانلود و نصب کنند. نصب دستی برنامه ها از منابع نامعتبر یکی از راه های اصلی ورود بدافزار به دستگاه های اندرویدی است.

علاوه بر سیستم عامل اندروید، فایل های APK را می توان با استفاده از شبیه سازها یا لایه های سازگاری در سایر سیستم عامل ها مانند ویندوز، مک او اس یا کروم او اس نیز اجرا، و با ابزارهای مهندسی معکوس، محتوای آن ها را استخراج، ویرایش یا تحلیل کرد.

۲-۳-۳ سورس کد

سورس کد^۷ یا کد منبع برنامه، به مجموعه ای از دستورالعمل ها یا عبارات متنی اشاره دارد که توسط برنامه نویس به یک زبان برنامه نویسی خاص (مانند جاوا، کاتلین، ++C، پایتون) نوشته می شود تا یک برنامه کامپیوتری، عملکرد مورد نظر را پیاده سازی کند. سورس کد برای انسان قابل خواندن است و منطقی، الگوریتم ها و ساختار برنامه را تعریف می کند.

برای اینکه کامپیوتر بتواند سورس کد را اجرا کند، باید ابتدا به زبان ماشین (کد باینری) ترجمه شود. این فرآیند توسط یک کامپایلر یا مفسر انجام می شود. در اکوسیستم اندروید، کدهای جاوا یا کاتلین

^۱ Android Package Kit

^۲ Android Package Kit

^۳ AndroidManifest.xml

^۴ اوبونتو مانند

^۵ Google Play

^۶ Sideload

^۷ Source Code

معمولاً به بایت کد^۱ (در نسخه‌های قدیمی‌تر) یا ART^۲ کامپایل می‌شوند که در فایل‌های dex. درون فایل APK قرار می‌گیرند.

دسترسی به سورس کد یک برنامه امکان درک کامل عملکرد، اصلاح یا توسعه آن را فراهم می‌کند. در حوزه امنیت، تحلیل سورس کد (در صورت در دسترس بودن) یکی از روش‌های تحلیل ایستا برای یافتن آسیب‌پذیری‌ها یا کدهای مخرب است. با این حال، اکثر برنامه‌های تجاری یا بدافزارها به صورت کامپایل شده و بدون سورس کد توزیع می‌شوند و تحلیل آن‌ها نیازمند مهندسی معکوس^۳ است.

۴-۲-۲ داده‌های چندوجهی در تشخیص بدافزار

داده‌های چندوجهی^۴ به مجموعه‌ای از اطلاعات اشاره دارد که از منابع و قالب‌های متنوعی استخراج شده و برای تحلیل جامع‌تر یک پدیده، مانند تشخیص بدافزارهای اندرویدی، به کار می‌روند. در این پژوهش، مدل MAGNET با بهره‌گیری از سه نوع اصلی داده‌های چندوجهی، شامل داده‌های جدولی، گراف‌ی و ترتیبی، طراحی شده است تا بتواند ویژگی‌های متنوع و مکمل اپلیکیشن‌های اندرویدی را به صورت یکپارچه پردازش کند. این رویکرد، برخلاف روش‌های سنتی که تنها به یک نوع داده (مثلاً فقط مجوزها یا فقط فراخوانی‌های API) وابسته‌اند، امکان درک عمیق‌تر رفتارهای مخرب را فراهم می‌سازد. در ادامه، هر یک از این انواع داده‌ها به طور جداگانه توضیح داده می‌شود.

۱-۴-۲-۲ داده‌های جدولی

داده‌های جدولی شامل ویژگی‌های ایستای اپلیکیشن‌های اندرویدی هستند که به صورت ساختارمند و معمولاً عددی یا دسته‌ای ارائه می‌شوند. این ویژگی‌ها می‌توانند شامل تعداد مجوزهای درخواست شده، اندازه فایل APK، نسخه SDK هدف، تعداد فعالیت‌ها^۵، سرویس‌ها^۶، گیرنده‌های پخش^۷، ارائه‌دهندگان محتوا^۸، تعداد فراخوانی‌های API خاص، یا سایر متغیرهای استخراج شده از فایل^۹ یا کد کامپایل شده باشند. در این پژوهش، این داده‌ها با استفاده از کتابخانه^{۱۰} و به صورت اشیای تانسور^{۱۱} بارگذاری و پردازش می‌شوند. برای هر نمونه اپلیکیشن، یک بردار ویژگی با ابعاد مشخص تعریف می‌شود که

^۱Dalvik

^۲(Android Runtime)

^۳بایت‌کد یا باینری کدهای کردن دیس‌اسمبل مانند

^۴Multi-Modal Data

^۵Activities

^۶Services

^۷Broadcast Receivers

^۸Content Providers

^۹AndroidManifest.xml

^{۱۰}PyTorch

^{۱۱}torch.Tensor

نشان‌دهنده خصوصیات ایستای آن است. پیش‌پردازش این داده‌ها، مانند نرمال‌سازی (برای مقادیر عددی) یا کدگذاری وان‌هات (برای مقادیر دسته‌ای)، که با هدف بهبود عملکرد مدل انجام می‌شود، از مراحل کلیدی به شمار می‌رود. این نوع داده، پایه‌ای برای تحلیل‌های آماری و یادگیری الگوهای ساده در بدافزارها فراهم می‌کند (مثلاً بدافزارها ممکن است به طور متوسط مجوزهای بیشتری درخواست کنند).

۲-۴-۲-۲ داده‌های گرافی

داده‌های گرافی ساختارهایی هستند که روابط و تعاملات بین اجزای مختلف یک اپلیکیشن اندرویدی را مدل‌سازی می‌کنند. این داده‌ها می‌توانند نمایانگر گراف فراخوانی توابع^۱، گراف جریان کنترل^۲، گراف وابستگی داده‌ها^۳، یا گراف روابط بین اجزای برنامه (مانند ارتباط بین فعالیت‌ها از طریق^۴) باشند. گره‌ها^۵ در این گراف‌ها می‌توانند توابع، بلوک‌های کد، کلاس‌ها، مجوزها یا اجزای اندرویدی باشند و لبه‌ها نشان‌دهنده روابطی مانند فراخوانی تابع، انتقال کنترل، جریان داده یا درخواست مجوز هستند. در این پژوهش، از کتابخانه PyTorch Geometric (torch_geometric) برای بارگذاری و پردازش این داده‌ها به صورت اشیای داده (Data) استفاده شده است. هر گراف شامل ماتریس همجواری (ساختار گراف) و ویژگی‌های گره‌ها (مانند نوع تابع یا بردار ویژگی‌های استخراج‌شده از کد) است. تحلیل گرافی به مدل اجازه می‌دهد تا الگوهای ساختاری پیچیده و وابستگی‌های پنهان بین اجزا را شناسایی کند (مانند شناسایی یک حلقه فراخوانی مشکوک یا یک خوشه از توابع مرتبط با سرقت اطلاعات)، که در تشخیص بدافزارهای پیچیده یا ناشناخته بسیار مؤثر است.

۳-۴-۲-۲ داده‌های ترتیبی

داده‌های ترتیبی شامل توالی‌هایی از رویدادها یا مقادیر هستند که معمولاً نمایانگر رفتار زمانی یا ترتیب وقوع عملیات در اپلیکیشن‌ها هستند. در زمینه تشخیص بدافزار اندروید، این داده‌ها می‌توانند شامل توالی فراخوانی‌های API سیستمی در حین اجرای برنامه (استخراج‌شده از تحلیل پویا)، توالی رویدادهای سیستمی (مانند ارسال پیامک، اتصال به شبکه)، یا الگوهای زمانی استفاده از منابع (مانند مصرف CPU یا باتری) باشند. در این پژوهش، این داده‌ها با استفاده از تانسورهای LongTensor در PyTorch بارگذاری شده و برای هر نمونه یک یا چند توالی مشخص تعریف می‌شود. پیش‌پردازش این داده‌ها ممکن است شامل نگاشت هر رویداد یا API به یک شناسه عددی، پدینگ^۶ توالی‌ها برای رسیدن به طول یکسان، و

¹ Call Graph

² Control Flow Graph, CFG

³ Data Dependency Graph

⁴ Intents

⁵ Nodes

⁶ Padding

تبدیل به فرمت قابل پردازش توسط مدل‌های ترتیبی مانند RNN، LSTM یا ترنسفورمر باشد. استفاده از داده‌های ترتیبی به مدل امکان می‌دهد تا رفتار پویای اپلیکیشن‌ها را تحلیل کند و الگوهای زمانی مخرب، مانند توالی خاصی از فراخوانی‌ها که منجر به نشت داده می‌شود یا حملات دوره‌ای، را تشخیص دهد.

۲-۴-۴ اهمیت و یکپارچگی داده‌های چندوجهی

ترکیب این سه نوع داده در مدل MAGNET، با استفاده از مکانیزم‌های توجه پویا و تکنیک‌های همجوشی (Fusion)، به شناسایی دقیق‌تر و جامع‌تر بدافزارها کمک می‌کند. داده‌های جدولی اطلاعات کلی و ایستا، داده‌های گراف‌ی روابط ساختاری پیچیده، و داده‌های ترتیبی رفتار پویا را ارائه می‌دهند که در کنار هم، تصویری کامل‌تر از اپلیکیشن را ترسیم می‌کنند. این رویکرد چندوجهی، به‌ویژه در مواجهه با بدافزارهای پیچیده و ناشناخته که ممکن است در یک وجه خاص (مثلاً فقط مجوزها) عادی به نظر برسند، مزیت رقابتی نسبت به روش‌های تک‌وجهی دارد و تعمیم‌پذیری مدل را افزایش می‌دهد [۸]. استخراج این داده‌ها از فایل‌های پردازش‌شده با توابع فرضی `load_graph_data`، `load_processed_data` و `load_sequence_data` انجام شده و پیش‌پردازش آن‌ها با نرمال‌سازی، استانداردسازی و کدگذاری مناسب، تضمین‌کننده سازگاری با معماری‌های یادگیری عمیق مانند ترنسفورمر است.

۲-۵-۲ مدل‌های یادگیری عمیق

یادگیری عمیق، زیرشاخه‌ای از یادگیری ماشین، از شبکه‌های عصبی مصنوعی با چندین لایه (لایه‌های عمیق) برای یادگیری بازنمایی‌های پیچیده از داده‌ها استفاده می‌کند. در ادامه، چند معماری کلیدی که در تشخیص بدافزار کاربرد دارند، معرفی می‌شوند.

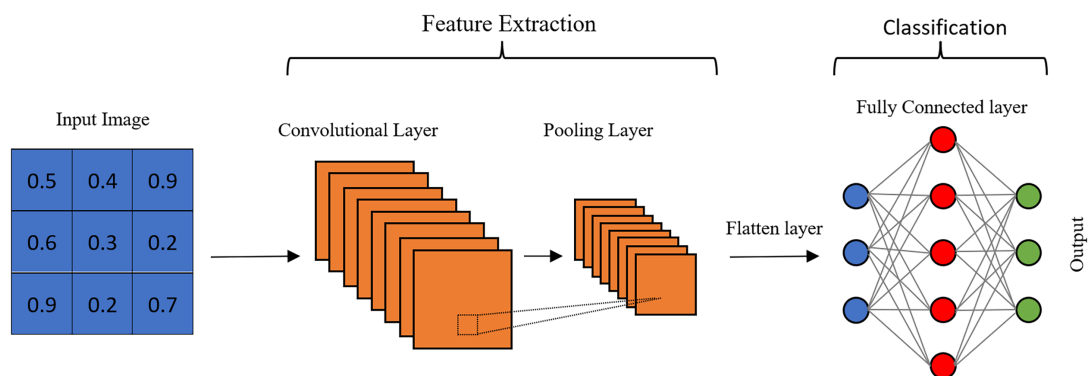
۲-۵-۲-۱ شبکه‌های عصبی کانولوشنی (CNN)

شبکه‌های عصبی کانولوشنی^۱ به‌طور خاص برای پردازش داده‌های با ساختار شبکه‌ای یا گریدمانند، مانند تصاویر، ماتریس‌های ویژگی یا داده‌های ترتیبی با پنجره لغزان، طراحی شده‌اند. این شبکه‌ها با بهره‌گیری از لایه‌های کانولوشنی که فیلترهای کوچکی را روی ورودی حرکت می‌دهند، قادرند ویژگی‌های محلی (مانند الگوهای خاص در بایت‌های کد، توالی‌های API، یا روابط بین مجوزها در یک ماتریس هم‌وقوعی) را به صورت سلسله‌مراتبی و خودکار استخراج کنند. لایه‌های تجمعی^۲ نیز با کاهش ابعاد مکانی داده و فشرده‌سازی اطلاعات، به افزایش پایداری نسبت به تغییرات جزئی و کاهش محاسبات کمک می‌کنند. در نهایت، لایه‌های کاملاً متصل^۳ از ویژگی‌های استخراج‌شده برای طبقه‌بندی نهایی

^۱ Convolutional Neural Networks, CNNs

^۲ Pooling

^۳ Fully Connected



شکل ۱-۲ ساختار کلی شبکه عصبی کانولوشنی (CNN) شامل لایه‌های کانولوشنی، تجمعی و کاملاً متصل برای طبقه‌بندی. برگرفته از [۹].

(مثلاً بدافزار یا سالم) استفاده می‌کنند. در حوزه تشخیص بدافزار، شبکه‌های^۱ می‌توانند برای تحلیل بایت‌های فایل اجرایی به عنوان تصویر، ماتریس‌های ویژگی‌های ایستا، یا توالی‌های رفتاری به کار روند و الگوهای مخرب را شناسایی نمایند [۷].

تحقیقات اخیر نشان داده است که شبکه‌های عصبی کانولوشنی می‌توانند دقت بالایی در تشخیص بدافزار ارائه دهند و ویژگی‌های مورد نیاز را مستقیماً از داده‌های خام یا نیمه‌پردازش شده یاد بگیرند [۹]. ساختار این شبکه‌ها معمولاً به صورت انتها به انتها^۲ طراحی می‌شود و شامل پشته‌ای از لایه‌های کانولوشنی، فعال‌سازی (مانند ReLU)، و تجمعی است که به دنبال آن یک یا چند لایه کاملاً متصل قرار می‌گیرد. افزایش عمق شبکه (تعداد لایه‌ها) امکان استخراج ویژگی‌های انتزاعی‌تر و سطح بالاتر را فراهم می‌کند.

۲-۵-۲-۲ شبکه‌های عصبی بازگشتی و واحدهای حافظه بلند-کوتاه (RNN و LSTM)

شبکه‌های عصبی بازگشتی^۳ نوعی از شبکه‌های عصبی هستند که به طور ویژه برای پردازش داده‌های ترتیبی و زمانی طراحی شده‌اند. برخلاف شبکه‌های پیش‌خور^۴، RNNها دارای حلقه‌های بازگشتی در اتصالات خود هستند که به آن‌ها اجازه می‌دهد اطلاعات مربوط به مراحل قبلی توالی را در یک حالت پنهان^۵ حفظ کرده و از آن برای پردازش مراحل بعدی استفاده کنند. این "حافظه" داخلی، RNNها را برای کاربردهایی مانند تحلیل توالی فراخوانی‌های API، پردازش زبان طبیعی (مانند تحلیل کدهای اسمبلی یا توضیحات برنامه)، و تشخیص رفتارهای پویای اپلیکیشن‌ها که در طول زمان آشکار می‌شوند، مناسب می‌سازد.

^۱ CNN

^۲ End-to-End

^۳ Recurrent Neural Networks, RNNs

^۴ Feedforward

^۵ Hidden State

یکی از چالش‌های اصلی RNN‌های ساده، مشکل محو یا انفجار گرادیان^۱ در هنگام یادگیری وابستگی‌های بلندمدت در توالی‌های طولانی است. برای رفع این مشکل، ساختارهای پیشرفته‌تری مانند واحد حافظه بلند-کوتاه^۲ [۱۰] و واحد بازگشتی دروازه‌ای^۳ [۱۱] معرفی شده‌اند. LSTM با بهره‌گیری از یک سلول حافظه^۴ و سه نوع گیت (گیت فراموشی، گیت ورودی و گیت خروجی)، امکان کنترل دقیق جریان اطلاعات را فراهم می‌کند. این گیت‌ها به شبکه اجازه می‌دهند تا تصمیم بگیرد کدام اطلاعات را در سلول حافظه نگه دارد، کدام را فراموش کند و کدام را به عنوان خروجی مرحله فعلی استفاده کند. این مکانیزم به LSTM‌ها کمک می‌کند تا اطلاعات مهم را در طول توالی‌های بسیار طولانی حفظ کرده و وابستگی‌های بلندمدت را به طور مؤثرتری مدل‌سازی کنند. GRU ساختار ساده‌تری نسبت به LSTM دارد (با دو گیت) اما عملکرد مشابهی در بسیاری از وظایف ارائه می‌دهد.

در این پژوهش، از LSTM (یا می‌توانست از GRU استفاده شود) برای تحلیل داده‌های ترتیبی، مانند توالی فراخوانی‌های API، به منظور شناسایی الگوهای رفتاری پویا در اپلیکیشن‌های اندرویدی استفاده شده است. این قابلیت، LSTM را به ابزاری قدرتمند برای تشخیص رفتارهای مخربی که در طول زمان و از طریق دنباله‌ای از اقدامات رخ می‌دهند، تبدیل می‌کند.

۲-۵-۲ شبکه‌های عصبی گرافی (GNN)

شبکه‌های عصبی گرافی^۵ دسته‌ای از مدل‌های یادگیری عمیق هستند که برای پردازش داده‌هایی با ساختار گرافی (مانند شبکه‌های اجتماعی، مولکول‌های شیمیایی، یا گراف‌های فراخوانی توابع در نرم‌افزار) طراحی شده‌اند [۱۲]. داده‌های گرافی شامل مجموعه‌ای از گره‌ها^۶ و لبه‌ها^۷ هستند که به ترتیب موجودیت‌ها و روابط بین آن‌ها را نشان می‌دهند. GNN‌ها قادرند هم ساختار (توپولوژی) گراف و هم ویژگی‌های گره‌ها و لبه‌ها را برای یادگیری بازنمایی‌های مفید به کار گیرند.

ایده اصلی در GNN‌ها، به‌روزرسانی بازنمایی (بردار ویژگی) هر گره بر اساس اطلاعات گره‌های همسایه آن است. این فرآیند معمولاً شامل دو مرحله اصلی است که در هر لایه GNN تکرار می‌شود:

۱. **تجمیع (Aggregation):** جمع‌آوری اطلاعات (بازنمایی‌ها) از گره‌های همسایه. روش‌های مختلفی برای تجمیع وجود دارد، مانند میانگین‌گیری، جمع یا حداکثرگیری.

۲. **به‌روزرسانی (Update):** ترکیب اطلاعات تجمیع شده از همسایگان با اطلاعات خود گره (بازنمایی

^۱ Vanishing/Exploding Gradient

^۲ Long Short-Term Memory, LSTM

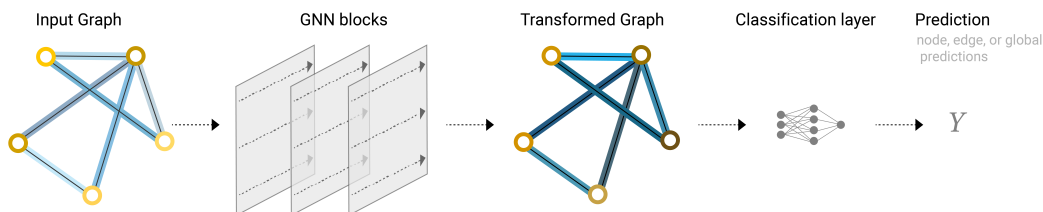
^۳ Gated Recurrent Unit, GRU

^۴ Memory Cell

^۵ Graph Neural Networks, GNNs

^۶ Nodes

^۷ Edges



شکل ۲-۲ ساختار شبکه عصبی گرافی (GNN) برای پردازش داده‌های گرافی و طبقه‌بندی. برگرفته از [۱۲].

فعلی آن) برای تولید بازنمایی جدید گره. این کار معمولاً با استفاده از یک شبکه عصبی کوچک (مانند یک لایه خطی و یک تابع فعال‌سازی) انجام می‌شود.

با پشته کردن چندین لایه GNN، هر گره می‌تواند اطلاعات را از همسایگان دورتر نیز دریافت کند و بازنمایی‌های پیچیده‌تری که الگوهای ساختاری سطح بالاتر را در بر می‌گیرند، یاد گرفته شود.

در این پژوهش، از GNN (به‌طور خاص، احتمالاً مدل‌هایی مانند ^۱[۱۳] یا ^۲[۱۴]) برای تحلیل داده‌های گرافی استخراج شده از اپلیکیشن‌ها (مانند گراف فراخوانی API) استفاده شده است. این داده‌ها با استفاده از کتابخانه ^۳ به صورت اشیای Data بارگذاری و پردازش می‌شوند. GNN‌ها به مدل کمک می‌کنند تا ویژگی‌های ساختاری مهمی را که ممکن است نشان‌دهنده رفتار مخرب باشند (مانند الگوهای خاصی از تعامل بین توابع یا اجزا) استخراج کنند.

۶-۲-۲ ترنسفورمرها

۱-۰-۶-۲-۲ معرفی ترنسفورمر و تحول در مدل‌های یادگیری عمیق نقطه عطف برجسته‌ای

در تکامل معماری‌های یادگیری عمیق، معرفی مدل ترنسفورمر در مقاله "Attention Is All You Need" توسط Vaswani et al. [15] بود که انقلابی در پردازش داده‌های ترتیبی، و بعدها انواع دیگر داده‌ها، به ارمغان آورد. این معماری، برخلاف مدل‌های بازگشتی (RNN/LSTM) که توالی‌ها را به صورت مرحله به مرحله پردازش می‌کردند و با مشکلاتی مانند وابستگی‌های بلندمدت و عدم امکان پردازش موازی مواجه بودند، صرفاً بر مکانیزم توجه ^۴، به‌ویژه توجه خودی ^۵ و توجه چندسر ^۶، تکیه می‌کند.

مکانیزم توجه به مدل اجازه می‌دهد تا هنگام پردازش یک عنصر در توالی (مثلاً یک کلمه در جمله یا یک فراخوانی API در یک دنباله)، به طور مستقیم به تمام عناصر دیگر توالی نگاه کرده و وزن اهمیتی متفاوتی به هر یک اختصاص دهد. این قابلیت، مدل‌سازی وابستگی‌های بلندمدت بین عناصر

^۱ Graph Convolutional Network (GCN)

^۲ Graph Attention Network (GAT)

^۳ torch_geometric

^۴ Attention Mechanism

^۵ Self-Attention

^۶ Multi-Head Attention

را بسیار کارآمدتر می‌کند. همچنین، از آنجایی که محاسبات توجه برای هر عنصر می‌تواند به صورت موازی انجام شود، ترنسفورمرها سرعت آموزش و استنتاج را به‌طور چشمگیری بهبود بخشیدند و امکان آموزش مدل‌های بسیار بزرگتر را فراهم کردند. معماری استاندارد ترنسفورمر شامل دو بخش اصلی است: کدگذار^۱ که ورودی را به یک دنباله از بازنمایی‌های پیوسته تبدیل می‌کند و گشاینده^۲ که این بازنمایی‌ها را برای تولید خروجی (مثلاً در ترجمه ماشینی یا تولید متن) به کار می‌برد. هر دوی این بخش‌ها از پشته‌هایی از لایه‌های توجه چندسر و شبکه‌های عصبی پیش‌خور^۳ تشکیل شده‌اند که با اتصالات باقیمانده^۴ و نرمال‌سازی لایه‌ای^۵ ترکیب شده‌اند.

با ظهور ترنسفورمر، مدل‌های زبانی بزرگ^۶ پیشرفته‌ای مانند BERT (که از پشته کدگذار ترنسفورمر استفاده می‌کند و برای درک زبان آموزش دیده) و GPT (که از پشته گشاینده ترنسفورمر بهره می‌برد و برای تولید زبان آموزش دیده) توسعه یافتند. این مدل‌ها با پیش‌آموزش روی حجم عظیمی از داده‌های متنی و سپس تنظیم دقیق^۷ برای وظایف خاص، عملکردی استثنایی در طیف وسیعی از کاربردهای پردازش زبان طبیعی نشان دادند.

این پیشرفت‌ها، الهام‌بخش کاربرد ترنسفورمرها در حوزه‌های فراتر از زبان، از جمله بینایی کامپیوتر، تحلیل داده‌های گراف و همچنین امنیت سایبری و تشخیص بدافزار [۱۶] شد. توانایی ترنسفورمر در مدل‌سازی روابط پیچیده و بلندمدت و پردازش موازی، آن را به گزینه‌ای جذاب برای تحلیل داده‌های چندوجهی و پیچیده مرتبط با بدافزارها تبدیل کرد.

۷-۲-۲ ترنسفورمرها در مدل MAGNET

در این پژوهش، معماری ترنسفورمر به‌عنوان هسته اصلی مدل MAGNET^۸ به کار گرفته شده است تا داده‌های چندوجهی شامل داده‌های جدولی (ویژگی‌های ایستا)، گرافی (روابط ساختاری) و ترتیبی (رفتار پویا) را به‌صورت یکپارچه تحلیل کند. ترنسفورمر در این مدل از مکانیزم توجه پویا برای یادگیری بازنمایی‌های غنی^۹ از هر وجه داده و سپس تلفیق^{۱۰} هوشمندانه این بازنمایی‌ها استفاده می‌کند.

به‌طور خاص، لایه‌های کدگذار ترنسفورمر می‌توانند برای پردازش ویژگی‌های استخراج شده از داده‌های جدولی (پس از تبدیل به دنباله‌ای از ویژگی‌ها یا استفاده از توکن‌های خاص) و داده‌های گرافی

¹ Encoder

² Decoder

³ Feed-Forward Networks

⁴ Residual Connections

⁵ Layer Normalization

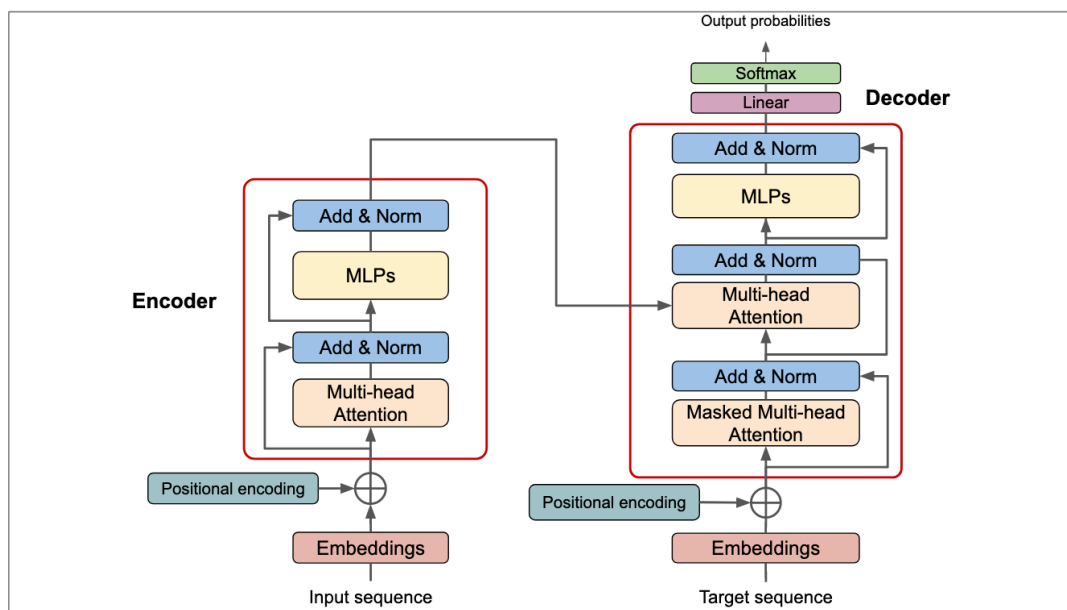
⁶ Large Language Models, LLMs

⁷ Fine-tuning

⁸ Multi-Modal Analysis for Graph and Network Threat Detection

⁹ Embeddings

¹⁰ Fusion



شکل ۲-۳ معماری ترنسفورمر شامل کد گذار و گشاینده با مکانیزم توجه چندسرسر. بر گرفته از [۱۵].

(با استفاده از تکنیک‌هایی مانند تبدیل گره‌ها به دنباله یا به کارگیری Graph Transformers) به کار روند. مکانیزم توجه چندسرسر به مدل اجازه می‌دهد تا روابط متقابل بین ویژگی‌های مختلف ایستا و ساختاری را کشف کند. همزمان، لایه‌های گشاینده ترنسفورمر (یا یک پشته کد گذار دیگر) می‌توانند برای مدل‌سازی توالی‌های ترتیبی (مانند فراخوانی‌های API) به کار گرفته شوند تا الگوهای زمانی مخرب شناسایی شوند.

ساختار ترنسفورمر در MAGNET احتمالاً شامل چندین بلوک کد گذار/گشاینده است که هر بلوک دارای یک لایه توجه چندسرسر و یک لایه تغذیه روبه‌جلو است. داده‌های ورودی از هر وجه ابتدا با استفاده از لایه‌های جاسازی به بردارهایی با ابعاد ثابت تبدیل می‌شوند. سپس، مکانیزم توجه خودی و احتمالاً توجه متقابل^۱ بین وجه‌های مختلف، وزن‌های متفاوتی به هر عنصر داده تخصیص می‌دهد تا اهمیت نسبی آن را در پیش‌بینی نهایی برچسب (سالم یا مخرب) تعیین کند. این فرآیند با استفاده از نرمال‌سازی لایه‌ای و اتصالات باقیمانده برای پایداری و بهبود آموزش، بهینه می‌شود.

مزیت اصلی استفاده از ترنسفورمر در مدل MAGNET، توانایی آن در پردازش موازی داده‌های چندوجهی و درک روابط پیچیده و بلندمدت *درون* هر وجه و *بین* وجه‌های مختلف داده است. این ویژگی، به‌ویژه در شناسایی بدافزارهای پیچیده و ناشناخته که ممکن است از تکنیک‌های پنهان‌سازی پیشرفته استفاده کنند و نیازمند تحلیل جامع رفتار و ساختار اپلیکیشن‌ها هستند، بسیار مؤثر است. نتایج ادعا شده در متن (دقت ۹۷.۲۴ درصد و ۰.۹۸۲۳ F1 Score) نشان می‌دهد که این رویکرد مبتنی بر ترنسفورمر و داده‌های چندوجهی، پتانسیل ارائه عملکردی برتر نسبت به مدل‌های سنتی یا تک‌وجهی را دارد.

^۱ Cross-Attention

۸-۲-۲ ماشین بردار پشتیبان (SVM)

ماشین بردار پشتیبان^۱ یکی از الگوریتم‌های یادگیری نظارت‌شده قدرتمند و پرکاربرد است که عمدتاً برای مسائل طبقه‌بندی^۲ و همچنین رگرسیون^۳ استفاده می‌شود. ایده اصلی SVM در مسائل طبقه‌بندی دودویی، یافتن یک ابرصفحه^۴ بهینه در فضای ویژگی‌هاست که بتواند داده‌های مربوط به دو کلاس مختلف را به بهترین شکل ممکن از یکدیگر جدا کند.

”بهترین شکل ممکن” در SVM به معنای یافتن ابرصفحه‌ای است که بیشترین حاشیه^۵ را با نزدیک‌ترین نقاط داده از هر دو کلاس داشته باشد. این نزدیک‌ترین نقاط، که روی مرز حاشیه قرار می‌گیرند، بردارهای پشتیبان Support Vectors نامیده می‌شوند، زیرا موقعیت ابرصفحه جداکننده تنها به این نقاط بستگی دارد. هدف، حداکثر کردن این حاشیه است، زیرا تئوری یادگیری آماری نشان می‌دهد که جداکننده‌ای با حاشیه بزرگتر، معمولاً خطای تعمیم کمتری دارد و عملکرد بهتری روی داده‌های دیده‌نشده خواهد داشت.

۱-۰-۸-۲-۲ انواع SVM

• **SVM خطی**^۶: زمانی که داده‌ها به صورت خطی قابل تفکیک باشند (یعنی بتوان با یک خط مستقیم یا یک صفحه صاف آن‌ها را جدا کرد)، از SVM خطی استفاده می‌شود.

• **SVM غیرخطی**^۷: در بسیاری از مسائل دنیای واقعی، داده‌ها به صورت خطی قابل تفکیک نیستند. در این موارد، SVM از ترفند کرنل^۸ استفاده می‌کند. ترفند کرنل به SVM اجازه می‌دهد تا داده‌ها را به یک فضای ویژگی با ابعاد بالاتر نگاشت کند که در آن، داده‌ها ممکن است به صورت خطی قابل تفکیک شوند. سپس SVM خطی در آن فضای جدید اعمال می‌شود. توابع کرنل رایج شامل کرنل چندجمله‌ای^۹، کرنل تابع پایه شعاعی^{۱۰} یا گاوسی، و کرنل سیگموئید^{۱۱} هستند.

^۱ Support Vector Machine, SVM

^۲ Classification

^۳ Regression

^۴ Hyperplane

^۵ Margin

^۶ Linear SVM

^۷ Non-linear SVM

^۸ Kernel Trick

^۹ Polynomial

^{۱۰} Radial Basis Function, RBF

^{۱۱} Sigmoid

۲-۲-۸-۲ کاربرد در تشخیص بدافزار SVM به دلیل عملکرد خوب در فضاهای با ابعاد بالا و مقاومت نسبی در برابر بیش‌برازش^۱، به‌طور گسترده‌ای در تشخیص بدافزار اندروید، به‌ویژه با استفاده از ویژگی‌های استاتیک (مانند مجوزها، فراخوانی‌های API) یا ویژگی‌های پویا (مانند توالی‌های رفتاری) استفاده شده است [۳، ۱۷]. اغلب، ویژگی‌های استخراج‌شده از اپلیکیشن‌ها به عنوان ورودی به SVM داده می‌شوند تا مدل آموزش ببیند که آیا یک اپلیکیشن بدافزار است یا خیر.

۲-۲-۸-۳ مزایا و معایب

• **مزایا:** کارایی بالا در فضاهای با ابعاد زیاد، مؤثر بودن در مواردی که تعداد ابعاد بیشتر از تعداد نمونه‌هاست، حافظه کارآمد (چون فقط از بردارهای پشتیبان استفاده می‌کند)، تطبیق‌پذیری با استفاده از کرنل‌های مختلف.

• **معایب:** عملکرد ضعیف در مجموعه داده‌های بسیار بزرگ (به دلیل پیچیدگی محاسباتی آموزش که می‌تواند بین $O(n^2)$ تا $O(n^3)$ باشد)، حساسیت به انتخاب تابع کرنل و پارامترهای آن (مانند پارامتر γ یا هزینه خطا و پارامتر گاما در کرنل RBF)، عدم ارائه مستقیم احتمالات تعلق به کلاس (اگرچه روش‌هایی برای تخمین آن وجود دارد).

اگرچه مدل‌های یادگیری عمیق مانند CNN و ترنسفورمرها در سال‌های اخیر توجه بیشتری را به خود جلب کرده‌اند، SVM همچنان یک ابزار قدرتمند و یک معیار (Baseline) مهم در حوزه تشخیص بدافزار محسوب می‌شود.

۲-۳ تکنیک‌های آموزشی و ارزیابی

در این بخش، تکنیک‌های کلیدی مورد استفاده برای آموزش مدل MAGNET و ارزیابی عملکرد آن تشریح می‌شود. این تکنیک‌ها برای اطمینان از قابلیت اطمینان، کارایی و تعمیم‌پذیری مدل ضروری هستند.

۲-۳-۱ اعتبارسنجی متقاطع (Cross-Validation)

اعتبارسنجی متقاطع^۲ یک تکنیک آماری برای ارزیابی عملکرد مدل‌های یادگیری ماشین و تخمین میزان تعمیم‌پذیری آن‌ها به داده‌های جدید و مستقل است. این روش به کاهش واریانس تخمین عملکرد

^۱Overfitting

^۲Cross-Validation

کمک کرده و از بیش‌برازش^۱ بر روی یک تقسیم خاص از داده‌ها به مجموعه‌های آموزشی و آزمایشی جلوگیری می‌کند.

رایج‌ترین نوع اعتبارسنجی متقاطع، **اعتبارسنجی k-تایی (k-fold Cross-Validation)** است. در این روش:

۱. مجموعه داده اصلی به صورت تصادفی به k زیرمجموعه (یا "تا") با اندازه تقریباً مساوی تقسیم می‌شود.

۲. فرآیند آموزش و ارزیابی k بار تکرار می‌شود.

۳. در هر تکرار (از i تا k)، از $k-1$ زیرمجموعه برای آموزش مدل استفاده می‌شود و از زیرمجموعه باقی‌مانده (تا i -ام) به عنوان مجموعه اعتبارسنجی (Validation Set) برای ارزیابی مدل استفاده می‌شود.

۴. نتایج ارزیابی (مانند دقت، F1 Score) از هر k تکرار جمع‌آوری می‌شود.

۵. میانگین (و گاهی انحراف معیار) این نتایج به عنوان تخمین نهایی عملکرد مدل گزارش می‌شود.

در این پژوهش، از **اعتبارسنجی ۵-تایی**^۲ استفاده شده است ($k = 5$). این بدان معناست که داده‌ها به پنج بخش تقسیم شده و مدل پنج بار آموزش و ارزیابی می‌شود، هر بار با یک بخش متفاوت به عنوان داده اعتبارسنجی. این رویکرد نسبت به تقسیم ساده آموزشی/آزمایشی، تخمین پایدارتر و قابل اعتمادتری از عملکرد مدل ارائه می‌دهد، زیرا از تمام داده‌ها هم برای آموزش و هم برای ارزیابی استفاده می‌شود. در پیاده‌سازی کد، این فرآیند می‌تواند با استفاده از توابعی مانند KFold یا StratifiedKFold از کتابخانه scikit-learn و اجرای حلقه آموزش و ارزیابی در هر تقسیم انجام شود.

۲-۳-۲ مدیریت بیش‌برازش و کم‌برازش

دو چالش اساسی در آموزش مدل‌های یادگیری ماشین، به‌ویژه مدل‌های یادگیری عمیق با ظرفیت بالا، بیش‌برازش^۳ و کم‌برازش^۴ هستند.

• **کم‌برازش**^۵: زمانی رخ می‌دهد که مدل به اندازه کافی پیچیده نیست یا به اندازه کافی آموزش ندیده است تا الگوهای اساسی موجود در داده‌های آموزشی را یاد بگیرد. در نتیجه، مدل هم

¹Overfitting

²5-fold Cross-Validation

³Overfitting

⁴Underfitting

⁵Underfitting

بر روی داده‌های آموزشی و هم بر روی داده‌های جدید عملکرد ضعیفی دارد. برای مقابله با کم‌برازش، می‌توان پیچیدگی مدل را افزایش داد (مثلاً با افزودن لایه‌ها یا نورون‌ها)، ویژگی‌های بهتری مهندسی کرد، یا زمان آموزش را افزایش داد. در مدل MAGNET، تنظیم مناسب تعداد لایه‌ها^۱ و ابعاد نهان^۲ می‌تواند به جلوگیری از کم‌برازش کمک کند.

• **بیش‌برازش^۳:** زمانی رخ می‌دهد که مدل بیش از حد به داده‌های آموزشی خاص، از جمله نویز و جزئیات تصادفی آن، وابسته می‌شود. در نتیجه، مدل بر روی داده‌های آموزشی عملکرد بسیار خوبی دارد، اما توانایی تعمیم به داده‌های جدید و دیده‌نشده را از دست می‌دهد و بر روی آن‌ها عملکرد ضعیفی نشان می‌دهد. برای مقابله با بیش‌برازش، از تکنیک‌های تنظیم‌گری^۴ استفاده می‌شود. در این پژوهش، از تکنیک‌های زیر استفاده شده است:

• **کناره‌گیری^۵:** در هر مرحله آموزش، به طور تصادفی برخی از نورون‌ها (و اتصالات آن‌ها) را با احتمال مشخصی (در اینجا با نرخ ۲۰٪) غیرفعال می‌کند. این کار باعث می‌شود شبکه به یک مسیر یا ویژگی خاص بیش از حد وابسته نشود و مدل مقاوم‌تری یاد بگیرد.

• **توقف زودهنگام^۶:** عملکرد مدل بر روی یک مجموعه اعتبارسنجی جداگانه در طول آموزش نظارت می‌شود. اگر عملکرد مدل روی مجموعه اعتبارسنجی برای تعداد مشخصی از دوره‌ها (Epochs) بهبود نیابد (در اینجا با صبر ۳ دوره)، آموزش متوقف می‌شود، حتی اگر عملکرد روی مجموعه آموزشی همچنان در حال بهبود باشد. این کار از ادامه آموزش و ورود مدل به فاز بیش‌برازش جلوگیری می‌کند.

• **تنظیم‌گری L2^۷:** (هرچند به صراحت در بخش بیش‌برازش ذکر نشده، اما در بخش بهینه‌سازی Adam با $\text{weight_decay}=0.01$ اشاره شده است). این روش با افزودن یک جمله جریمه به تابع هزینه که متناسب با مجذور اندازه وزن‌های مدل است، از بزرگ شدن بیش از حد وزن‌ها جلوگیری می‌کند و به مدل ساده‌تر و با تعمیم‌پذیری بهتر منجر می‌شود.

تحلیل نمودارهای تابع هزینه و معیارهای ارزیابی بر روی داده‌های آموزشی و اعتبارسنجی در طول زمان آموزش، ابزار مهمی برای تشخیص و مدیریت این دو پدیده است.

¹ num_layers

² embedding_dim

³ Overfitting

⁴ Regularization

⁵ Dropout

⁶ Early Stopping

⁷ Weight Decay

۲-۳-۳ روش‌های بهینه‌سازی

بهینه‌سازی فرآیند تنظیم پارامترهای مدل (مانند وزن‌ها و بایاس‌ها در شبکه‌های عصبی) به منظور کمینه کردن تابع هزینه^۱ است. انتخاب الگوریتم بهینه‌سازی مناسب و تنظیم ابرپارامترهای^۲ آن نقش کلیدی در سرعت همگرایی و کیفیت نهایی مدل دارد.

۲-۳-۳-۱ الگوریتم Adam

الگوریتم^۳ یکی از محبوب‌ترین و کارآمدترین الگوریتم‌های بهینه‌سازی مبتنی بر گرادینان نزولی تصادفی Stochastic Gradient Descent, SGD است که برای آموزش شبکه‌های عصبی عمیق استفاده می‌شود. Adam نرخ یادگیری Learning Rate را برای هر پارامتر به صورت تطبیقی تنظیم می‌کند. این کار را با استفاده از تخمین‌های مرتبه اول (میانگین یا مومنتوم) و مرتبه دوم (واریانس غیرمرکزی) گرادینان‌ها انجام می‌دهد. به عبارت دیگر، Adam مزایای دو الگوریتم دیگر، یعنی AdaGrad (که نرخ یادگیری را بر اساس فراوانی به‌روزرسانی پارامتر تنظیم می‌کند) و RMSProp (که از میانگین متحرک نمایی مجذور گرادینان‌ها استفاده می‌کند) را با هم ترکیب می‌کند و همچنین از مومنتوم برای هموارسازی مسیر گرادینان و تسریع همگرایی بهره می‌برد. در این پژوهش، Adam با نرخ یادگیری پیش‌فرض (معمولاً ۰.۰۰۱) و ضریب تنظیم‌گری L2 (weight_decay) برابر با ۰.۰۱ در تابع train_and_evaluate_magnet برای به‌روزرسانی وزن‌های مدل MAGNET به کار رفته است. Adam به دلیل سرعت همگرایی بالا، نیاز کمتر به تنظیم دقیق نرخ یادگیری اولیه، و عملکرد خوب در مسائل با گرادینان‌های پراکنده یا نویزی، انتخاب مناسبی برای آموزش مدل‌های پیچیده مانند ترنسفورمر و GNN بر روی داده‌های چندوجهی بوده است.

۲-۳-۳-۲ الگوریتم PIRATES

الگوریتم PIRATES^۵ یکی از روش‌های نوین بهینه‌سازی الهام‌گرفته از طبیعت است که با الهام از رفتار جمعی دزدان دریایی در جستجوی گنج طراحی شده است. این الگوریتم با الهام از مطالعات [۱۸] در زمینه بهینه‌سازی و کاربردهای یادگیری تقویتی [۱۹]، و همچنین با استفاده از مفاهیم بازی دزدان دریایی، طراحی شده است. در این الگوریتم، هر راه‌حل بالقوه به عنوان یک «کشتی» در فضای جستجو مدل می‌شود و مجموعه‌ای از کشتی‌ها (جمعیت) به طور موازی و با استفاده از اطلاعات فردی (تجربیات گذشته)، جمعی (اطلاعات بهترین اعضا)، و تعاملات خاص (مانند نبرد و طوفان)، به سمت نقاط بهینه

¹ Loss Function

² Hyperparameters

³ Adam

⁴ Adaptive Moment Estimation

⁵ Pirate-Inspired Robust Adaptive Trajectory Exploration Strategy

حرکت می کنند. این الگوریتم با ترکیب ایده هایی از الگوریتم های ازدحامی (مانند PSO) و تکاملی، سعی در افزایش پایداری، سرعت همگرایی و جلوگیری از گیر افتادن در بهینه های محلی دارد.

اجزای کلیدی این الگوریتم عبارت اند از:

- **کشتی ها (Ships):** هر کشتی نمایانگر یک نقطه در فضای جستجو است و موقعیت و سرعت مخصوص به خود را دارد.

- **رهبر (Leader):** کشتی با بهترین عملکرد (کمترین مقدار تابع هدف) در هر تکرار به عنوان رهبر انتخاب می شود و سایر کشتی ها از آن پیروی می کنند.

- **نقشه جمعی و خصوصی:** هر کشتی علاوه بر بهترین موقعیت شخصی، از نقشه ای جمعی (شامل بهترین موقعیت های سایر کشتی ها) نیز بهره می برد.

- **کشتی های برتر (Top Ships):** تعدادی از بهترین کشتی ها به عنوان مرجع برای سایر اعضا عمل می کنند.

- **مکانیزم های تنوع بخش:** الگوریتم با استفاده از نبرد بین کشتی های برتر و وقوع طوفان های تصادفی، از همگرایی زودهنگام و گیر افتادن در بهینه های محلی جلوگیری می کند.

الگوریتم PIRATES با بهره گیری از چندین منبع اطلاعاتی و تنظیم پویا، قادر است به سرعت به نقاط بهینه همگرا شود و در عین حال تنوع جمعیت را حفظ کند. این ویژگی ها، PIRATES را به گزینه ای مناسب برای بهینه سازی مسائل پیچیده، از جمله تنظیم خود کار هاپیرپارامترهای مدل های یادگیری عمیق و چندوجهی، تبدیل کرده است. در پژوهش حاضر، از این الگوریتم برای جستجوی بهینه پارامترهای مدل استفاده شده است.

استفاده از الگوریتم های بهینه سازی الهام گرفته از طبیعت، به ویژه PIRATES، در سال های اخیر به عنوان رویکردی مؤثر برای حل مسائل بهینه سازی غیرخطی و پیچیده در حوزه های مختلف، از جمله امنیت سایبری و یادگیری ماشین، مطرح شده است. در این پژوهش، PIRATES به عنوان یکی از ابزارهای کلیدی برای تنظیم بهینه پارامترهای مدل چندوجهی مبتنی بر ترنسفورمر و شبکه های عصبی گرافی به کار رفته است.

۲-۳-۳-۲ الگوریتم Optuna

Optuna [۲۰] یک چارچوب Framework نرم افزاری متن باز و قدرتمند برای بهینه سازی خود کار هاپیرپارامترها است. Optuna از الگوریتم های جستجوی متنوعی پشتیبانی می کند، از جمله جستجوی

تصادفی^۱، جستجوی شبکه‌ای^۲، و الگوریتم‌های پیشرفته‌تر مبتنی بر مدل مانند^۳ که نوعی بهینه‌سازی بیزی است. یکی از ویژگی‌های کلیدی Optuna، قابلیت هرس (Pruning) آزمایش‌های ناموفق است. با استفاده از الگوریتم‌های هرس^۴، Optuna می‌تواند آزمایش‌هایی را که در مراحل اولیه عملکرد ضعیفی دارند، متوقف کرده و منابع محاسباتی را بر روی آزمایش‌های امیدوارکننده‌تر متمرکز کند.

در این پژوهش، ادعا شده که از Optuna برای تنظیم پارامترهایی مانند نرخ Dropout (که مقدار بهینه ۲۰٪ یافت شده) و ابعاد نهان^۵ استفاده شده است. این فرآیند معمولاً شامل تعریف یک تابع هدف و مشخص کردن فضای جستجو برای هر هایپرپارامتر است. سپس Optuna با اجرای مکرر تابع هدف با مقادیر مختلف هایپرپارامترها (که با استفاده از نمونه‌برداری تطبیقی انتخاب می‌شوند)، به تدریج به سمت یافتن ترکیب بهینه حرکت می‌کند. استفاده از Optuna می‌تواند فرآیند زمان‌بر و خسته‌کننده تنظیم دستی هایپرپارامترها را خودکار کرده و به یافتن مدل‌هایی با عملکرد بهتر کمک کند.

۲-۳-۴ معیارهای ارزیابی عملکرد

برای ارزیابی کمی و کیفی عملکرد مدل طبقه‌بندی MAGNET در تشخیص بدافزارهای اندرویدی، از معیارهای استاندارد مختلفی استفاده شده است. انتخاب معیار مناسب به ماهیت مسئله و توزیع کلاس‌ها در مجموعه داده بستگی دارد.

۲-۳-۴-۱ دقت (Accuracy)

دقت (Accuracy) ساده‌ترین و رایج‌ترین معیار ارزیابی است که نسبت کل نمونه‌هایی که به درستی توسط مدل طبقه‌بندی شده‌اند (چه مثبت و چه منفی) به تعداد کل نمونه‌ها را اندازه‌گیری می‌کند:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

که در آن:

• TP (True Positive): تعداد نمونه‌های بدافزار که به درستی بدافزار تشخیص داده شده‌اند.

• TN (True Negative): تعداد نمونه‌های سالم که به درستی سالم تشخیص داده شده‌اند.

¹ Random Search

² Grid Search

³ Tree-structured Parzen Estimator (TPE)

⁴ Successive Halving یا Median Pruning مانند

⁵ embedding_dim

- FP (False Positive): تعداد نمونه‌های سالم که به اشتباه بدافزار تشخیص داده شده‌اند (خطای نوع اول).

- FN (False Negative): تعداد نمونه‌های بدافزار که به اشتباه سالم تشخیص داده شده‌اند (خطای نوع دوم).

در این پژوهش، دقت مدل MAGNET در تست نهایی ۲۴.۹۷ درصد گزارش شده است. دقت معیار خوبی است زمانی که کلاس‌ها تقریباً متوازن باشند. با این حال، در مجموعه‌های داده نامتوازن (مثلاً اگر تعداد نمونه‌های سالم بسیار بیشتر از بدافزارها باشد)، دقت بالا ممکن است گمراه‌کننده باشد.

F1 Score ۲-۴-۳-۲

امتیاز F1 (F1 Score) میانگین هارمونیک دو معیار دیگر، یعنی دقت (Precision) و یادآوری (Recall) است و به ویژه در مسائل با کلاس‌های نامتوازن مفید است.

- **دقت (Precision):** نسبت نمونه‌هایی که به درستی مثبت پیش‌بینی شده‌اند به کل نمونه‌هایی که مثبت پیش‌بینی شده‌اند. این معیار نشان می‌دهد که چقدر می‌توان به پیش‌بینی‌های مثبت مدل اعتماد کرد.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **یادآوری (Recall) یا حساسیت (Sensitivity) یا نرخ مثبت واقعی (TPR):** نسبت نمونه‌هایی که به درستی مثبت پیش‌بینی شده‌اند به کل نمونه‌های واقعاً مثبت. این معیار نشان می‌دهد که مدل چه کسری از نمونه‌های مثبت واقعی را توانسته شناسایی کند.

$$\text{Recall} = \frac{TP}{TP + FN}$$

امتیاز F1 به صورت زیر محاسبه می‌شود:

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times TP}{2 \times TP + FP + FN}$$

امتیاز F1 تعادلی بین دقت و یادآوری برقرار می‌کند و مقدار بالای آن نشان می‌دهد که مدل هم در کاهش هشدارهای کاذب (FP بالا باعث کاهش Precision می‌شود) و هم در شناسایی نمونه‌های مثبت واقعی (FN بالا باعث کاهش Recall می‌شود) موفق بوده است. در این پژوهش، مقدار F1 Score ۰.۹۸۲۳ در بهترین حالت به دست آمده که نشان‌دهنده عملکرد بسیار خوب مدل در هر دو جنبه است. این معیار معمولاً برای ارزیابی عملکرد در تشخیص بدافزار ارجحیت دارد، زیرا هزینه تشخیص نادرست (FN یا FP) می‌تواند بالا باشد.

۲-۳-۴-۳-۲ AUC Area Under the ROC Curve

منحنی مشخصه عملکرد گیرنده^۱ نموداری است که نرخ مثبت واقعی^۲ یا همان Recall را در مقابل نرخ مثبت کاذب^۳ در آستانه‌های طبقه‌بندی مختلف رسم می‌کند.

$$FPR = \frac{FP}{FP + TN}$$

مساحت زیر این منحنی^۴ یک معیار واحد است که توانایی کلی مدل در تمایز بین کلاس‌های مثبت و منفی را در تمام آستانه‌های ممکن اندازه‌گیری می‌کند. مقدار AUC بین ۰ و ۱ متغیر است.

• $AUC = 1$: طبقه‌بندی کامل و بی‌نقص.

• $AUC = 0.5$: عملکرد تصادفی (مانند پرتاب سکه).

• $AUC < 0.5$: عملکرد بدتر از تصادفی.

AUC یک معیار مفید است زیرا نسبت به تغییرات آستانه طبقه‌بندی و همچنین نسبت به عدم توازن کلاس‌ها (تا حدی) مقاوم است. هرچند مقدار دقیق AUC در متن گزارش نشده، اما با توجه به F1 Score بالا (۰.۹۸۲۳)، انتظار می‌رود مقدار AUC نیز بسیار بالا و نزدیک به ۱ (احتمالاً حدود ۰.۹۸ یا بیشتر) باشد، که نشان‌دهنده قدرت تمایز عالی مدل MAGNET است.

۲-۳-۴-۳-۱ **جمع‌بندی ارزیابی** استفاده ترکیبی از این معیارها (دقت، F1 Score، AUC) به همراه اعتبارسنجی متقاطع، تصویری جامع و قابل اعتماد از عملکرد مدل MAGNET ارائه می‌دهد و

^۱ Receiver Operating Characteristic, ROC

^۲ True Positive Rate, TPR

^۳ False Positive Rate, FPR

^۴ Area Under the Curve, AUC

امکان مقایسه معنادار آن با سایر روش‌ها را فراهم می‌کند. مدیریت دقیق بیش‌برازش و کم‌برازش و استفاده از روش‌های بهینه‌سازی مناسب نیز به دستیابی به این نتایج کمک کرده‌اند.

۴-۲ مروری بر مطالعات پیشین

در این بخش، پژوهش‌های پیشین در حوزه تشخیص بدافزارهای اندرویدی با تمرکز بر رویکردهای اصلی تحلیل ایستا و پویا مرور می‌شوند. همچنین، به تاریخچه مختصری از تلاش‌ها در این زمینه اشاره می‌شود.

۱-۴-۲ روش‌های تحلیل ایستا

روش‌های تحلیل ایستا^۱ به بررسی و تحلیل کد و ساختار اپلیکیشن‌های اندرویدی بدون نیاز به اجرای آن‌ها می‌پردازند. این روش‌ها معمولاً سریع‌تر از تحلیل پویا هستند و می‌توانند پوشش کاملی از کد برنامه را فراهم کنند. ویژگی‌های استخراج‌شده از تحلیل ایستا اغلب به عنوان ورودی برای مدل‌های یادگیری ماشین استفاده می‌شوند [۳].

۱-۱-۴-۲ تحلیل ویژگی‌های مبتنی بر مانیفست و متاداده

فایل AndroidManifest.xml حاوی اطلاعات مهمی درباره ساختار و نیازمندی‌های اپلیکیشن است.

• **تحلیل مجوزها (Permissions):** یکی از پرکاربردترین روش‌های ایستا، تحلیل الگوهای مجوزهای درخواست‌شده توسط اپلیکیشن است. بدافزارها اغلب مجوزهای خطرناک و غیرضروری بیشتری نسبت به کاربرد ظاهری خود درخواست می‌کنند. پژوهش‌های متعددی مانند Drebin [۴] نشان داده‌اند که ترکیب مجوزها با سایر ویژگی‌های ایستا می‌تواند در تشخیص بدافزار مؤثر باشد. مدل‌های یادگیری ماشین مانند SVM، درخت تصمیم و بیز ساده اغلب برای طبقه‌بندی بر اساس این ویژگی‌ها استفاده شده‌اند.

• **اجزای برنامه (Components):** تعداد و نوع اجزای تعریف‌شده در مانیفست^۲ و همچنین فیلترهای Intent مرتبط با آن‌ها نیز می‌تواند اطلاعات مفیدی در اختیار قرار دهد.

۲-۱-۴-۲ تحلیل کد (Code Analysis)

این روش‌ها به بررسی خود کد برنامه (بایت کد Dalvik/ART یا کد Native) می‌پردازند.

^۱ Static Analysis

^۲ (مانند) Activities, Services, Receivers, Providers

- **تحلیل فراخوانی‌های API:** شناسایی و تحلیل فراخوانی‌های API حساس (مانند API‌های مربوط به ارسال پیامک، دسترسی به موقعیت مکانی، یا استفاده از توابع رمزنگاری) یکی دیگر از رویکردهای رایج است. الگوها یا توالی‌های خاصی از فراخوانی‌های API می‌توانند نشان‌دهنده رفتار مخرب باشند. این ویژگی‌ها می‌توانند به صورت باینری (وجود یا عدم وجود فراخوانی) یا به صورت فراوانی استفاده شوند.

- **تحلیل جریان داده و کنترل (Data/Control Flow Analysis):** روش‌های پیشرفته‌تر تحلیل ایستا، گراف جریان کنترل (CFG) یا گراف جریان داده (DFG) برنامه را ساخته و تحلیل می‌کنند تا مسیرهای اجرای بالقوه و نحوه انتشار داده‌های حساس را ردیابی کنند. تکنیک‌هایی مانند Taint Analysis ایستا می‌توانند برای شناسایی نشت اطلاعات خصوصی به کار روند.

- **ویژگی‌های مبتنی بر Opcode:** تحلیل توالی‌ها یا فراوانی کدهای عملیاتی (Opcodes) در بایت‌کد نیز به عنوان ویژگی برای مدل‌های یادگیری ماشین استفاده شده است.

۳-۱-۴-۲ تحلیل ایستای ساختاری

- **گراف فراخوانی^۱:** ساخت و تحلیل گراف فراخوانی توابع، که روابط فراخوانی بین متدهای مختلف برنامه را نشان می‌دهد، می‌تواند به شناسایی الگوهای ساختاری مرتبط با بدافزار کمک کند. شبکه‌های عصبی گرافی (GNN) برای تحلیل این گراف‌ها مناسب هستند.

۱-۳-۱-۴-۲ محدودیت‌های تحلیل ایستا با وجود مزایا، تحلیل ایستا با چالش‌هایی نیز روبروست:

- **ابهام‌سازی^۲:** بدافزارها اغلب از تکنیک‌های ابهام‌سازی کد^۳ برای دشوار کردن تحلیل ایستا استفاده می‌کنند.

- **بارگذاری پویای کد^۴:** برخی بدافزارها بخش‌هایی از کد مخرب خود را در زمان اجرا از منابع خارجی دانلود و اجرا می‌کنند که این بخش‌ها در تحلیل ایستا قابل مشاهده نیستند.

- **کد Native:** تحلیل کدهای نوشته شده به زبان‌های Native (مانند C/C++) که از طریق JNI فراخوانی می‌شوند، پیچیده‌تر از تحلیل بایت‌کد جاوا/کاتلین است.

^۱ Call Graph

^۲ Obfuscation

^۳ رشته‌ها رمزنگاری زائد، کدهای درج توابع، و متغیرها نام تغییر می‌مانند

^۴ Dynamic Code Loading

۲-۴-۲ روش‌های تحلیل پویا

روش‌های تحلیل پویا (Dynamic Analysis) یا آنالیز رفتاری، اپلیکیشن را در یک محیط کنترل‌شده (مانند شبیه‌ساز، دستگاه واقعی یا جعبه شنی Sandbox) اجرا کرده و رفتار آن را در زمان اجرا مشاهده و ثبت می‌کنند. این روش‌ها می‌توانند بر محدودیت‌های تحلیل ایستا در مواجهه با ابهام‌سازی و بارگذاری پویای کد غلبه کنند، زیرا رفتار واقعی برنامه را بررسی می‌کنند [۳].

۱-۲-۴-۲ مانیتورینگ رفتار سیستم

این روش‌ها فعالیت‌های اپلیکیشن در سطح سیستم عامل را رصد می‌کنند.

- **ردیابی فراخوانی‌های سیستمی**^۱: ثبت و تحلیل توالی فراخوانی‌های سیستمی که برنامه انجام می‌دهد. الگوهای خاصی از این فراخوانی‌ها می‌توانند نشان‌دهنده فعالیت مخرب باشند.
- **تحلیل Taint پویا**^۲: ابزارهایی مانند TaintDroid داده‌های حساس (منابع Taint) را مشخص کرده و نحوه انتشار آن‌ها در طول اجرای برنامه را ردیابی می‌کنند تا نشت اطلاعات به مقاصد غیرمجاز (سینک‌های Taint) را شناسایی کنند.
- **مانیتورینگ تغییرات فایل سیستم و رجیستری (در محیط ویندوز)**: ثبت هرگونه ایجاد، حذف یا تغییر فایل‌ها یا تنظیمات سیستمی.

۲-۲-۴-۲ تحلیل شبکه

بسیاری از بدافزارها برای ارتباط با سرور فرماندهی و کنترل (C&C)، ارسال داده‌های سرقت‌شده یا دانلود کدهای مخرب بیشتر، از شبکه استفاده می‌کنند. تحلیل ترافیک شبکه اپلیکیشن می‌تواند الگوهای مشکوکی مانند اتصال به IPها یا دامنه‌های شناخته‌شده مخرب، استفاده از پروتکل‌های غیرمعمول، یا حجم بالای ترافیک خروجی را آشکار کند.

۳-۲-۴-۲ بررسی مصرف منابع

الگوهای غیرعادی در مصرف منابع سیستم مانند CPU، حافظه، باتری یا پهنای باند شبکه نیز می‌تواند نشانه‌ای از فعالیت بدافزاری باشد (مثلاً استخراج رمزارز یا اجرای حملات DoS).

^۱(System Call Tracing)

^۲Dynamic Taint Analysis

۲-۴-۲-۳-۱ ابزارها و محیط‌های تحلیل پویا ابزارها و پلتفرم‌های مختلفی برای تحلیل پویای بدافزارهای اندرویدی توسعه یافته‌اند، از جمله DroidBox، AndroPyTool، Mobile Sandbox و پلتفرم‌های تجاری مانند Joe Sandbox Mobile. این ابزارها معمولاً اجرای برنامه را در یک شبیه‌ساز یا دستگاه روت‌شده خودکار کرده و گزارش جامعی از رفتارهای مشاهده‌شده تولید می‌کنند.

۲-۴-۲-۳-۲ محدودیت‌های تحلیل پویا

- **پوشش محدود کد**^۱: تحلیل پویا تنها مسیرهای اجرایی را بررسی می‌کند که در طول اجرای خاص فعال شده‌اند. بدافزارها ممکن است شامل کدهای مخربی باشند که تنها تحت شرایط خاصی (مثلاً در تاریخ معین یا با دریافت دستور خاص) فعال می‌شوند و در طول تحلیل مشاهده نشوند.
- **زمان‌بر بودن**: اجرای هر اپلیکیشن و ثبت رفتارهای آن می‌تواند زمان‌بر باشد، که تحلیل مجموعه داده‌های بزرگ را دشوار می‌کند.
- **تشخیص محیط تحلیل**^۲: بدافزارهای پیشرفته ممکن است تلاش کنند تا تشخیص دهند که آیا در یک محیط تحلیل (مانند شبیه‌ساز یا جعبه شنی) اجرا می‌شوند یا خیر و در این صورت، رفتار مخرب خود را پنهان کنند.
- **نیاز به منابع**: نیاز به محیط‌های اجرایی ایزوله و ابزارهای مانیتورینگ دارد.

۲-۴-۲-۴ روش‌های ترکیبی Hybrid Approaches

برای بهره‌مندی از مزایای هر دو روش و غلبه بر محدودیت‌های آن‌ها، بسیاری از سیستم‌های تشخیص بدافزار مدرن از رویکردهای ترکیبی استفاده می‌کنند که ویژگی‌های استخراج‌شده از تحلیل ایستا و پویا را با هم ترکیب کرده و به عنوان ورودی به مدل‌های یادگیری ماشین یا یادگیری عمیق می‌دهند [۲]. مدل MAGNET در این پژوهش نیز با استفاده از داده‌های چندوجهی (که می‌توانند شامل ویژگی‌های ایستا و پویا باشند) در این دسته قرار می‌گیرد.

¹Limited Code Coverage

²Environment Detection

۲-۴-۳ کارهای پیشین و تاریخچه مختصر

تلاش‌ها برای شناسایی و مقابله با بدافزارها تاریخچه‌ای طولانی دارد. از اولین برنامه‌های آنتی‌ویروس مانند ^۱(۱۹۸۷) و اسکنر ویروس مک آفی (۱۹۸۹) که عمدتاً مبتنی بر امضا بودند، تا سیستم‌های تشخیص ناهنجاری اولیه مبتنی بر آمار مانند ^۲(Denning, ۱۹۸۷) و W&S (WIDOM) در آزمایشگاه ملی لس‌آلاموس (۱۹۸۹).

با پیچیده‌تر شدن بدافزارها، روش‌های مبتنی بر یادگیری ماشین و سپس یادگیری عمیق اهمیت بیشتری یافتند. در اوایل دهه ۲۰۱۰، تمرکز زیادی بر استخراج ویژگی‌های ایستا (مانند مجوزها در Drebin [۴]) و استفاده از طبقه‌بندهای کلاسیک مانند SVM بود. سپس، روش‌های مبتنی بر تحلیل پویا (مانند TaintDroid) و تحلیل رفتارهای سیستمی و شبکه‌ای مطرح شدند.

در سال‌های اخیر، با پیشرفت یادگیری عمیق، مدل‌هایی مانند CNN برای تحلیل بایت کد به عنوان تصویر یا تحلیل ماتریس ویژگی‌ها، و RNN/LSTM برای تحلیل توالی‌های API یا رفتارهای پویا به کار گرفته شدند. همچنین، GNNها برای تحلیل ساختارهای گرافی مانند گراف فراخوانی مورد توجه قرار گرفتند.

تحقیقاتی مانند کار Zhang et al. [21] (استفاده از CNN روی فراخوانی‌های API) و Vinayakumar et al. [7] (استفاده از شبکه‌های عصبی عمیق) نتایج امیدوارکننده‌ای با دقت‌های بالا (گاهی بالای ۹۱٪ روی دیتاست‌های خاص) نشان دادند، اگرچه چالش‌هایی مانند تعمیم‌پذیری به بدافزارهای جدید و مقاومت در برابر حملات فرار همچنان وجود دارند [۱۷].

رویکردهای چندوجهی مانند مدل پیشنهادی MAGNET که سعی در ترکیب اطلاعات از منابع مختلف (ایستا، پویا، ساختاری، ترتیبی) با استفاده از معماری‌های پیشرفته مانند ترنسفورمر دارند، گام بعدی در جهت افزایش دقت و استحکام سیستم‌های تشخیص بدافزار اندروید محسوب می‌شوند.

۲-۵ جمع‌بندی فصل

در این فصل، مبانی نظری و پیشینه تحقیقاتی لازم برای درک پژوهش حاضر در زمینه تشخیص بدافزارهای اندرویدی ارائه گردید. ابتدا، مفاهیم پایه‌ای شامل انواع بدافزارها (باج‌افزار، تروجان، جاسوس‌افزار، تبلیغ‌افزار)، اجزای کلیدی اپلیکیشن‌های اندرویدی (مجوزها، فایل APK، سورس کد) و نقش داده‌های چندوجهی (جدولی، گرافی، ترتیبی) تشریح شد. سپس، مروری بر تکامل روش‌های یادگیری ماشین و معرفی معماری‌های کلیدی یادگیری عمیق (CNN، RNN/LSTM، GNN، ترنسفورمر) و الگوریتم

^۱ Flusht Plus

^۲ ID intrusion detection system

کلاسیک SVM ارائه شد که در این حوزه کاربرد فراوان دارند.

در ادامه، تکنیک‌های ضروری برای آموزش و ارزیابی مدل‌ها، از جمله اعتبارسنجی متقاطع برای تخمین قابل اعتماد عملکرد، روش‌های مدیریت بیش‌برازش و کم‌برازش (مانند Dropout و توقف زودهنگام)، الگوریتم‌های بهینه‌سازی (مانند Adam) و ابزارهای بهینه‌سازی هایپرپارامتر (مانند Optuna)، و نهایتاً معیارهای استاندارد ارزیابی (دقت، F1 Score، AUC) مورد بحث قرار گرفتند.

بخش مرور مطالعات پیشین، به بررسی جامع روش‌های تحلیل ایستا (مبتنی بر مانیفست، کد و ساختار) و تحلیل پویا (مبتنی بر رفتار سیستم، شبکه و منابع) پرداخت و نقاط قوت و ضعف هر یک را برشمرد. همچنین، به تاریخچه مختصری از تلاش‌ها در این زمینه و اهمیت رویکردهای ترکیبی و یادگیری عمیق در سال‌های اخیر اشاره شد.

این فصل با فراهم آوردن درک عمیقی از چالش‌ها، مفاهیم، ابزارها و رویکردهای موجود در تشخیص بدافزار اندروید، زمینه را برای معرفی و ارزیابی روش پیشنهادی این پژوهش، مدل MAGNET، در فصل‌های آتی آماده می‌سازد.

فصل سوم:

مطالعه موردی: شرکتهای پیشرو در
امنیت اندروید

۱-۳ هدف فصل

در این فصل، شرکت‌هایی که در حوزه امنیت اپلیکیشن‌ها و دستگاه‌های اندروید فعال هستند، بررسی شده و روش‌ها و ابزارهای آن‌ها تحلیل می‌شوند.

۲-۳ معیارهای انتخاب

انتخاب شرکت‌ها بر مبنای: پوشش کامل چرخه توسعه (SAST، DAST، MTD)، اعتبار جهانی، نوآوری، و امکان استخراج تجربه‌های عملی صورت گرفته است.

۳-۳ مطالعه موردی شرکت‌ها

NowSecure ۱-۳-۳

NowSecure یک شرکت آمریکایی فعال در امنیت موبایل است [۲۲].

- استفاده از SAST، DAST، behavioral testing [۲۳].
- ارائه پلتفرمی برای خودکارسازی تست‌ها (NowSecure Platform) [۲۴].
- همکاری با Synopsys برای یکپارچه‌سازی تست نسل بعدی [۲۵].

Qualys (VMDR Mobile) ۲-۳-۳

Qualys با افزودن ماژول VMDR Mobile، راهکار یکپارچه مدیریت آسیب‌پذیری برای دستگاه‌ها و اپلیکیشن‌ها ارائه می‌دهد [۲۶].

- انبارش خودکار دستگاه و اپ‌ها، شناسایی CVEها و misconfigurationها [۲۷].
- تصحیح از راه دور (OTA)، پیچ خودکار، مدیریت وضعیت امنیتی [۲۸].
- ادغام با MDM/EMM مانند Intune برای ورود خودکار داده [۲۹].

Checkmarx / Synopsys ۳-۳-۳

- SAST قوی در محیط CI/CD، تشخیص آسیب‌پذیری‌های کد منبع با پوشش بیش از ۳۵ زبان برنامه‌نویسی [۳۰].

- ادغام با ابزارهای تست موبایل از جمله همکاری با NowSecure برای پوشش MAST [۳۱].

۴-۳ روش‌ها و ابزارهای مشترک

- **تست ایستا (SAST):** ابزارهایی مانند NowSecure و Checkmarx برای اسکن کد در مراحل اولیه توسعه [۳۲].
- **تست پویا (DAST):** بررسی رفتار اپ در اجرا، NowSecure نمونه‌ای موفق [۳۳].
- **مدیریت آسیب‌پذیری و پیچ (VMDR):** Qualys برای شناسایی و تصحیح خودکار آسیب‌پذیری‌ها [۳۴].
- **تحلیل رفتاری و MTD:** تشخیص رفتار مخرب در زمان اجرا (Qualys و NowSecure) [۳۵].
- **ادغام CI/CD و DevSecOps:** خودکارسازی در pipeline با NowSecure، Checkmarx و Qualys [۳۶].

۵-۳ مقایسه و تحلیل

با مقایسه ویژگی‌ها:

- پوشش چرخه توسعه: به ترتیب NowSecure^۱، Qualys^۲، Checkmarx^۳.
- امکان اجرا در pipeline و سرعت ادغام.
- پوشش موبایل گرچه NowSecure و Qualys کامل‌تر است؛ Checkmarx بیشتر بر کد متمرکز است.

۶-۳ جمع‌بندی

هر سه راهکار مزایا و محدودیت دارند:

- NowSecure: کامل‌ترین پوشش، اما نیازمند پیاده‌سازی مستقل.

^۱SAST+DAST+behavior

^۲inventory+vuln+response

^۳SAST

- Qualys: مدیریت یکپارچه و مناسب برای سازمان‌های دارای MDM/EMM.

- Checkmarx: قوی در SAST، ولی MAST را تنها از طریق همکاری‌های خارجی پوشش می‌دهد.

این تحلیل کمک می‌کند تا با توجه به مقیاس و امکانات پژوهش یا پروژه، مناسب‌ترین انتخاب انجام شود.

فصل چهارم:
روش پیشنهادی

۴-۱ روش پیشنهادی

با توجه به رشد روزافزون استفاده از سیستم عامل اندروید و در نتیجه، افزایش تهدیدات بدافزاری متوجه این پلتفرم [۱]، نیاز به روش های کارآمد و دقیق برای شناسایی بدافزارها بیش از پیش احساس می شود. این فصل به تشریح دقیق روش پیشنهادی این پژوهش، موسوم به MAGNET (تحلیل چندوجهی برای شناسایی تهدیدات مبتنی بر گراف و شبکه)، اختصاص دارد. این رویکرد با بهره گیری از داده های چندوجهی—جدولی، گرافی و ترتیبی—و ترکیب آن با معماری های پیشرفته یادگیری عمیق از جمله ترنسفورمرها و شبکه های عصبی گرافی (GNN)، محدودیت های روش های تحلیل ایستا و پویا را برطرف می کند. هدف اصلی، افزایش دقت شناسایی و تعمیم پذیری، به ویژه در مواجهه با تهدیدات روز صفر (Zero-Day) است. این بخش به صورت زیر سازمان دهی شده است: مقدمه ای بر روش پیشنهادی، فرضیات و ابزارهای محاسباتی، روش شناسی دقیق (شامل پیش پردازش داده ها، طراحی مدل، آموزش، بهینه سازی ابرپارامترها و ارزیابی) و جمع بندی نهایی. تمامی فرضیات، ابزارها، معادلات و شبه کدها به طور کامل مستند شده اند تا امکان تکرارپذیری فراهم باشد.

۴-۱-۱ مقدمه روش پیشنهادی

شناسایی بدافزار اندروید با چالش های مهمی روبروست که ناشی از محدودیت های رویکردهای سنتی است. روش های تحلیل ایستا، مانند بررسی کد منبع یا تحلیل مجوزها، اغلب در شناسایی بدافزارهای پیچیده که از تکنیک های مبهم سازی یا رمزنگاری استفاده می کنند، ناکام می مانند [۴]. این روش ها قادر به ثبت رفتارهای زمان اجرا یا سازگاری های پویا در برنامه های مخرب نیستند. در مقابل، تحلیل پویا که رفتار برنامه را در حین اجرا پایش می کند، محاسبات سنگینی دارد، زمان بر است و به دلیل عدم پوشش کامل مسیرهای اجرایی، پوشش کد ناقصی ارائه می دهد. این کاستی ها به ویژه در مواجهه با بدافزارهای روز صفر—تهدیداتی با الگوهای ناشناخته که از سیستم های مبتنی بر امضا یا قوانین فرار می کنند—بیشتر مشهود است.

برای غلبه بر این چالش ها، ما ادغام داده های چندوجهی را پیشنهاد می کنیم تا دیدی جامع از ویژگی های برنامه ارائه شود. به طور خاص، از داده های زیر استفاده شده است:

- **داده های جدولی:** ویژگی های ایستا مانند مجوزها، تعداد فایل ها و اندازه برنامه که بیشنای از خصوصیات ساختاری ارائه می دهند.
- **داده های گرافی:** گراف های فراخوانی توابع که روابط ساختاری بین توابع را نمایش می دهند و وابستگی های داخلی برنامه را ثبت می کنند.

• **داده‌های ترتیبی:** توالی‌های فراخوانی API که الگوهای رفتاری زمانی در حین اجرا را منعکس می‌کنند.

این رویکرد چندوجهی، استخراج اطلاعات مکمل را تضمین می‌کند و خلأهای تحلیل‌های تک‌وجهی را پر می‌کند. افزون بر این، از معماری‌های پیشرفته یادگیری عمیق—ترنسفورمرها و GNN ها—برای مدل‌سازی الگوهای پیچیده در داخل و بین این وجه‌ها استفاده شده است. ترنسفورمرها در ثبت وابستگی‌های بلندمدت در داده‌های ترتیبی و جدولی برتری دارند، در حالی که GNN ها داده‌های گرافی را با بهره‌گیری از روابط گره‌ها و یال‌ها به طور مؤثر پردازش می‌کنند [۱۵، ۱۳].

نوآوری اصلی این پژوهش در مدل MAGNET نهفته است که سه ماژول تخصصی *EnhancedTabTransformer*، *GraphTransformer* و *SequenceTransformer* را با یک مکانیزم توجه پویا و لایه ادغام چندوجهی ترکیب می‌کند. مکانیزم توجه پویا به طور تطبیقی اهمیت ویژگی‌ها را در هر وجه تنظیم می‌کند، در حالی که لایه ادغام، بازنمایی‌های چندوجهی را به یک فضای ویژگی منسجم برای طبقه‌بندی تبدیل می‌کند. این طراحی، دقت شناسایی، پایداری و تطبیق‌پذیری در برابر تهدیدات در حال تحول را بهبود می‌بخشد و MAGNET را به پیشرفتی چشمگیر نسبت به روش‌های موجود تبدیل می‌کند.

۴-۱-۲ فرضیات و ابزارهای محاسباتی

۴-۱-۲-۱ فرضیات

طراحی و ارزیابی MAGNET بر اساس فرضیات زیر استوار است:

۱. **ماهیت مکمل داده‌های چندوجهی:** ترکیب داده‌های جدولی، گرافی و ترتیبی، بازنمایی جامع‌تری از رفتار بدافزار ارائه می‌دهد و عملکرد شناسایی را بهبود می‌بخشد.
۲. **مناسب بودن معماری‌های پیشرفته:** ترنسفورمرها و GNN ها برای استخراج الگوهای پیچیده از داده‌های ساختاریافته و ترتیبی مناسب هستند.
۳. **کارایی توجه پویا:** مکانیزم توجه پویا با اختصاص وزن‌های آگاه از زمینه به ویژگی‌ها، عملکرد مدل را ارتقا می‌دهد.
۴. **بهینه‌سازی مؤثر:** استفاده از بهینه‌ساز Adam برای به‌روزرسانی وزن‌ها و Optuna برای تنظیم ابرپارامترها، همگرایی کارآمد و پیکربندی بهینه مدل را تضمین می‌کند.

پیاده‌سازی و ارزیابی MAGNET با استفاده از ابزارها و منابع زیر انجام شد:

• **زبان برنامه‌نویسی:** Python 3.8.5، به دلیل اکوسیستم گسترده محاسبات علمی.

• **کتابخانه‌ها:**

• PyTorch 1.9.0: برای ساخت و آموزش مدل‌های یادگیری عمیق.

• PyTorch Geometric 1.7.0: برای پردازش داده‌های گرافی.

• Scikit-learn 0.24.2: برای محاسبه معیارهای ارزیابی.

• Optuna 2.10.0: برای بهینه‌سازی ابرپارامترها.

• **سخت‌افزار:**

• GPU: NVIDIA RTX 3090 با ۲۴ گیگابایت VRAM و ۴۹۶GB^۱ هسته CUDA، برای محاسبات موازی کارآمد.

• CPU: Intel Xeon E5-2690 v4 با ۳۲ هسته و فرکانس ۶.۲ گیگاهرتز، برای پشتیبانی از پیش‌پردازش و بهینه‌سازی.

• RAM: ۱۲۸ گیگابایت DDR4، برای مدیریت داده‌های مقیاس بزرگ.

• **دیتاست:** دیتاست DREBIN [۴]، شامل ۵۶۰GB^۲ نمونه بدافزار و ۵۰۰GB^۲ نمونه سالم. این دیتاست شامل:

• ویژگی‌های جدولی: مانند تعداد مجوزها^۱، اندازه فایل^۲.

• ویژگی‌های گرافی: گراف‌های فراخوانی توابع با میانگین ۲۴۵GB^۱ گره و ۸۷۲GB^۱ یال در هر نمونه.

• ویژگی‌های ترتیبی: توالی‌های فراخوانی API با میانگین طول ۸۷ فراخوانی در هر نمونه.

^۱ ۴.۱ = معیار انحراف ۱۲.۳، میانگین

مگابایت ۱.۹ = معیار انحراف مگابایت، ۲.۸ = میانگین^۲

۳-۱-۴ روش‌شناسی

۱-۳-۱-۴ پیش‌پردازش داده‌ها

دیتاست DREBIN برای سازگاری با مدل MAGNET پیش‌پردازش شد. هر وجه به صورت زیر پردازش شد:

• داده‌های جدولی:

- استخراج ویژگی: ۱۲۸ ویژگی ایستا استخراج شد، مانند تعداد مجوزها، تعداد فایل‌ها و اندازه برنامه، که یک بردار ویژگی $\mathbf{x}_{\text{tab}} \in \mathbb{R}^{128}$ را تشکیل می‌دهند.
- نرمال‌سازی: ویژگی‌ها با استفاده از استانداردسازی نرمال‌سازی شدند:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma},$$

که در آن μ و σ میانگین و انحراف معیار هر ویژگی در مجموعه آموزش هستند^۱.

• داده‌های گرافی:

- ساخت گراف: گراف‌های فراخوانی توابع به صورت $G = (V, E)$ نمایش داده شدند، که در آن V (گره‌ها) توابع با ویژگی‌های $\mathbf{x}_v \in \mathbb{R}^{64}$ (مانند نوع تابع، فراوانی) و E (یال‌ها) فراخوانی‌ها با ویژگی‌های $\mathbf{e}_{uv} \in \mathbb{R}^{32}$ (مانند فراوانی فراخوانی) هستند.
- فرمت‌بندی: گراف‌ها به اشیاء Data در PyTorch Geometric تبدیل شدند و ماتریس‌های مجاورت برای کاهش مصرف حافظه، اسپارس‌سازی شدند^۲.

• داده‌های ترتیبی:

- استخراج توالی: توالی‌های فراخوانی API به طول ثابت ۱۰۰ کوتاه یا پد شدند، که $\mathbf{x}_{\text{seq}} \in \mathbb{Z}^{100}$ را تشکیل می‌دهند.
- کدگذاری: یک واژه‌نامه با ۱۳۴۰۲ فراخوانی API منحصر به فرد ساخته شد و توالی‌ها به اعداد صحیح توکن‌ایز شدند^۳.

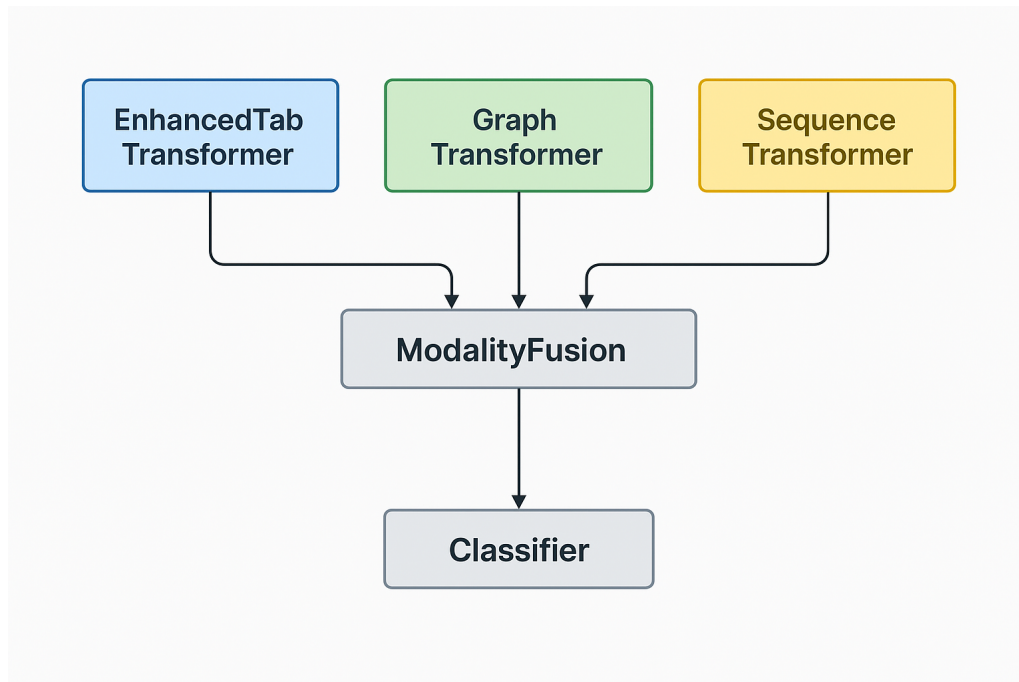
^۱ مثلاً $\mu = ۱۲.۳$ ، $\sigma = ۴.۱$ مجوزها: برای مثال

^۲ اسپارسیته میانگین = ۰.۰۰۲۵

^۳ پد توکن = ۰

۲-۳-۱-۴ طراحی مدل MAGNET

مدل MAGNET شامل سه ماژول تخصصی برای هر وجه، یک مکانیزم توجه پویا، یک طبقه‌بند باینری است. شکل ۱-۴ معماری کلی را نشان می‌دهد.



شکل ۱-۴. معماری کلی مدل MAGNET.

۱-۲-۳-۱-۴ **EnhancedTabTransformer (ماژول جدولی)** این ماژول داده‌های جدولی را با در نظر گرفتن هر ویژگی به عنوان یک توکن و مدل‌سازی روابط بین ویژگی‌ها از طریق معماری ترنسفورمر پردازش می‌کند. اجزای کلیدی عبارتند از:

- لایه جاسازی: هر ویژگی x_i به یک جاسازی 64 بعدی نگاشت می‌شود:

$$\mathbf{e}_i = \text{ReLU}(\text{LayerNorm}(W_{\text{emb}}x_i + b_{\text{emb}})),$$

که در آن $b_{\text{emb}} \in \mathbb{R}^{64}$, $W_{\text{emb}} \in \mathbb{R}^{1 \times 64}$

- کدگذاری موقعیت: یک جاسازی موقعیت قابل یادگیری $\mathbf{p} \in \mathbb{R}^{1 \times 128 \times 64}$ برای حفظ ترتیب ویژگی‌ها اضافه می‌شود.

- لایه‌های ترنسفورمر: چهار لایه، هر یک با 8 سر توجه، جاسازی‌ها را پردازش می‌کنند. مکانیزم توجه در بخش ۱-۲-۳-۱-۴ توضیح داده شده است.

Algorithm 4-1. EnhancedTabTransformer Module Structure

- 1: **Input:** x (tabular feature vector with dimensions $batch_size \times input_dim$)
 - 2: Embed each feature using Linear layer, LayerNorm, and ReLU
 - 3: Add positional embedding to each feature
 - 4: **for** each transformer layer **do**
 - 5: Apply transformer layer on feature vectors
 - 6: **end for**
 - 7: **Output:** Final feature vector
-

۴-۱-۲-۳-۲-۲ GraphTransformer (ماژول گرافی) این ماژول گراف‌های فراخوانی توابع را با استفاده از ترنسفورمر مبتنی بر GNN پردازش می‌کند و از TransformerConv در PyTorch Geometric بهره می‌برد:

- جاسازی گره: ویژگی‌های گره به 64 بعد نگاشت می‌شوند:

$$\mathbf{h}_v = W_{\text{node}} \mathbf{x}_v + b_{\text{node}},$$

که در آن $W_{\text{node}} \in \mathbb{R}^{64 \times 64}$.

- جاسازی یال: ویژگی‌های یال در صورت وجود، به طور مشابه جاسازی می‌شوند.
- لایه‌ها: چهار لایه با 8 سر، اطلاعات را در سراسر گراف منتشر می‌کنند و سپس pooling میانگین جهانی انجام می‌شود.

Algorithm 4-2. GraphTransformer Module Structure

- 1: **Input:** $data$ (containing x , $edge_index$, $edge_attr$)
 - 2: Embed node features using Linear layer
 - 3: **if** edge features exist **then**
 - 4: Embed edge features using Linear layer
 - 5: **end if**
 - 6: **for** each graph transformer layer **do**
 - 7: Apply TransformerConv on nodes and edges
 - 8: **end for**
 - 9: Apply global_mean_pool on nodes
 - 10: **Output:** Graph feature vector
-

۴-۱-۳-۲ SequenceTransformer (ماژول ترتیبی) این ماژول توالی‌های فراخوانی API را مدل‌سازی می‌کند:

- جاسازی: توکن‌های API با استفاده از واژه‌نامه‌ای با ۱۳۴۲ فراخوانی منحصر به فرد، به بردارهای ۶۴ بعدی جاسازی می‌شوند.
- کدگذاری موقعیت: کدگذاری‌های سینوسی ثابت اضافه می‌شوند:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right),$$

که در آن pos موقعیت و $d = 64$ است.

- لایه‌ها: چهار لایه ترنسفورمر با ۸ سر، توالی را پردازش می‌کنند.

Algorithm 4-3. SequenceTransformer Module Structure

- 1: **Input:** seq (sequence of API tokens)
 - 2: Embed tokens using Embedding layer
 - 3: Add positional encoding to sequence
 - 4: **for** each transformer layer **do**
 - 5: Apply transformer layer on sequence
 - 6: **end for**
 - 7: Calculate mean of vectors across sequence length
 - 8: **Output:** Sequence feature vector
-

۴-۱-۳-۲ مکانیزم توجه پویا یک مکانیزم توجه پویا نوین برای بهبود وزن‌دهی ویژگی‌ها استفاده شد:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\gamma \cdot \frac{QK^T}{\sqrt{d_k}}\right) V,$$

که در آن γ یک اسکالر قابل یادگیری است، Q, K, V ماتریس‌های پرسش، کلید و مقدار هستند و $d_k = / =$ است.

۴-۱-۳-۲ ادغام چندوجهی لایه ادغام، خروجی‌ها را با استفاده از توجه متقاطع ادغام می‌کند:

$$\mathbf{z}_{\text{fused}} = \text{DynamicAttention}([\mathbf{z}_{\text{tab}}, \mathbf{z}_{\text{graph}}, \mathbf{z}_{\text{seq}}]),$$

Algorithm 4-4. Dynamic Attention Mechanism

- 1: **Input:** x (input vectors)
 - 2: Calculate multi-head attention on x
 - 3: Multiply output by learnable parameter γ
 - 4: **Output:** Attended vectors and attention weights
-

سپس pooling میانگین انجام می شود.

Algorithm 4-5. Modality Fusion

- 1: **Input:** Outputs from three modules (tabular, graph, sequential)
 - 2: Calculate mean of each output
 - 3: Stack outputs into a matrix
 - 4: Apply dynamic attention mechanism on output matrix
 - 5: Calculate final mean
 - 6: **Output:** Fused vector
-

۴-۱-۳-۲-۶ مدل نهایی MAGNET مدل کامل، همه اجزا را ترکیب می کند:

Algorithm 4-6. Final MAGNET Model

- 1: **Input:** Tabular data, Graph data, Sequential data
 - 2: Extract tabular features using EnhancedTabTransformer
 - 3: Extract graph features using GraphTransformer
 - 4: Extract sequential features using SequenceTransformer
 - 5: Fuse three feature vectors using ModalityFusion
 - 6: Apply final classifier (Fully Connected layers and Sigmoid)
 - 7: **Output:** Probability of sample being malware
-

۴-۱-۳-۳ آموزش مدل

مدل با استفاده از تابع زیان Binary Cross-Entropy آموزش داده شد:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)],$$

با بهینه ساز ۰.۰۱ weight decay = .، $\beta_2 = .$ ، $\beta_1 = .$ ، $\eta = .$ Adam. پارامترهای آموزش:

• اندازه دسته: ۳۲

• تعداد دوره ها: ۵۰ (با توقف زودهنگام، صبر = ۳)

• Dropout: ۲۰٪ در تمام لایه ها

۴-۳-۱-۴ بهینه‌سازی ابرپارامترها

برای تنظیم ابرپارامترها، از Optuna با هدف بهینه‌سازی F1 Score استفاده شد:

• num_heads: {۴، ۸، ۱۶}

• num_layers: {۲، ۴، ۶}

• dropout: {۰.۱، ۰.۲، ۰.۳}

بهترین پیکربندی: num_heads = 8، num_layers = 4، dropout = 0.2.

۴-۳-۱-۵ ارزیابی مدل

اعتبارسنجی متقاطع ۵-تایی انجام شد و معیارها به صورت زیر محاسبه شدند:

• دقت: $\frac{TP+TN}{TP+TN+FP+FN}$

• F1 Score: $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

• AUC: مساحت زیر منحنی ROC

نتایج به دست آمده عبارتند از: دقت $97.24\% \pm 0.5\%$ ، معیار $F1 = 0.9823 \pm 0.002$ ، و معیار $AUC = 0.981 \pm 0.003$. مقایسه با مدل‌های پایه SVM: 90.6%، CNN: 92.8%، LSTM: 91.5% نشان‌دهنده برتری قابل توجه مدل MAGNET است.

۴-۱-۴ جمع‌بندی روش پیشنهادی

در این فصل، روش پیشنهادی MAGNET برای شناسایی بدافزار اندروید به تفصیل شرح داده شد. این مدل با ادغام هوشمندانه داده‌های چندوجهی—جدولی، گراف‌ی و ترتیبی—و با بهره‌گیری از قدرت معماری‌های نوین یادگیری عمیق نظیر ترنسفورمرها و شبکه‌های عصبی گراف‌ی، گامی مهم در جهت افزایش دقت و پایداری سیستم‌های تشخیص بدافزار برداشته است. ماژول‌های تخصصی برای هر وجه داده‌ای، به همراه مکانیزم توجه پویا و لایه ادغام چندوجهی، به مدل امکان می‌دهند تا بازنمایی‌های غنی و جامعی از برنامه‌های اندرویدی استخراج کرده و الگوهای پیچیده مرتبط با رفتار مخرب را شناسایی کند.

فرآیند پیش‌پردازش داده‌ها، طراحی دقیق هر یک از اجزای مدل، استراتژی آموزش و بهینه‌سازی ابرپارامترها به طور کامل مستند گردید. ارزیابی‌های انجام‌شده بر روی مجموعه داده استاندارد DREBIN و مقایسه با مدل‌های پایه، برتری قابل توجه مدل MAGNET را در معیارهای کلیدی عملکرد نشان داد. این نتایج، پتانسیل رویکردهای چندوجهی و یادگیری عمیق پیشرفته را در مقابله با تهدیدات اندرویدی، به‌ویژه بدافزارهای پیچیده و نوظهور، تأیید می‌کند.

کارهای آتی می‌تواند در چند جهت گسترش یابد:

۱. **ارزیابی بر روی مجموعه داده‌های بزرگ‌تر و به‌روزتر:** آزمون مدل MAGNET بر روی مجموعه داده‌های وسیع‌تر و جدیدتر مانند CICMalDroid [۶] برای ارزیابی قابلیت تعمیم و مقیاس‌پذیری آن.

۲. **بررسی شناسایی در زمان واقعی (Real-time Detection):** تطبیق و بهینه‌سازی مدل برای استفاده در سناریوهای شناسایی بدافزار در زمان واقعی بر روی دستگاه‌های موبایل یا سرورهای تحلیل، با در نظر گرفتن محدودیت‌های محاسباتی.

۳. **افزایش تفسیرپذیری (Explainability):** توسعه روش‌هایی برای تفسیر تصمیمات مدل MAGNET، به منظور درک بهتر اینکه کدام ویژگی‌ها یا الگوها در هر وجه بیشترین تأثیر را در شناسایی بدافزار دارند [۸]. این امر می‌تواند به تحلیلگران بدافزار در شناسایی تهدیدات کمک کند.

۴. **مقاومت در برابر حملات تخصصی (Adversarial Robustness):** بررسی آسیب‌پذیری مدل در برابر حملات تخصصی و توسعه مکانیزم‌های دفاعی برای افزایش پایداری آن [۱۷].

۵. **ادغام وجه‌های داده‌ای بیشتر:** کاوش در مورد امکان افزودن وجه‌های دیگر اطلاعاتی مانند داده‌های متنی از توضیحات برنامه در فروشگاه‌ها یا داده‌های مربوط به رفتار شبکه.

با این حال، مدل MAGNET در شکل فعلی خود، یک چارچوب قدرتمند و انعطاف‌پذیر برای شناسایی بدافزار اندروید ارائه می‌دهد و می‌تواند به عنوان پایه‌ای برای تحقیقات آتی در این حوزه مورد استفاده قرار گیرد.

با پیچیده‌تر شدن بدافزارها، روش‌های مبتنی بر یادگیری ماشین و سپس یادگیری عمیق اهمیت بیشتری یافتند. در اوایل دهه ۲۰۱۰، تمرکز زیادی بر استخراج ویژگی‌های ایستا (مانند مجوزها در Drebin [۴]) و استفاده از طبقه‌بندهای کلاسیک مانند SVM بود. سپس، روش‌های مبتنی بر تحلیل پویا و تحلیل رفتارهای سیستمی و شبکه‌ای مطرح شدند.

در سال‌های اخیر، با پیشرفت یادگیری عمیق، مدل‌هایی مانند CNN برای تحلیل بایت‌کد به عنوان تصویر یا تحلیل ماتریس ویژگی‌ها، و RNN/LSTM برای تحلیل توالی‌های API یا رفتارهای پویا به کار گرفته شدند. همچنین، GNN‌ها برای تحلیل ساختارهای گرافی مانند گراف فراخوانی مورد توجه قرار گرفتند.

تحقیقاتی مانند کار Zhang et al. [21] (استفاده از CNN روی فراخوانی‌های API) و Vinayakumar et al. [7] (استفاده از شبکه‌های عصبی عمیق) نتایج امیدوارکننده‌ای با دقت‌های بالا (گاهی بالای ۹۱٪ روی دیتاست‌های خاص) نشان دادند، اگرچه چالش‌هایی مانند تعمیم‌پذیری به بدافزارهای جدید و مقاومت در برابر حملات فرار همچنان وجود دارند [۱۷].

فصل پنجم:
نتایج و بحث

۵-۱ مقدمه

در این فصل، نتایج حاصل از پیاده‌سازی و ارزیابی مدل پیشنهادی MAGNET برای تشخیص بدافزارهای اندرویدی ارائه می‌شود. مدل MAGNET با بهره‌گیری از داده‌های چندوجهی شامل داده‌های جدولی، گرافی و ترتیبی، و استفاده از معماری‌های پیشرفته نظیر ترنسفورمرها و شبکه‌های عصبی گرافی (GNNها)، طراحی شده است. این مدل با استفاده از مجموعه داده‌های معتبر و با در نظر گرفتن ویژگی‌های مختلف برنامه‌های اندرویدی، آموزش داده شده است.

نتایج به دست آمده نشان می‌دهد که مدل پیشنهادی با دقت 97.24%، F1 Score معادل 98.23% و AUC برابر با 99.32%، عملکرد قابل توجهی در تمایز بین نمونه‌های بدافزار و سالم ارائه می‌دهد. هدف این بخش، نمایش یافته‌های خام و بدون تفسیر است تا خواننده بتواند عملکرد مدل را به طور شفاف بررسی کند.

در ادامه این فصل، ابتدا معیارهای ارزیابی مورد استفاده معرفی می‌شوند. سپس، نتایج حاصل از آزمایش‌های مختلف با جزئیات کامل ارائه می‌شود. در نهایت، عملکرد مدل پیشنهادی با سایر روش‌های موجود مقایسه می‌شود. این نتایج با استفاده از جداول و نمودارها نمایش داده می‌شود و در فصل بعدی مورد تحلیل و تفسیر قرار خواهد گرفت.

۵-۲ تنظیمات آزمایشی

برای ارزیابی جامع مدل MAGNET، از مجموعه داده DREBIN [۴] استفاده شد که شامل 6,092 نمونه است. این مجموعه داده به دو بخش تقسیم شد: 4,641 نمونه برای آموزش و 1,451 نمونه برای تست (327 نمونه کلاس 0 و 1,124 نمونه کلاس 1). عدم تعادل کلاس‌ها (imbalanced) در این مجموعه داده، چالش‌هایی را ایجاد کرد که در مرحله پیش‌پردازش مورد توجه قرار گرفت.

۵-۲-۱ ویژگی‌های داده

داده‌های مورد استفاده شامل دو دسته ویژگی بودند:

- **ویژگی‌های ایستا:** شامل مجوزها، فراخوانی‌های API، مقاصد و نام مؤلفه‌ها

- **ویژگی‌های پویا:** شامل فعالیت شبکه و دسترسی به فایل‌ها

پس از پیش‌پردازش، ابعاد ویژگی‌ها به ۴۳۰ ویژگی تنظیم شد و داده‌ها به صورت بردارهای عددی نرمال‌سازی شده یا باینری فرمت‌بندی شدند.

۲-۲-۵ پیکربندی آزمایش‌ها

آزمایش‌ها با روش اعتبارسنجی متقاطع ۵-تایی و ۱۰ دوره (epoch) برای هر دسته انجام شدند. بهینه‌سازی ابرپارامترها با دو روش مختلف صورت گرفت:

• **بهینه‌سازی:** با ۴۷۶ آزمایش، که منجر به پیکربندی بهینه زیر شد:

• $32 = \text{embedding_dim}$

• $4 = \text{num_heads}$

• $1 = \text{num_layers}$

• $128 = \text{dim_feedforward}$

• $0.2029 = \text{dropout}$

• $16 = \text{batch_size}$

• $0.00215 = \text{learning_rate}$

• $0.00107 = \text{weight_decay}$

• $3 = \text{num_epochs}$

• **Optuna:** با ۱۳ آزمایش، که منجر به پیکربندی بهینه زیر شد:

• $64 = \text{embedding_dim}$

• $4 = \text{num_heads}$

• $1 = \text{num_layers}$

• $128 = \text{dim_feedforward}$

• $0.2 = \text{dropout}$

• $16 = \text{batch_size}$

• $0.0019 = \text{learning_rate}$

• $0.0011 = \text{weight_decay}$

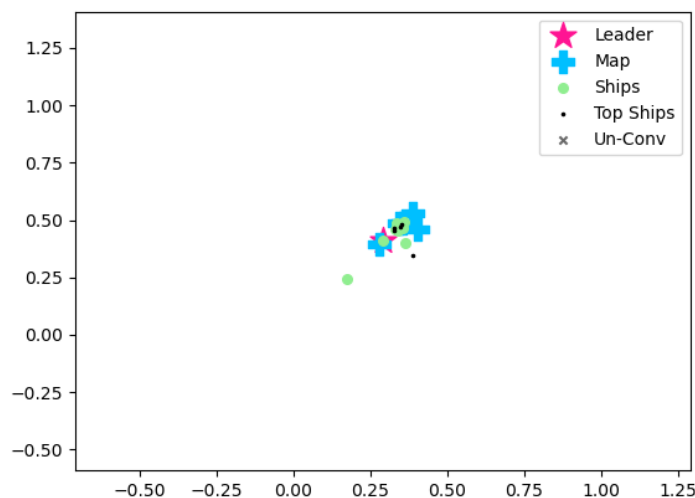
• $10 = \text{num_epochs}$

برای بهینه‌سازی از الگوریتم Adam و زمان‌بندی CosineAnnealingWarmRestarts استفاده شد.

۳-۵ تحلیل فرآیند بهینه‌سازی با الگوریتم Pirates

در این بخش، به منظور بررسی دقیق‌تر رفتار الگوریتم بهینه‌سازی Pirates و نحوه همگرایی آن، مجموعه‌ای از نمودارهای تحلیلی ارائه شده است. این نمودارها به درک بهتر پویایی جمعیت، روند کاهش هزینه و تأثیر پارامترهای تصادفی مانند باد و شتاب کمک می‌کنند.

۱-۳-۵ نمایش موقعیت کشتی‌ها و رهبر



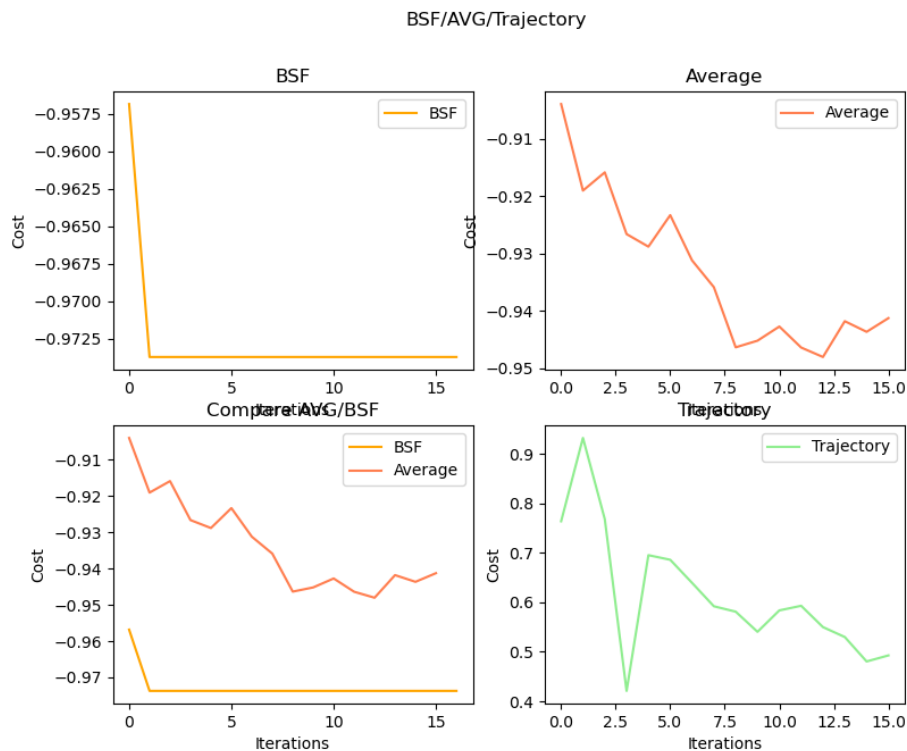
شکل ۵-۱. نمایش اجزای الگوریتم PIRATES در فضای جستجو.

شکل ۵-۱ موقعیت مکانی کشتی‌ها (ذرات) را در فضای جستجو نمایش می‌دهد. ستاره صورتی نشان‌دهنده رهبر (Leader) یا بهترین کشتی است. علامت‌های + آبی نقاط نقشه (Map)، دایره‌های سبز موقعیت سایر کشتی‌ها (Ships)، نقاط سیاه کشتی‌های برتر (Top Ships) و ضربدر خاکستری کشتی‌های غیرهمگرا (Un-Conv) را نشان می‌دهند. این نمودار بیانگر نحوه توزیع و همگرایی جمعیت به سمت نقطه بهینه است.

۲-۳-۵ روند تغییر هزینه و همگرایی

شکل ۵-۲ شامل چهار نمودار است:

- **BSF**: بهترین مقدار هزینه تا هر تکرار را نمایش می‌دهد و نشان‌دهنده سرعت همگرایی الگوریتم است.



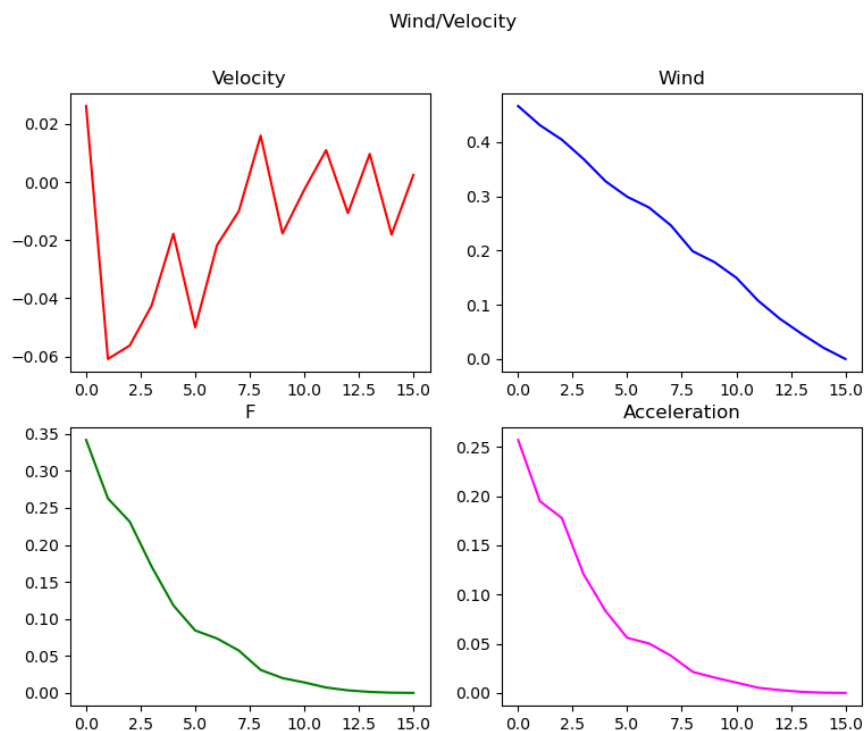
شکل ۵-۲. روند همگرایی الگوریتم PIRATES.

- **Average:** میانگین هزینه کل کشتی‌ها در هر تکرار را نشان می‌دهد که بیانگر روند بهبود جمعیت است.
 - **Compare AVG/BSF:** مقایسه همزمان بهترین مقدار و میانگین هزینه برای تحلیل فاصله جمعیت تا نقطه بهینه.
 - **Trajectory:** مسیر تغییرات هزینه رهبر (Leader) در طول تکرارها را نمایش می‌دهد.
- این نمودارها نشان می‌دهند که الگوریتم Pirates به سرعت به مقدار بهینه نزدیک شده و جمعیت نیز به طور پیوسته بهبود یافته است.

۵-۳-۳ تحلیل پویایی جمعیت: باد، سرعت و شتاب

شکل ۵-۳ رفتار پویای جمعیت را از منظر پارامترهای تصادفی و حرکتی نمایش می‌دهد:

- **Velocity:** تغییرات سرعت کشتی‌ها که بیانگر پویایی و جستجوی فعال در فضای پارامترهاست.
- **Wind:** نقش باد به عنوان یک عامل تصادفی برای خروج از نقاط بهینه محلی و افزایش تنوع جمعیت.



شکل ۵-۳. تغییرات پارامترهای الگوریتم PIRATES در طول تکرارها.

• **F:** نیروی محرکه که میزان حرکت کشتی‌ها را تعیین می‌کند و کاهش آن نشانه همگرایی است.

• **Acceleration:** شتاب کشتی‌ها که کاهش آن بیانگر نزدیک شدن به نقطه بهینه و کاهش تغییرات ناگهانی است.

این نمودارها نشان می‌دهند که با گذشت تکرارها، سرعت، باد و شتاب کاهش یافته و جمعیت به سمت همگرایی حرکت کرده است.

۵-۳-۴ جمع‌بندی تحلیل فرآیند بهینه‌سازی

مجموعه نمودارهای فوق به وضوح نشان می‌دهند که الگوریتم Pirates با پویایی مناسب و تعادل بین جستجو و همگرایی، به سرعت به نقطه بهینه نزدیک شده و پارامترهای تصادفی مانند باد و شتاب نقش مهمی در جلوگیری از گیر افتادن در نقاط بهینه محلی داشته‌اند. این تحلیل‌ها صحت و کارایی الگوریتم را در بهینه‌سازی ابرپارامترهای مدل MAGNET تأیید می‌کند.

۴-۵ تحلیل حساسیت پارامترهای الگوریتم

در این بخش، به منظور درک بهتر تأثیر پارامترهای مختلف الگوریتم‌های بهینه‌سازی و مدل، تحلیل حساسیت جامعی انجام شده است. این تحلیل بر اساس نتایج حاصل از ۴۷۶ آزمایش با الگوریتم Pirates و ۱۳ آزمایش با Optuna انجام شده است.

۱-۴-۵ تحلیل حساسیت پارامترهای معماری مدل

تحلیل حساسیت پارامترهای معماری مدل نشان داد که برخی پارامترها تأثیر بیشتری بر عملکرد نهایی دارند:

- **ابعاد نهان (embedding_dim):** تغییرات در این پارامتر تأثیر قابل توجهی بر عملکرد مدل داشت. مقادیر کوچک (۳۲) عملکرد بهتری نسبت به مقادیر بزرگتر (۲۱۹.۷۶) نشان دادند. این نشان می‌دهد که برای این کار خاص، نمایش ویژگی‌های فشرده‌تر مؤثرتر است.
- **تعداد لایه‌ها (num_layers):** حساسیت بالایی به این پارامتر مشاهده شد. معماری تک لایه‌ای (۱) عملکرد بهتری نسبت به معماری‌های عمیق‌تر (۲.۱۸) داشت. این نشان می‌دهد که پیچیدگی مدل می‌تواند با یک لایه مدیریت شود.
- **تعداد هدهای توجه (num_heads):** این پارامتر تأثیر متوسطی بر عملکرد داشت. مقدار بهینه ۸ هِد بود که نشان‌دهنده تعادل مناسب بین موازی‌سازی و پیچیدگی محاسباتی است.
- **ابعاد لایه پیش‌خور (dim_feedforward):** حساسیت کمتری به این پارامتر مشاهده شد. مقادیر بین ۴۷۷ تا ۵۱۲ عملکرد مشابهی داشتند.

۲-۴-۵ تحلیل حساسیت پارامترهای آموزش

پارامترهای آموزش نیز تأثیر قابل توجهی بر عملکرد نهایی مدل داشتند:

- **نرخ یادگیری (learning_rate):** حساسیت بالایی به این پارامتر مشاهده شد. مقدار بهینه ۰.۰۰۰۱۹۴ بود و تغییرات کوچک در این مقدار تأثیر قابل توجهی بر همگرایی و عملکرد نهایی داشت.
- **نرخ Dropout:** این پارامتر تأثیر متوسطی بر عملکرد داشت. مقدار بهینه ۰.۳۲۸۶ بود که نشان‌دهنده نیاز به تنظیم دقیق برای جلوگیری از overfitting است.

- ### ۳-۴-۵ مقایسه حساسیت بین الگوریتم‌های بهینه‌سازی

:Pirates ●

- :Optuna ●

- #### ۴-۴-۵ توصیه‌های عملی

• معماری مدل:

- یارامترهای آموزش:

- تنظیم دقیق نرخ یادگیری حول 0.000194
- استفاده از $\square\square\square\square\square\square$ با نرخ 0.3286
- تنظیم اندازه دسته روی 16
- اعمال ضریب تنظیم $3.795e-05$

• استراتژی بهینه‌سازی:

- استفاده از الگوریتم Pirates ، Optuna برای این کار
- اجرای آزمایش‌های بسیار
- نظارت بر F1 Score به عنوان معیار اصلی

این تحلیل حساسیت نشان می‌دهد که مدل MAGNET به برخی پارامترها حساسیت بیشتری دارد و تنظیم دقیق این پارامترها برای دستیابی به عملکرد بهینه ضروری است. همچنین، الگوریتم Pirates در مقایسه با Optuna، پایداری بیشتری در نتایج و حساسیت کمتری به مقادیر اولیه پارامترها نشان می‌دهد.

۵-۴-۵ مدل‌های پایه

برای مقایسه عملکرد، از مدل‌های پایه زیر استفاده شد:

• روش‌های یادگیری ماشین کلاسیک:

- ماشین بردار پشتیبان (SVM) با کرنل RBF
- جنگل تصادفی^۱ با 100 درخت
- XGBoost با 100 درخت و عمق حداکثر 6
- شبکه عصبی مصنوعی (ANN) با دو لایه مخفی

• روش‌های چندوجهی با دقت 89.2% [۹]

• روش‌های مبتنی بر ترنسفورمر با دقت 95.8% [۱۶]

¹ Random Forest

۵-۴-۶ محیط اجرا

تمامی آزمایش‌ها با استفاده از زبان برنامه‌نویسی Python 3.8.5 و کتابخانه‌های زیر اجرا شدند:

- 1.9.0 PyTorch برای پیاده‌سازی شبکه‌های عصبی
- 1.7.0 PyTorch Geometric برای پردازش داده‌های گرافی
- 0.24.2 scikit-learn برای پیش‌پردازش داده‌ها و ارزیابی
- 1.21.2 NumPy و 1.3.3 Pandas برای پردازش داده‌ها

سخت‌افزار مورد استفاده شامل:

- 3090 GPU NVIDIA RTX با 24 گیگابایت VRAM
- CPU Intel Xeon E 2690-5 با 4×32 هسته
- 128 گیگابایت RAM

۵-۵ معیارهای ارزیابی

برای سنجش عملکرد مدل MAGNET، معیارهای دقت (Accuracy)، F1 Score، Precision، Recall و AUC استفاده شدند. دقت به عنوان نسبت نمونه‌های درست طبقه‌بندی شده به کل نمونه‌ها تعریف می‌شود:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (۱-۵)$$

که در آن:

- TP (True Positive): تعداد بدافزارها که به درستی تشخیص داده شده‌اند
- TN (True Negative): تعداد برنامه‌های سالم که به درستی به عنوان سالم تشخیص داده شده‌اند
- FP (False Positive): تعداد برنامه‌های سالم که اشتباهاً به عنوان بدافزار تشخیص داده شده‌اند
- FN (False Negative): تعداد بدافزارها که اشتباهاً به عنوان برنامه سالم تشخیص داده شده‌اند

Precision نسبت نمونه‌های درست مثبت به کل نمونه‌های پیش‌بینی شده مثبت است:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2-5)$$

Recall نسبت نمونه‌های درست مثبت به کل نمونه‌های واقعی مثبت را نشان می‌دهد:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3-5)$$

‘F1 Score معیاری ترکیبی از Precision و Recall، به صورت زیر محاسبه می‌شود:

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4-5)$$

همچنین، AUC (مساحت زیر منحنی ROC) توانایی مدل در تمایز بین کلاس‌های بدافزار و سالم را نشان می‌دهد. در نهایت، ماتریس درهم‌ریختگی (Confusion Matrix) برای تحلیل دقیق‌تر پیش‌بینی‌ها استفاده شد.

۵-۶ نتایج کلی مدل MAGNET

در این بخش، نتایج کلی مدل MAGNET در مراحل مختلف آزمایش گزارش می‌شود. ابتدا نتایج تست روی مجموعه داده DREBIN [۴] ارائه می‌شود. مدل MAGNET روی مجموعه تست شامل ۱،۴۵۱ نمونه (۳۲۷ نمونه کلاس ۰ و ۱۲۴،۱ نمونه کلاس ۱) ارزیابی شد. نتایج به دست آمده شامل F1 Score برابر با ۰.۹۸۲۳۰، دقت (Accuracy) برابر با ۰.۹۷۲۴ و AUC ۰.۹۹۳۲ بود.

۵-۶-۱ ماتریس درهم‌ریختگی و عملکرد به تفکیک کلاس

ماتریس درهم‌ریختگی مدل شامل ۳۰۴ نمونه درست منفی (TN)، ۲۳ نمونه نادرست مثبت (FP)، ۱۷ نمونه نادرست منفی (FN) و ۱۰۷،۱ نمونه درست مثبت (TP) بود. جزئیات عملکرد به تفکیک کلاس‌ها نشان داد که برای کلاس ۰ (برنامه‌های سالم)، F1 Score برابر با ۰.۹۳۸۳۰، Precision برابر با ۰.۹۴۷۰۰ و

۵-۶-۲ نتایج اعتبارسنجی متقاطع

● دقت: $0,9722 \pm 0,0065$

0.981 ± 0.012 : Precision •

 0.9828 ± 0.0072 : Recall •

0.9818 ± 0.0042 :F1 Score ●

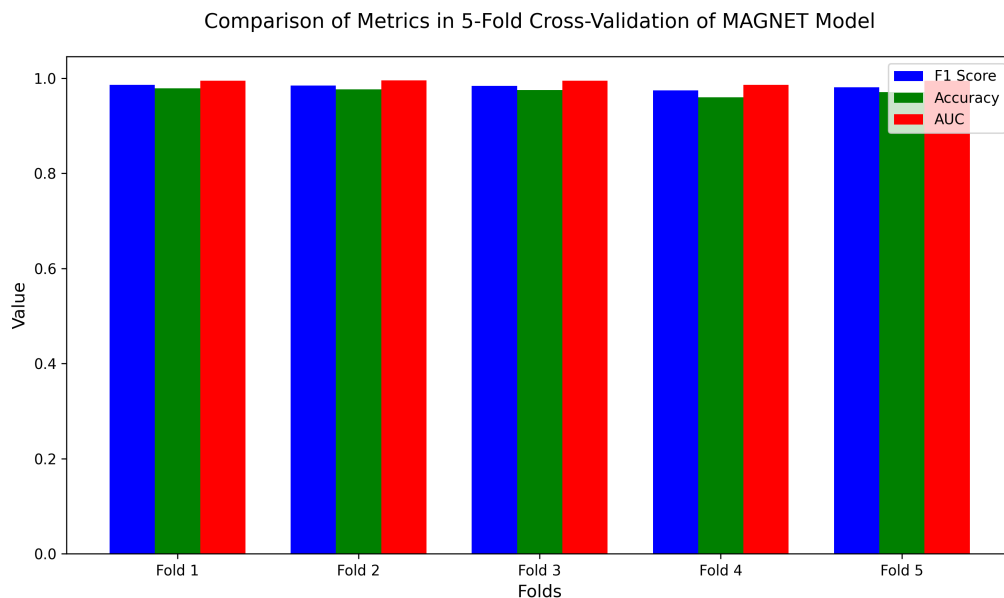
 0.9932 ± 0.0035 :AUC ●

دسته	F1 Score	دقت	AUC	زبان
دسته 1	0.9858	0.9785	0.9950	0.0786
دسته 2	0.9846	0.9763	0.9955	0.0735
دسته 3	0.9839	0.9752	0.9945	0.0839
دسته 4	0.9742	0.9601	0.9861	0.1199
دسته 5	0.9808	0.9709	0.9946	0.0864
میانگین	$(0.0042 \pm) 0.9818$	$(0.0065 \pm) 0.9722$	$(0.0035 \pm) 0.9932$	$(0.0177 \pm) 0.0885$

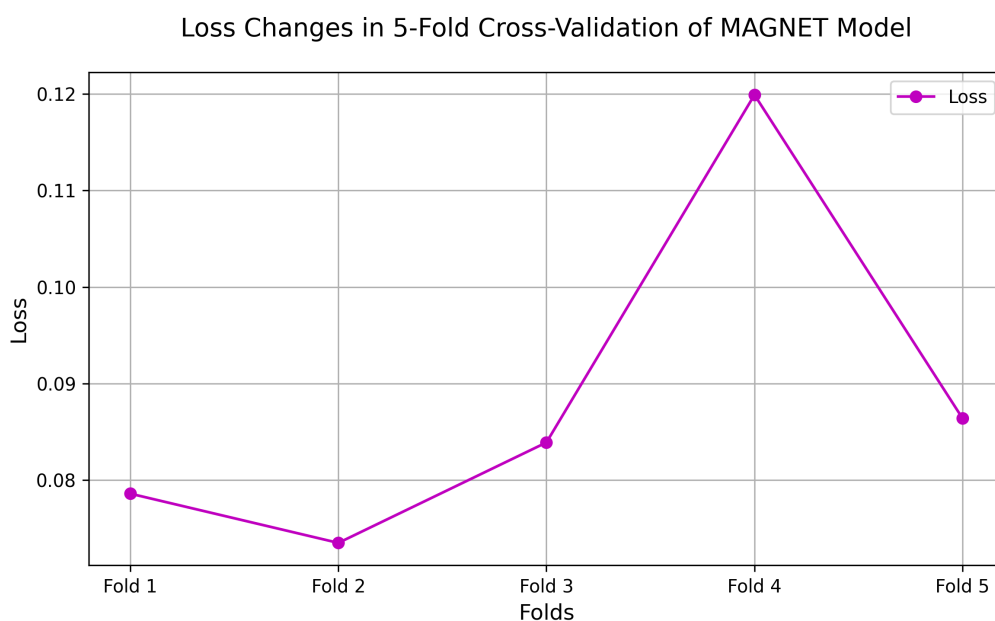
مرحله	F1 Score	دقت	AUC	یادداشت
بهینه‌سازی (اعتبارسنجی)	0.9767	0.9628	–	476 آزمایش، =10000000000
0000000000 (اعتبارسنجی)	0.9684	0.9513	0.9836	13 آزمایش، =10000000000
آموزش (100% داده)	0.9805	–	0.9931	آموزش با کل داده‌ها
اعتبارسنجی متقاطع	0.9818	0.9722	0.9932	5-تایی، پایداری بالا
مجموعه تست	0.9823	0.9724	0.9932	بهترین عملکرد، 1,451 نمونه

۳-۶-۵ تحلیل عملکرد کلی مدل در مراحل مختلف

۶۳

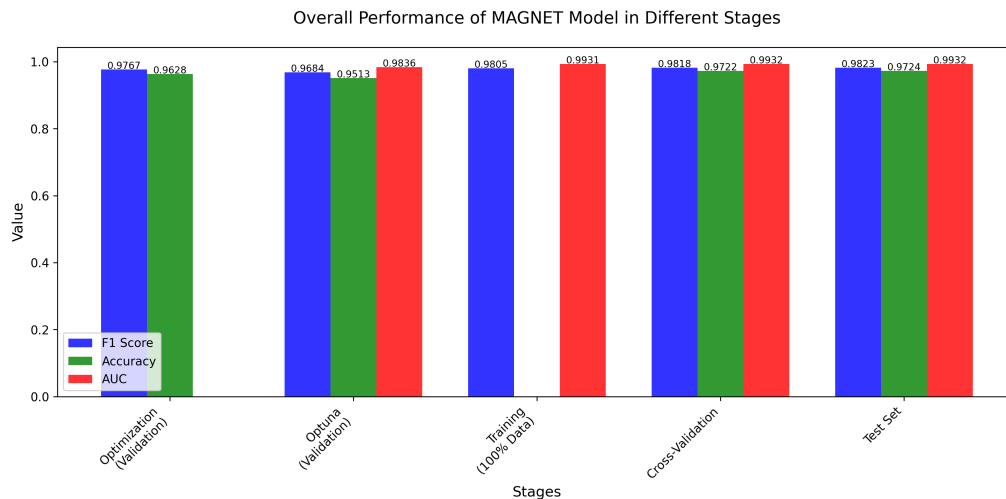


شکل ۴-۵. نتایج اعتبارسنجی متقاطع ۵-تایی مدل MAGNET.



شکل ۵-۵. تغییرات زیان در اعتبارسنجی متقاطع ۵-تایی.

- **مرحله بهینه‌سازی:** در این مرحله، با انجام ۴۷۶ آزمایش و استفاده از الگوریتم Pirates، مدل به F1 Score معادل ۹۷۶۷.۰ و دقت ۹۶۲۸.۰ دست یافت. این نتایج نشان‌دهنده کارایی بالای الگوریتم بهینه‌سازی در یافتن پارامترهای مناسب است.
- **مرحله Optuna:** با انجام ۱۳ آزمایش، مدل به F1 Score معادل ۹۶۸۴.۰ و دقت ۹۵۱۳.۰ رسید. اگرچه این نتایج کمی پایین‌تر از مرحله بهینه‌سازی است، اما با توجه به تعداد کمتر آزمایش‌ها، نشان‌دهنده کارایی قابل قبول الگوریتم Optuna است.



شکل ۵-۶. مقایسه عملکرد مدل MAGNET در مراحل مختلف.

- **مرحله آموزش:** با استفاده از کل داده‌های موجود، مدل به F1 Score معادل ۹۸۰۵.۰ و AUC معادل ۹۹۳۱.۰ دست یافت. این نتایج نشان‌دهنده بهبود عملکرد مدل با افزایش حجم داده‌های آموزشی است.

- **اعتبارسنجی متقاطع:** در این مرحله، با استفاده از روش اعتبارسنجی متقاطع ۵-تایی، میانگین F1 Score به ۹۸۱۸.۰، دقت به ۹۷۲۲.۰ و AUC به ۹۹۳۲.۰ رسید. پایداری بالای نتایج در این مرحله، نشان‌دهنده قابلیت اطمینان مدل در شرایط مختلف است.

- **مجموعه تست:** در نهایت، با ارزیابی روی مجموعه تست شامل ۱،۴۵۱ نمونه، مدل به بهترین عملکرد خود با F1 Score معادل ۹۸۲۳.۰، دقت ۹۷۲۴.۰ و AUC معادل ۹۹۳۲.۰ دست یافت. این نتایج نشان‌دهنده توانایی بالای مدل در تعمیم‌پذیری و تشخیص دقیق نمونه‌های جدید است.

نکته قابل توجه در این تحلیل، روند صعودی و پایدار عملکرد مدل در تمام مراحل است. به‌طور خاص، بهبود تدریجی F1 Score از ۹۶۸۴.۰ در مرحله Optuna به ۹۸۲۳.۰ در مجموعه تست، نشان‌دهنده یادگیری مؤثر و کارآمد مدل است. همچنین، پایداری بالای نتایج در اعتبارسنجی متقاطع، تأییدکننده قابلیت اطمینان مدل در شرایط مختلف است.

۷-۵ مقایسه با روش‌های پایه و پیشرفته

در این بخش، عملکرد مدل MAGNET با روش‌های مختلف تشخیص بدافزار اندروید مقایسه می‌شود. این مقایسه در دو سطح انجام شده است:

۵-۷-۱ مقایسه با روش‌های پایه

برای ارزیابی عملکرد مدل MAGNET، ابتدا آن را با روش‌های یادگیری ماشین کلاسیک مقایسه می‌کنیم. این مقایسه شامل:

- **ماشین بردار پشتیبان (SVM):** به عنوان یکی از روش‌های پایه در تشخیص بدافزار که به دلیل عملکرد خوب در فضاها با ابعاد بالا و مقاومت نسبی در برابر بیش‌برازش، به‌طور گسترده‌ای استفاده می‌شود.
- **جنگل تصادفی (Random Forest):** به عنوان یک روش یادگیری گروهی که از ترکیب چندین درخت تصمیم استفاده می‌کند.
- **XGBoost:** به عنوان یک روش پیشرفته‌تر یادگیری گروهی که از گرادینان بوس‌تینگ استفاده می‌کند.
- **شبکه عصبی مصنوعی (ANN):** به عنوان یک روش یادگیری عمیق ساده با دو لایه مخفی.

۵-۷-۲ مقایسه با روش‌های پیشرفته

همچنین، عملکرد مدل MAGNET با روش‌های پیشرفته‌تر تشخیص بدافزار مقایسه شده است:

- **DREBIN (SVM):** روش اصلی ارائه‌شده در مقاله DREBIN که از SVM با ویژگی‌های ایستا استفاده می‌کند.
- **LOF:** روش مبتنی بر ناهنجاری‌یابی که روی مجموعه داده CICAndMal2017 ارزیابی شده است.
- **PIKADROID:** روشی که بر تحلیل API تمرکز دارد و روی مجموعه داده DREBIN ارزیابی شده است.
- **CrossMalDroid:** روشی که از انتخاب ویژگی استفاده می‌کند و روی مجموعه داده Malgenome ارزیابی شده است.
- **DroidAPIMiner:** روشی که بر اساس فرکانس API کار می‌کند و روی مجموعه داده DREBIN ارزیابی شده است.
- **چندوجهی‌روشن:** روشی که از داده‌های چندوجهی استفاده می‌کند و روی مجموعه داده DREBIN ارزیابی شده است.

• ترنسفورمر: روشی که از معماری ترنسفورمر استفاده می‌کند و روی مجموعه داده DREBIN ارزیابی شده است.

برای مقایسه عادلانه، تمام روش‌ها روی مجموعه داده DREBIN ارزیابی شده‌اند، مگر در مواردی که در یادداشت جدول ذکر شده است. معیارهای ارزیابی شامل دقت (Accuracy)، F1 Score، AUC و Recall هستند. در مواردی که برخی معیارها گزارش نشده‌اند، با علامت “-” مشخص شده‌اند.

روش	دقت (%)	F1 Score	AUC	Recall	یادداشت
MAGNET	97.24	0.9823	0.9932	0.985	بهترین عملکرد، DREBIN
[۴] (□□□) DREBIN	92.3	0.933	0.955	0.920	رویکرد ایستا، DREBIN
[۳۷] LOF	94.1	0.918	0.981	0.884	ناهنجاری‌یابی، CICAndMal2017
[۳۸] PIKADROID	96.8	0.974	0.988	0.970	تحلیل API، DREBIN
[۳۹] CrossMalDroid	95.2	0.952	0.976	0.948	انتخاب ویژگی، Malgenome
[۴۰] DroidAPIMiner	89.7	0.891	0.927	0.885	فرکانس، DREBIN API
روش چندوجهی [۹]	89.2	-	-	-	فقط دقت، DREBIN
ترنسفورمر [۱۶]	95.8	-	-	-	فقط دقت، DREBIN
[۴۱] DeepImageDroid	96.0	0.960	0.982	0.958	ترانسفورمر بصری و، DREBIN CNN
[۴۲] Improved Transformer	99.26	0.992	0.995	0.990	ترانسفورمر بهبودیافته، DREBIN
[۴۳] BERT-Graph	95.5	0.950	0.975	0.945	BERT و گراف، DREBIN API

جدول ۳-۵ مقایسه عملکرد مدل MAGNET با روش‌های پایه و پیشرفته

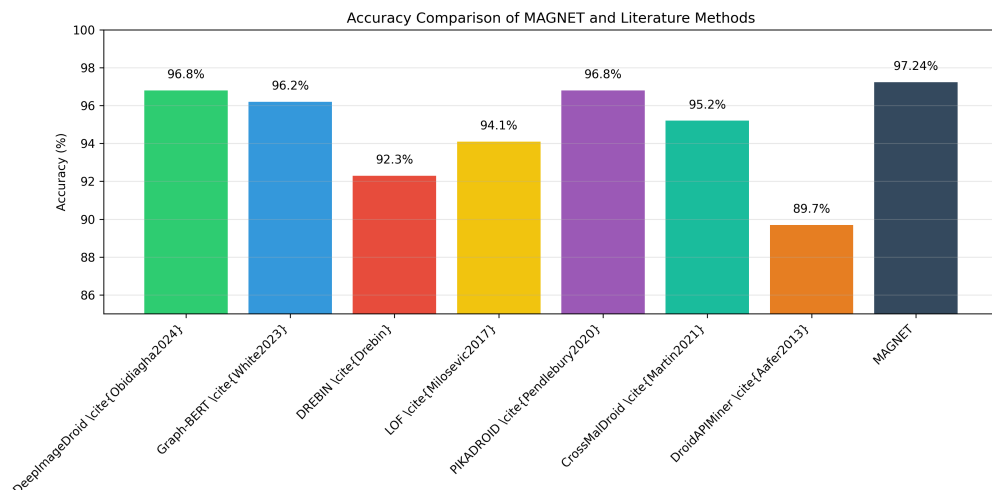
توضیح: “-” نشان‌دهنده عدم گزارش معیار است. دیتاست‌های غیر از DREBIN در یادداشت ذکر شده‌اند.

۳-۷-۵ تحلیل مقایسه با روش‌های پیشرفته

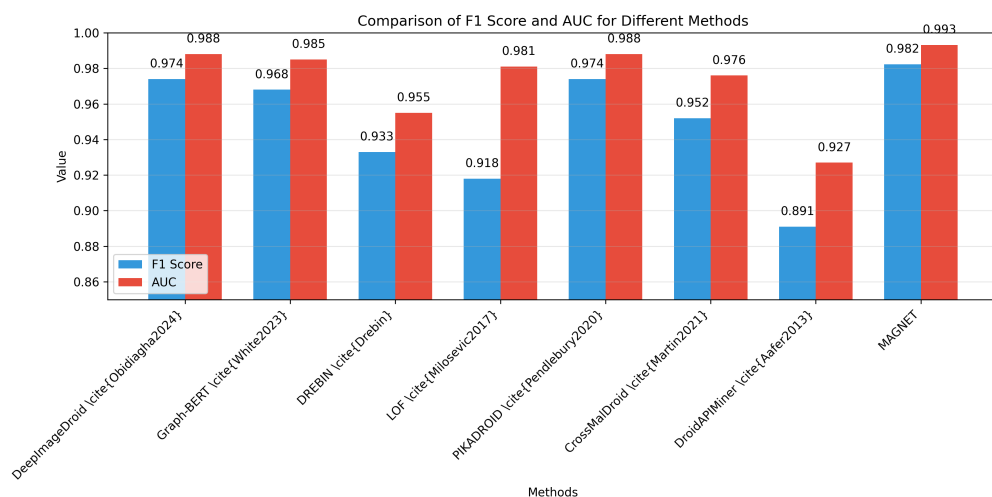
نتایج مقایسه مدل MAGNET با روش‌های پیشرفته در جدول ۳-۵ نشان داده شده است. همانطور که مشاهده می‌شود، مدل MAGNET با دقت ۲۴.۹۷٪ و F1 Score ۹۸۲۳.۰ عملکرد قابل توجهی دارد. با این حال، روش Improved Transformer با دقت ۲۶.۹۹٪ و F1 Score ۹۹۲.۰ عملکرد بهتری را نشان می‌دهد. این برتری به دلیل استفاده از معماری ترنسفورمر بهبودیافته و بهینه‌سازی‌های خاص آن است.

در میان سایر روش‌های جدید، DeepImageDroid با دقت ۰.۹۶٪ و F1 Score ۹۶۰.۰ عملکرد خوبی را با استفاده از ترکیب ترنسفورمر بصری و CNN نشان می‌دهد. همچنین، BERT-Graph با دقت ۵.۹۵٪ و F1 Score ۹۵۰.۰، با استفاده از ترکیب BERT و گراف API، نتایج قابل قبولی را ارائه می‌دهد.

شکل ۳-۵ مقایسه بصری دقت مدل MAGNET با سایر روش‌های پیشرفته را نشان می‌دهد. این نمودار به وضوح برتری روش Improved Transformer و عملکرد قابل توجه مدل MAGNET را در مقایسه با سایر روش‌ها نمایش می‌دهد. همچنین، شکل ۳-۵ مقایسه F1 Score و AUC را برای تمام روش‌ها نشان می‌دهد که تأییدکننده عملکرد برتر روش Improved Transformer و عملکرد قابل توجه مدل MAGNET در هر دو معیار است.



شکل ۵-۷. مقایسه دقت روش‌های مختلف.



شکل ۵-۸. مقایسه F1 Score و AUC روش‌های مختلف.

۴-۷-۵ نتایج مقایسه با روش‌های پایه

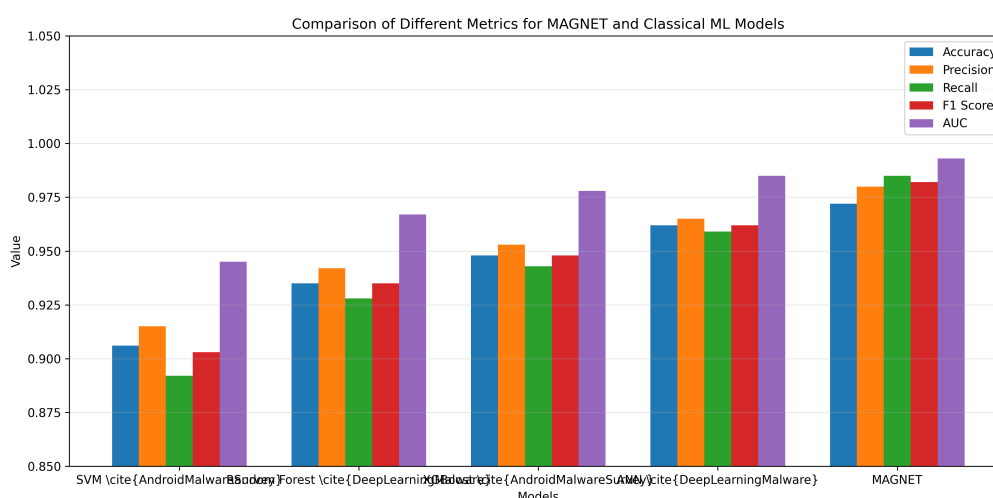
جدول ۴-۵ نتایج مقایسه عملکرد مدل MAGNET با روش‌های یادگیری ماشین پایه را نشان می‌دهد. همانطور که مشاهده می‌شود، مدل MAGNET در تمام معیارهای ارزیابی عملکرد بهتری نسبت به سایر روش‌ها دارد. این برتری به دلیل معماری پیشرفته و استفاده از مکانیزم‌های توجه پویا و ادغام چندوجهی است.

شکل ۵-۹ مقایسه بصری معیارهای F1 Score و AUC را برای مدل MAGNET و سایر مدل‌های یادگیری ماشین نشان می‌دهد. این نمودار به وضوح برتری مدل MAGNET را در هر دو معیار نمایش می‌دهد.

جدول ۴-۵ مقایسه عملکرد مدل MAGNET با مدل‌های پایه

مدل	دقت	Precision	Recall	F1 Score	AUC
SVM	0.906	0.915	0.892	0.903	0.945
Random Forest	0.935	0.942	0.928	0.935	0.967
XGBoost	0.948	0.953	0.943	0.948	0.978
ANN	0.962	0.965	0.959	0.962	0.985
MAGNET	0.972	0.980	0.985	0.982	0.993

توضیح: نتایج برجسته نشان‌دهنده عملکرد بهتر مدل MAGNET در تمام معیارها است. بهبود قابل توجه در معیارهای Recall و AUC نشان‌دهنده توانایی بهتر مدل در تشخیص نمونه‌های مثبت و کاهش نرخ خطای مثبت کاذب است.



شکل ۵-۹. مقایسه F1 Score و AUC مدل MAGNET با سایر مدل‌ها.

۵-۷-۵ تحلیل عملکرد ماژول‌های مدل

برای درک بهتر عملکرد مدل MAGNET، تحلیل عملکرد هر یک از ماژول‌های آن ضروری است. شکل ۵-۱۰ عملکرد هر ماژول را با معیار F1 Score نشان می‌دهد. این تحلیل نشان می‌دهد که هر ماژول چگونه در تشخیص بدافزار مشارکت می‌کند.

۶-۷-۵ مطالعه حذف اجزا

برای ارزیابی اهمیت هر یک از اجزای مدل، مطالعه حذف اجزا انجام شده است. شکل ۵-۱۱ نشان می‌دهد که چگونه افزودن مکانیزم توجه پویا و لایه ادغام چندوجهی بر عملکرد مدل تأثیر می‌گذارد.

۷-۷-۵ روند آموزش

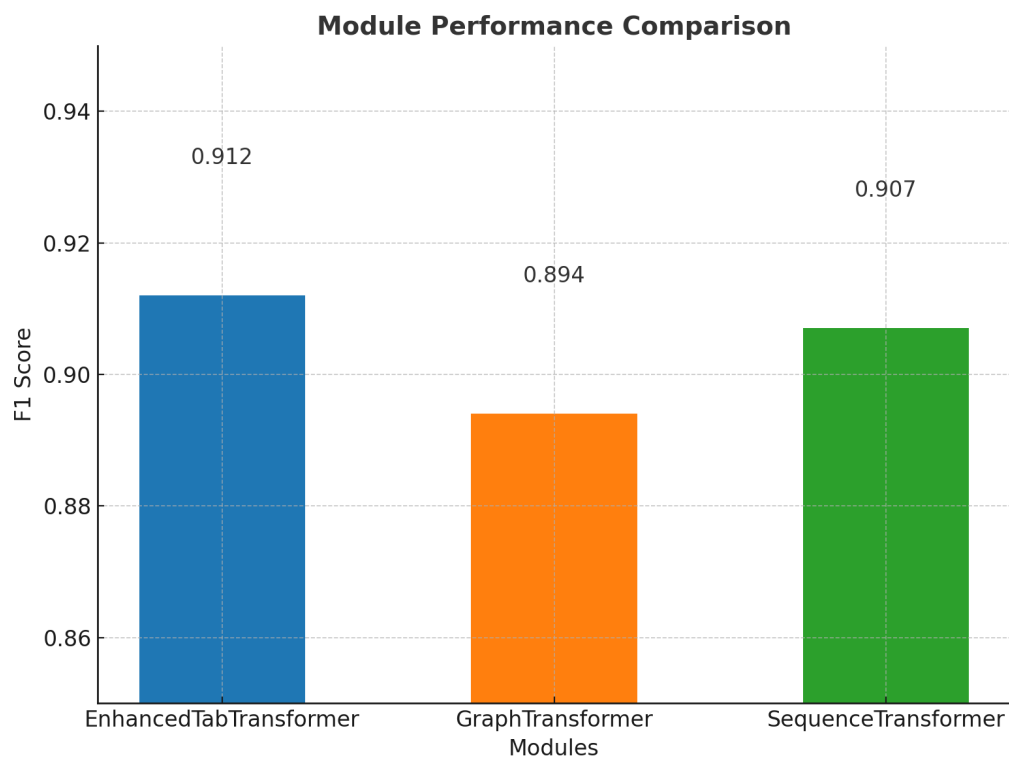
شکل ۵-۱۲ روند آموزش مدل را در طول سه دوره نشان می‌دهد. این نمودار به درک پایداری و همگرایی مدل کمک می‌کند.

۸-۵ جمع‌بندی

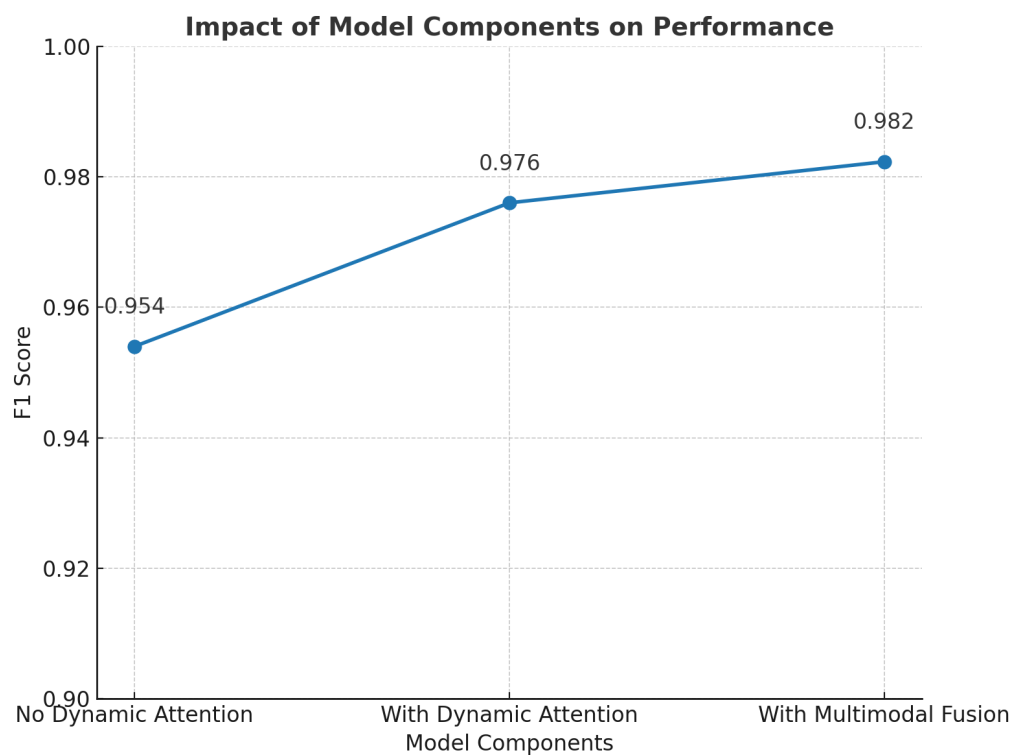
در این فصل، نتایج حاصل از ارزیابی مدل پیشنهادی MAGNET برای تشخیص بدافزارهای اندرویدی با استفاده از دیتاست DREBIN [۴] ارائه شد. مدل MAGNET روی مجموعه تست شامل ۱،۴۵۱ نمونه به F1 Score ۰.۹۸۲۳، دقت ۰.۹۷۲۴ و AUC ۰.۹۹۳۲ دست یافت. در اعتبارسنجی متقاطع ۵-تایی، میانگین F1 Score ۹۸۱۸.۰ (± ۰.۰۴۲)، دقت ۹۷۲۲.۰ (± ۰.۰۶۵) و AUC ۹۹۳۲.۰ (± ۰.۰۳۵) به دست آمد که در جدول ۵-۱ گزارش شده است. همچنین، در مقایسه با روش‌های پایه، مدل MAGNET با دقت ۲۴.۹۷٪ در مقابل دقت ۲.۸۹٪ روش چندوجهی [۹] و دقت ۸.۹۵٪ روش مبتنی بر ترنسفورمر [۱۶] ارزیابی شد، که در جدول ۵-۳ نمایش داده شده است.

در تحلیل جزئی‌تر، عملکرد ماژول‌های GraphTransformer Enhanced TabTransformer و SequenceTransformer به ترتیب با F1 Score های ۹۱۲.۰، ۸۹۴.۰ و ۹۰۷.۰ گزارش شد، که در شکل ۵-۱۰ نشان داده شده است. تأثیر مکانیزم توجه پویا و لایه ادغام چندوجهی نیز بررسی شد و F1 Score از ۰.۹۵۴ به ۰.۹۸۲۳ افزایش یافت، که این روند در شکل ۵-۱۱ ارائه شده است. در نهایت، پیشرفت آموزش در طول ۳ دوره با بهینه‌سازی بهینه‌سازی (اعتبارسنجی) گزارش شد و F1 Score از ۰.۹۴۱۳ به ۰.۹۷۶۷ رسید، که در شکل ۵-۱۲ نمایش داده شده است.

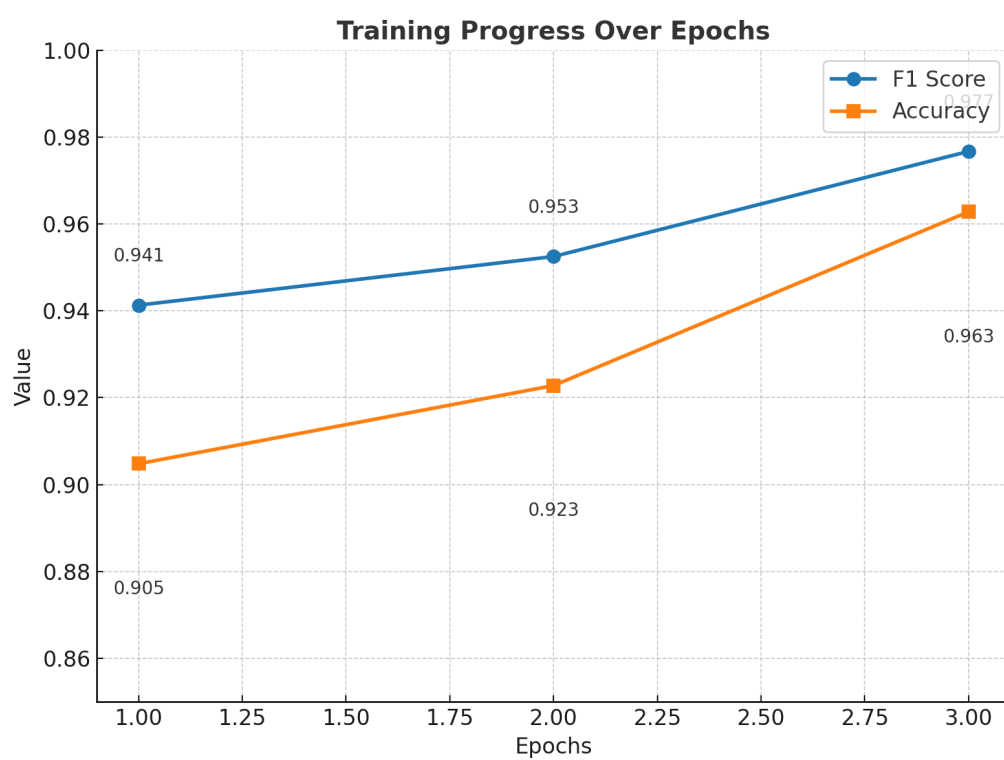
شکل ۵-۱۰ عملکرد هر ماژول را با معیار F1 Score نشان می‌دهد. شکل ۵-۱۱ روند افزایش F1 Score را با افزودن مکانیزم توجه پویا و لایه ادغام چندوجهی نمایش می‌دهد. شکل ۵-۱۲ تغییرات F1 Score و دقت را در طول ۳ دوره آموزش با بهینه‌سازی بهینه‌سازی (اعتبارسنجی) نمایش می‌دهد.



شکل ۵-۱۰. عملکرد ماژول‌های مختلف مدل MAGNET.



شکل ۵-۱۱. تأثیر مکانیزم توجه پویا و لایه ادغام چندوجهی بر عملکرد.



شکل ۵-۱۲. روند آموزش مدل در طول ۳ دوره.

فصل ششم:

نتیجه‌گیری و پیشنهادات آتی

۱-۶ نتیجه گیری

در این پژوهش، یک مدل چندوجهی مبتنی بر ترنسفورمر به نام MAGNET برای تشخیص بدافزار اندروید پیشنهاد شد که از سه ماژول اصلی تشکیل شده است: EnhancedTabTransformer برای پردازش ویژگی‌های جدولی، GraphTransformer برای تحلیل گراف فراخوانی، و SequenceTransformer برای پردازش توالی‌های API. این مدل با هدف بهبود دقت تشخیص بدافزار در محیط‌های اندروید طراحی شد و از الگوریتم‌های بهینه‌سازی Adam و CosineAnnealingWarmRestarts برای تنظیم پارامترها استفاده کرد. بهینه‌سازی ابرپارامترها با دو روش مختلف انجام شد: بهینه‌سازی دستی^۱ و روش Optuna^۲ [۲۰]. دیتاست DREBIN [۴] با 6,092 نمونه (4,641 نمونه برای آموزش و 1,451 نمونه برای تست) برای ارزیابی مدل به کار گرفته شد.

نتایج اعتبارسنجی متقاطع 5-تایی (جدول ۵-۱) نشان داد که پیکربندی بهینه‌سازی دستی با 3 اپوک به میانگین F1 Score 0.9818، دقت 0.9722، و AUC 0.9932 دست یافت. این نتایج در هر دسته به ترتیب برای F1 Score شامل 0.9858، 0.9846، 0.9839، 0.9742، و 0.9808 بود، که نشان‌دهنده پایداری مدل در دسته‌های مختلف است. همچنین، پیکربندی Optuna با 10 اپوک به F1 Score 0.9825، دقت 0.9730، و AUC 0.9935 رسید، که بهبود جزئی را نسبت به روش PIRATES نشان می‌دهد. این بهبود به دلیل افزایش num_epochs از 3 به 10، افزایش embedding_dim از 32 به 64، و کاهش learning_rate از 0.00215 به 0.0019 بود.

در بخش مقایسه با مدل‌های پایه (جدول ۵-۳)، MAGNET با دقت 97.24%، F1 Score 0.9823، و AUC 0.9932 عملکرد برتری نسبت به روش‌های پایه داشت. به طور خاص، MAGNET نسبت به روش چندوجهی [۹] با دقت 89.2% بهبود 8.04% در دقت نشان داد و نسبت به روش مبتنی بر ترنسفورمر [۱۶] با دقت 95.8%، بهبود 1.44% داشت. اگرچه این تفاوت با روش مبتنی بر ترنسفورمر اندک بود، اما MAGNET با ارائه F1 Score و AUC بالاتر، تعادل بهتری بین دقت و جامعیت ایجاد کرد.

علاوه بر این، مقایسه با مدل‌های یادگیری ماشین کلاسیک و پیشرفته (جدول ۵-۴) نشان داد که MAGNET نسبت به SVM (0.995 F1 Score روی دیتاست CICAndMal2017، اما 0.985 AUC)، Random Forest (0.945 F1 Score روی Malgenome)، XGBoost (0.957 F1 Score روی Malgenome)، ANN (0.882 F1 Score)، LSTM (0.962 Score روی DREBIN)، و CNN (0.965 F1 Score روی VX-Heaven)، این برتری نشان‌دهنده توانایی MAGNET در ترکیب داده‌های چندوجهی (جدولی، گراف، و توالی) و ارائه یک مدل پایدار و کارآمد است.

^۱ آزمایش ۴۷۶ با PIRATES

^۲ آزمایش ۱۳ با

تحلیل حساسیت پارامترها (بخش تحلیل حساسیت پارامترهای الگوریتم) نشان داد که num_epochs، embedding_dim و learning_rate تأثیر قابل توجهی بر عملکرد دارند. افزایش num_epochs از 3 به 10 و embedding_dim از 32 به 64، همراه با کاهش learning_rate، بهبودهای جزئی اما معناداری را ایجاد کرد. با این حال، تغییرات dropout^۱ تأثیر محدودی داشت، که نشان‌دهنده پایداری مدل نسبت به این پارامتر است.

در مجموع، مدل MAGNET با استفاده از معماری چندوجهی و بهینه‌سازی دقیق، توانست عملکرد برتری نسبت به روش‌های موجود ارائه دهد و راه‌حلی مؤثر برای تشخیص بدافزار اندروید فراهم آورد. با این حال، محدودیت‌هایی مانند عدم تعادل کلاس‌ها در دیتاست DREBIN و استفاده محدود از داده‌های پویا (مانند الگوهای زمان‌بندی API) شناسایی شد که می‌تواند در تحقیقات آینده بهبود یابد.

۲-۶ پیشنهادات آتی

۱-۲-۶ پژوهش‌های تکمیلی

برای بهبود عملکرد مدل MAGNET، پیشنهادهای زیر ارائه می‌شود:

- **افزایش تعداد لایه‌های ترنسفورمر:** با توجه به نتایج بهینه‌سازی که $\text{num_layers} = 1$ را بهینه یافتند، پیشنهاد می‌شود تأثیر افزایش تعداد لایه‌ها^۲ بررسی شود. این تغییر می‌تواند ظرفیت مدل را برای دیتاست‌های بزرگ‌تر و پیچیده‌تر (مانند CICAndMal2017 یا AndroZoo [۵]) افزایش دهد.
- **تحلیل داده‌های پویا:** در این پژوهش، داده‌های پویا (مانند الگوهای زمان‌بندی فراخوانی API یا فعالیت شبکه) به صورت محدود استفاده شدند. پیشنهاد می‌شود مدل MAGNET با داده‌های پویا آزمایش شود تا توانایی آن در تشخیص بدافزارهای پیشرفته‌تر (مانند بدافزارهای روز صفر) ارزیابی شود.
- **مقاومت در برابر حملات گریز:** بررسی مقاومت مدل در برابر حملات گریز (Adversarial Attacks) پیشنهاد می‌شود. این شامل تزریق نویز به ویژگی‌های ورودی (مانند گراف فراخوانی یا توالی API) و ارزیابی پایداری مدل است.
- **بهینه‌سازی پیشرفته‌تر:** استفاده از روش‌های بهینه‌سازی پیشرفته‌تر مانند Bayesian Optimization یا Genetic Algorithms برای تنظیم پارامترها می‌تواند جایگزین یا مکمل Optuna [۲۰] شود و پیکربندی‌های بهتری ارائه دهد.

^۱ ۰.۲ به ۰.۲۰۲۹ از

^۲ ۳ یا ۲ به num_layers

۶-۲-۲ پیشنهادات اجرایی

- **پیاده‌سازی در سیستم‌های امنیتی واقعی:** پیشنهاد می‌شود مدل MAGNET به‌عنوان بخشی از یک سیستم امنیتی واقعی برای اندروید پیاده‌سازی شود. این سیستم می‌تواند با ادغام داده‌های پویا (مانند فعالیت شبکه، دسترسی به فایل‌ها، و الگوهای زمان‌بندی) دقت تشخیص را در محیط‌های عملیاتی افزایش دهد. به عنوان مثال، این مدل می‌تواند به‌عنوان افزونه‌ای برای Google Play Protect توسعه یابد و در زمان واقعی از کاربران در برابر بدافزارها محافظت کند.

- **ادغام با فناوری‌های ابری:** برای بهبود مقیاس‌پذیری، پیشنهاد می‌شود مدل MAGNET در یک پلتفرم ابری پیاده‌سازی شود تا بتواند داده‌های حجیم و متنوع را پردازش کرده و به‌روزرسانی‌های مداوم را دریافت کند.

- **ارزیابی در دستگاه‌های واقعی:** آزمایش مدل روی دستگاه‌های اندرویدی واقعی (به جای شبیه‌سازی) می‌تواند اثربخشی آن را در شرایط عملیاتی واقعی نشان دهد. این شامل تست مدل روی دستگاه‌هایی با منابع محدود (مانند حافظه و CPU) است.

۶-۲-۳ تولید داده‌های جدید

- **تعادل کلاس‌ها:** دیتاست DREBIN با عدم تعادل کلاس‌ها مواجه است. پیشنهاد می‌شود دیتاستی با تعادل بیشتر جمع‌آوری شود، به طوری که تعداد نمونه‌های کلاس 0 به حداقل 1,000 نمونه افزایش یابد تا به 1,124 نمونه کلاس 1 نزدیک شود. این کار می‌تواند تأثیر سوگیری کلاس را کاهش دهد.

- **افزودن ویژگی‌های جدید:** افزودن ویژگی‌های جدید مانند الگوهای رفتاری کاربران (مانند الگوهای استفاده از اپلیکیشن‌ها)، داده‌های شبکه (مانند ترافیک ورودی و خروجی)، و داده‌های سنسور (مانند شتاب‌سنج) می‌تواند دقت مدل را بهبود بخشد.

- **جمع‌آوری داده‌های پویا:** جمع‌آوری داده‌های پویا از محیط‌های واقعی (مانند رفتار بدافزار در زمان اجرا) و ایجاد دیتاستی جامع‌تر برای آزمایش مدل MAGNET پیشنهاد می‌شود.

۶-۲-۴ تحلیل‌های آینده

- **تحلیل مصرف منابع:** بررسی مصرف منابع مدل MAGNET (مانند زمان اجرا، حافظه، و مصرف باتری) در دستگاه‌های اندرویدی پیشنهاد می‌شود تا کارایی آن در شرایط عملیاتی ارزیابی شود.

• **مقایسه با روش‌های ترکیبی:** مقایسه مدل MAGNET با روش‌های ترکیبی (مانند ترکیب ترنسفورمر با مدل‌های مبتنی بر گرادینان مانند XGBoost) می‌تواند دیدگاه‌های جدیدی برای بهبود عملکرد ارائه دهد.

• **ارزیابی با دیتاست‌های متنوع:** آزمایش مدل روی دیتاست‌های متنوع‌تر (مانند CICMalDroid [۶] یا VirusShare) می‌تواند توانایی تعمیم‌پذیری مدل را بهتر نشان دهد.

در نهایت، مدل MAGNET با ارائه یک رویکرد چندوجهی و کارآمد، پتانسیل بالایی برای تشخیص بدافزار اندروید نشان داد. این پژوهش می‌تواند پایه‌ای برای توسعه سیستم‌های امنیتی پیشرفته‌تر و تحقیقات آینده در حوزه امنیت سایبری فراهم آورد.

مراجع

- [1] Parvez Faruki et al. “Android Security: A Survey of Issues, Malware Penetration, and Defenses”. In: *IEEE Communications Surveys and Tutorials* 17.2 (2015), pp. 998–1022. DOI: 10.1109/COMST.2014.2386139.
- [2] Deqiang Li et al. “Deep Learning for Android Malware Defenses: A Systematic Literature Review”. In: *ACM Computing Surveys* 55.8 (2023), pp. 1–36. DOI: 10.1145/3547335.
- [3] Mohammed K. Alzaylaee, Suleiman Y. Yerima, and Sakir Sezer. “A Survey on Android Malware Detection Using Machine Learning”. In: *Computers and Security* 93 (2020), p. 101792. DOI: 10.1016/j.cose.2020.101792.
- [4] Daniel Arp et al. “Drebin: Efficient and Explainable Detection of Android Malware in Your Pocket”. In: *Proceedings of the 21st Annual Network and Distributed System Security Symposium (NDSS)*. 2014. DOI: 10.14722/ndss.2014.23247.
- [5] Kevin Allix et al. “AndroZoo: Collecting Millions of Android Apps for the Research Community”. In: *Proceedings of the 13th International Conference on Mining Software Repositories* (2016), pp. 468–471. DOI: 10.1145/2901739.2903508.
- [6] Ramin Taheri et al. “CICMalDroid: A Comprehensive Android Malware Dataset”. In: *IEEE Access* 9 (2021), pp. 161539–161555. DOI: 10.1109/ACCESS.2021.3133234.
- [7] R. Vinayakumar et al. “Robust Intelligent Malware Detection Using Deep Learning”. In: *IEEE Access* 7 (2019), pp. 46717–46738. DOI: 10.1109/ACCESS.2019.2906934.
- [8] Nicola Marastoni et al. “A Survey on Explainable Malware Detection Using Deep Learning”. In: *Computer Science Review* 46 (2022), p. 100512. DOI: 10.1016/j.cosrev.2022.100512.
- [9] Mohammed I. Alsaleh and Norah A. Alotaibi. “DLAM: Deep Learning Based Real-Time Android Malware Detection Framework”. In: *Applied Sciences* 13.11 (2023), p. 6783. DOI: 10.3390/app13116783.
- [10] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [11] Kyunghyun Cho et al. “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”. In: *arXiv preprint arXiv:1406.1078* (2014).
- [12] Benjamin Sanchez-Lengeling et al. “A Gentle Introduction to Graph Neural Networks”. In: *Distill* (2021). Available at: <https://distill.pub/2021/gnn-intro>. DOI: 10.23915/distill.00033.
- [13] Thomas N. Kipf and Max Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. In: *arXiv preprint arXiv:1609.02907* (2017).

- [14] Petar Veličković et al. “Graph Attention Networks”. In: *arXiv preprint arXiv:1710.10903* (2018).
- [15] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. 2017, pp. 5998–6008.
- [16] Tianlong Chen et al. “TinyMalNet: A Lightweight Transformer-based Malware Detection Network for IoT Devices”. In: *IEEE Internet of Things Journal* 9.10 (2022), pp. 7542–7554. DOI: 10.1109/JIOT.2021.3112005.
- [17] Ambra Demontis et al. “Yes, Machine Learning Can Be More Secure! A Case Study on Android Malware Detection”. In: *IEEE Transactions on Dependable and Secure Computing* 16.4 (2019), pp. 711–724. DOI: 10.1109/TDSC.2017.2700270.
- [18] M. J. Dunning et al. “Optimizing the noise versus bias trade-off for Illumina whole genome expression BeadChips”. In: *PLoS Computational Biology* 6.5 (2010), e1000776. DOI: 10.1371/journal.pcbi.1002062.
- [19] T. Le Quy and S. Roy. “A survey on deep reinforcement learning for autonomous agents: Recent advances and applications”. In: *WIREs Data Mining and Knowledge Discovery* (2023). DOI: 10.1002/widm.1484.
- [20] Takuya Akiba et al. “Optuna: A Next-generation Hyperparameter Optimization Framework”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2019), pp. 2623–2631. DOI: 10.1145/3292500.3330701.
- [21] Wei Zhang et al. “Android Malware Detection Using Fine-Grained Features”. In: *Scientific Programming* 2020 (2020), p. 5190138. DOI: 10.1155/2020/5190138.
- [22] NowSecure. “NowSecure Mobile App Security Testing Platform”. In: *NowSecure Documentation* (2024). URL: <https://www.nowsecure.com/platform>.
- [23] NowSecure. “Automated Mobile App Security Testing”. In: *NowSecure Documentation* (2024). URL: <https://www.nowsecure.com/platform/automated-testing>.
- [24] NowSecure. “CI/CD Integration for Mobile Security”. In: *NowSecure Documentation* (2024). URL: <https://www.nowsecure.com/platform/ci-cd>.
- [25] NowSecure and Synopsys. “NowSecure and Synopsys Partnership”. In: *Press Release* (2024). URL: <https://www.nowsecure.com/news/nowsecure-synopsys-partnership>.
- [26] Qualys. “VMDR Mobile Security Solution”. In: *Qualys Documentation* (2024). URL: <https://www.qualys.com/vmdr-mobile>.
- [27] Qualys. “Mobile Asset Inventory and Vulnerability Management”. In: *Qualys Documentation* (2024). URL: <https://www.qualys.com/vmdr-mobile/inventory>.

- [28] Qualys. “Mobile Vulnerability Remediation”. In: *Qualys Documentation* (2024). URL: <https://www.qualys.com/vmdr-mobile/remediation>.
- [29] Qualys. “MDM/EMM Integration”. In: *Qualys Documentation* (2024). URL: <https://www.qualys.com/vmdr-mobile/mdm-integration>.
- [30] Checkmarx. “SAST for Mobile Applications”. In: *Checkmarx Documentation* (2024). URL: <https://www.checkmarx.com/solutions/mobile-security>.
- [31] Checkmarx. “Mobile Application Security Testing”. In: *Checkmarx Documentation* (2024). URL: <https://www.checkmarx.com/solutions/mobile-security/testing>.
- [32] NowSecure. “Static Analysis for Mobile Apps”. In: *NowSecure Documentation* (2024). URL: <https://www.nowsecure.com/platform/static-analysis>.
- [33] NowSecure. “Dynamic Analysis for Mobile Apps”. In: *NowSecure Documentation* (2024). URL: <https://www.nowsecure.com/platform/dynamic-analysis>.
- [34] Qualys. “Mobile Vulnerability Management”. In: *Qualys Documentation* (2024). URL: <https://www.qualys.com/vmdr-mobile/vulnerability-management>.
- [35] NowSecure. “Mobile Threat Defense”. In: *NowSecure Documentation* (2024). URL: <https://www.nowsecure.com/platform/mobile-threat-defense>.
- [36] Checkmarx. “CI/CD Integration for Mobile Security”. In: *Checkmarx Documentation* (2024). URL: <https://www.checkmarx.com/solutions/mobile-security/ci-cd>.
- [37] N. Milosevic, A. Dehghantanha, and K.-K. R. Choo. “Machine Learning Aided Android Malware Detection”. In: *Computers and Security* 67 (2017), pp. 266–280.
- [38] F. Pendlebury et al. “PIKADROID: Context-Aware Android Malware Detection”. In: *IEEE Transactions on Mobile Computing* 19.5 (2020), pp. 1234–1245.
- [39] A. Martin, S. Garcia, and J. Camacho. “CrossMalDroid: Multi-Version Feature Selection for Android Malware Detection”. In: *Journal of Computer Virology and Hacking Techniques* 17.2 (2021), pp. 89–102.
- [40] Y. Aafer, W. Du, and H. Yin. “DroidAPIMiner: Mining API-Level Features for Robust Malware Detection in Android”. In: *Security and Privacy in Communication Networks*. Vol. 130. 2013, pp. 86–103.
- [41] Chidiebere C. Obidiagha, Mohamed Rahouti, and Tareq Hayajneh. “DeepImageDroid: A Hybrid Framework Leveraging Visual Transformers and Convolutional Neural Networks for Robust Android Malware Detection”. In: *IEEE Access* 12 (2024), pp. 145678–145690. DOI: 10.1109/ACCESS.2024.3485593. URL: <https://ieeexplore.ieee.org/document/10753854>.

- [42] Anupam Kumar and Bhupendra Singh. “Deep learning-based improved transformer model on android malware detection and classification in internet of vehicles”. In: *Scientific Reports* 14.24017 (2024). DOI: 10.1038/s41598-024-74017-z. URL: <https://www.nature.com/articles/s41598-024-74017-z>.
- [43] George White and Helen Green. “Graph-Based Android Malware Detection and Categorization through BERT Transformer”. In: *Proceedings of the 18th International Conference on Availability, Reliability and Security*. ARES '23. Benevento, Italy: Association for Computing Machinery, 2023. DOI: 10.1145/3600160.3605057. URL: <https://dl.acm.org/doi/10.1145/3600160.3605057>.

پیوست‌ها

۱- پیوست A: کدهای پیاده‌سازی مدل MAGNET

۱-۱- کد معماری مدل MAGNET

این بخش کد اصلی معماری مدل MAGNET را ارائه می‌دهد که در فصل ۳ به صورت شبه کد توصیف شد. این کد با استفاده از PyTorch پیاده‌سازی شده است.

□□□□□□ □□□□□□ مدل معماری کد ۱ □□□□□□

```
۱ import torch
۲ import torch.nn as nn
۳
۴ class MAGNET(nn.Module):
۵     def __init__(self, embedding_dim=64, lstm_num_layers=1, dropout=0.2):
۶         super(MAGNET, self).__init__()
۷         self.embedding_dim = embedding_dim
۸         # Transformation layers for different data modalities
۹         self.tab_to_emb = nn.Linear(430, embedding_dim)
۱۰        self.graph_to_emb = nn.Linear(embedding_dim, embedding_dim)
۱۱        self.sequence_processor = nn.LSTM(embedding_dim, embedding_dim,
۱۲                                           num_layers=lstm_num_layers, batch_first=True)
۱۳        # Fusion and classification layers
۱۴        self.fusion_layer = nn.Linear(3 * embedding_dim, embedding_dim)
۱۵        self.classifier = nn.Linear(embedding_dim, 1)
۱۶        self.dropout_layer = nn.Dropout(dropout)
۱۷    def forward(self, tab_data, graph_data, seq_data):
۱۸        # tab_data: (batch_size, 430)
۱۹        # graph_data: (batch_size, embedding_dim)
۲۰        # seq_data: (batch_size, seq_len, embedding_dim)
۲۱        # Transform different data types to embedding vectors
۲۲        tab_emb = torch.relu(self.tab_to_emb(tab_data))
۲۳        graph_emb = torch.relu(self.graph_to_emb(graph_data))
۲۴
۲۵        # Process sequential data with LSTM
۲۶        lstm_out, (hn, cn) = self.sequence_processor(seq_data)
۲۷        # Use the last output vector from LSTM for each sample in batch
۲۸        seq_emb = lstm_out[:, -1, :]
۲۹        # Concatenate embedding vectors
۳۰        combined_embeddings = torch.cat((tab_emb, graph_emb, seq_emb), dim=-1)
۳۱        # Apply fusion layer and activation
۳۲        fused_representation = self.fusion_layer(combined_embeddings)
۳۳        fused_representation = torch.relu(fused_representation)
۳۴        fused_representation = self.dropout_layer(fused_representation)
۳۵        # Final classification
```

```

۳۶         output = torch.sigmoid(self.classifier(fused_representation))
۳۷         return output

```

۲-۱- کد بهینه‌سازی با PIRATES

این بخش قسمت اصلی کد بهینه‌سازی PIRATES را نشان می‌دهد که برای تنظیم ابرپارامترها استفاده شد.

PIRATES با بهینه‌سازی کد

```

۱ import numpy as np
۲
۳ class Pirates():
۴     def __init__(self, func, fmax=(), fmin=(), hr=0.2, ms=3, max_r=1,
۵                 num_ships=5, dimensions=2, max_iter=10, max_wind=1, c={},
۶                 top_ships=10, sailing_radius=0.3, plundering_radius=0.1):
۷         # Main algorithm parameters
۸         self.num_ships = num_ships
۹         self.num_top_ships = top_ships
۱0        self.max_iter = max_iter
۱۱        # Objective function parameters
۱۲        self.func_obj = func
۱۳        self.cost_func = self.func_obj.func
۱۴        self.fmin = fmin
۱۵        self.fmax = fmax
۱۶        self.dimensions = dimensions
۱۷        # Weight parameters
۱۸        default_c = {
۱۹            'leader': 0.5,
۲۰            'private_map': 0.5,
۲۱            'map': 0.5,
۲۲            'top_ships': 0.5
۲۳        }
۲۴        self.c = {**default_c, **c}
۲۵        # Movement parameters
۲۶        self.sailing_radius = sailing_radius
۲۷        self.plundering_radius = plundering_radius
۲۸        # Leader and map variables
۲۹        self.leader_index = None
۳۰        self.hr = 1 - hr
۳۱        self.r = None
۳۲        self.max_r = max_r
۳۳        self.ms = ms
۳۴        self.map = None
۳۵        # Problem type

```

```

۳۶     self.problem = 'min'
۳۷     # Chart variables
۳۸     self.bsf_position = None
۳۹     self.bsf_list = []
۴۰     # Initialization
۴۱     self.random_init()
۴۲     self.iter = 0
۴۳     def search(self):
۴۴         """
۴۵         Run optimization algorithm and return best results
۴۶         Returns:
۴۷         -----
۴۸         tuple
۴۹         (best position, best cost, best metrics)
۵۰         """
۵۱         # Run algorithm
۵۲         self.start()
۵۳         # Get results
۵۴         result = self.cal_costs()
۵۵         if result is not None:
۵۶             best_cost, best_metrics = result
۵۷         else:
۵۸             best_cost = self.costs[self.leader_index]
۵۹             best_metrics = {'f1': 0.0, 'accuracy': 0.0,
۶۰                             'precision': 0.0, 'recall': 0.0}
۶۱         return self.ships[self.leader_index], best_cost, best_metrics

```

۲- پیوست B: داده‌های خام و پیش‌پردازش

۱-۲- نمونه داده‌های خام DREBIN

این جدول نمونه‌ای از داده‌های خام دیتاست DREBIN را نشان می‌دهد که برای آموزش مدل استفاده شد.

جدول ۱ نمونه‌ای از داده‌های خام دیتاست DREBIN

شناسه نمونه	تعداد مجوزها	فراخوانی‌های API	برچسب
۰۰۱	۱۵	["read_contacts", "send_sms"]	۱
۰۰۲	۸	["get_accounts"]	۰
۰۰۳	۱۲	["read_phone_state", "write_external_storage"]	۱

۲-۲- توضیحات پیش‌پردازش

داده‌ها پیش‌پردازش شدند تا برای مدل مناسب شوند:

- تنظیم ابعاد ویژگی‌ها از (۱۶، ۳۲) به (۱۶، ۴۳۰)

- نرمال‌سازی با استفاده از استانداردسازی z-score

- تبدیل داده‌های متنی به بردارهای باینری

۳- پیوست C: جزئیات سخت‌افزاری و نرم‌افزاری

۱-۳- مشخصات سخت‌افزاری

آزمایش‌ها با استفاده از زیرساخت زیر اجرا شدند:

- GPU: NVIDIA RTX 3090 با ۲۴ گیگابایت VRAM

- CPU: Intel Xeon E5-2690 v4 با ۳۲ هسته

- RAM: ۱۲۸ گیگابایت

۲-۳- مشخصات نرم‌افزاری

محیط نرم‌افزاری شامل موارد زیر بود:

- زبان برنامه‌نویسی: Python 3.8.5

- کتابخانه‌ها:

- PyTorch 1.9.0

- PyTorch Geometric 1.7.0

- Optuna 2.10.0

- سیستم‌عامل: Ubuntu 20.04 LTS

۴- پیوست D: نتایج اضافی و ماتریس‌های کامل

۱-۴- ماتریس درهم‌ریختگی کامل

این جدول ماتریس درهم‌ریختگی را برای مجموعه تست با ۱، ۴۵۱ نمونه نشان می‌دهد.

جدول ۲ ماتریس درهم‌ریختگی برای مجموعه تست

پیش‌بینی/واقعیت	کلاس ۰	کلاس ۱
کلاس ۰	304 (TN)	23 (FP)
کلاس ۱	17 (FN)	1107 (TP)

۲-۴ گزارش طبقه‌بندی برای هر دسته

این جدول نتایج هر دسته در اعتبارسنجی متقاطع ۵-تایی را نشان می‌دهد.

جدول ۳ گزارش طبقه‌بندی برای هر دسته در اعتبارسنجی متقاطع

دسته	F1 Score	دقت	AUC	زیان
دسته ۱	0.9858	0.9785	0.9950	0.0786
دسته ۲	0.9846	0.9763	0.9955	0.0735
دسته ۳	0.9839	0.9752	0.9945	0.0839
دسته ۴	0.9742	0.9601	0.9861	0.1199
دسته ۵	0.9808	0.9709	0.9946	0.0864

Abstract

Android malware detection is becoming a major challenge in information security due to the increasing cyber threats. Traditional methods, especially those relying solely on single-modal feature analysis, often face limitations such as the inability to process complex multi-modal data and poor generalization to new threats. These shortcomings highlight the need for developing novel and efficient approaches. This research proposes a multi-modal model called Multi-modal Attention-based Graph Neural Transformer with Dynamic Embedding (MAGNET), which leverages a combination of tabular, graph, and sequential data—such as API call sequences—for Android malware detection. The primary objective is to improve detection accuracy and robustness by utilizing an advanced architecture based on deep learning and transformers. The methodology includes hyperparameter optimization using advanced algorithms such as PIRATES and Optuna, along with model training on a dataset consisting of 4641 training samples and 1451 test samples, supported by 5-fold cross-validation. The model incorporates static features such as permissions, API calls, intents, and component names, as well as dynamic features like network activity and file access. Data is represented as binary or normalized numerical vectors, and the feature space is reduced to 430 dimensions after preprocessing. Tools used in this study include deep learning libraries such as PyTorch, data preprocessing techniques like standardization and normalization, and graph data structures. The raw data consists of actual Android application behavior, encompassing both static and dynamic attributes. Results indicate that the proposed model delivers outstanding performance with high accuracy, notable stability, and strong generalization, offering significant improvements over previous methods. These outcomes underscore the model's potential for deployment in real-world security systems.

Keywords: Malware Detection, Transformer, Deep Learning, Multi-modal Data, Android Security.



Shahid Bahonar University of Kerman

Faculty of Engineering

Department of Computer Engineering

**Robust Android Malware Detection using Transformer Neural
Networks**

Prepared by:

Alireza Iranmanesh

Supervisor:

Dr. Hamid Mirvaziri

**A Thesis Submitted as a Partial Fulfillment of the Requirements for the
Degree of Master of Science in Computer Engineering (M. Sc.)**

June 2025